

UNIVERSITY OF ALEPPO

FACULTY OF INFORMATICS ENGINEERING

DEPARTMENT OF COMPUTER NETWORKS

BACHELOR GRADUATION PROJECT

Residential Smart Grid Prototype

Students:

Ez Aldin Waez

Abdullah Naal

Mohammad Labaniah

Abdo Kialy

Ruby Abbassy

Supervisor:

Dr. Fadi Farha

Academic Year:

2024-2025

*This project is submitted in fulfillment of one of the requirements
for the degree of Bachelor of Science in Informatics Engineering*

September 2025

DEDICATION

*To my parents, who believed in me when I didn't believe in myself.
To my siblings, who always pushed me to be better.
To my friends, who made my journey enjoyable and memorable.*

Ez Aldin Waez

*To my parents, for providing a stable foundation from which to explore.
To my classmates, for the productive friction that led to growth.
To fellow scientists, for building upon the work of those who came before.*

Abdullah Naal

*To my family, the source of my strength and motivation.
To my teachers, who shaped my understanding of the world.*

Mohammad Labaniah

*To my family, whose support has been my anchor.
To my colleagues, who made this collaborative work possible.*

Abdo Kialy

*To everyone who supported and stood by me.
To those who loved me and shared my joys and hardships.
To those who believed in me in moments of weakness.
To those who gave me hope and lit my path with their smiles.*

Ruby Abbassy

ABSTRACT

Residential solar energy systems create distribution inefficiencies where individual houses generate excess energy while others lack energy, resulting in wasted resources and increased grid dependence despite available energy. The Residential Smart Grid Prototype (RSGP) addresses this problem by implementing an intelligent power management solution that optimizes energy allocation among houses with varying consumption patterns. The system employs a distributed simulation architecture consisting of houses simulation using Attack-Decay-Sustain-Release envelope modeling for realistic device behavior, solar system simulation integrating National Solar Radiation Database data with pvlib and pvwatts libraries, and power management implementing a statistical learning algorithm for virtual battery allocation based on consumption patterns. The power management algorithm partitions shared physical battery among individual houses through a virtual battery system with dynamically adjusted allocation weights, employing normalized load deviation calculations and fairness constraints to ensure fair energy distribution while minimizing grid dependence. Implementation includes real-time monitoring interfaces through tkinter-based dashboard systems and Raspberry Pi GPIO controllers enabling physical interaction through button controls and LED feedback. Results of validation demonstrate correlation between ADSR device modeling and published consumption data, with system performance metrics indicating reduced utility grid dependence.

Keywords: Smart grid, residential energy management, virtual battery systems, solar power distribution, adaptive learning algorithms, power management, distributed simulation, ADSR modeling, energy optimization

CONTENTS

1	Introduction	1
2	Project Preview	3
2.1	System Architecture Overview	3
2.2	Core Simulation Components	4
2.3	System Integration and Communication	4
2.4	Technology Stack and Development Tools	5
2.5	Conclusion and Future Development Potential	7
3	Houses Simulation	9
3.1	Introduction	9
3.2	Architecture Overview	9
3.3	The Houses Simulator	10
3.4	Individual House Modeling	11
3.5	Device Modeling Framework	11
3.6	Integration with Power Management	14
3.7	Data Collection	15
4	Solar System Simulation	17
4.1	Introduction	17
4.2	Solar System Architecture Overview	17
4.3	Photovoltaic Panel Modeling	18
4.4	Battery Energy Storage System	21
4.5	Inverter Control System	22
4.6	Weather Data Integration	24
4.7	Power Management Interface	25
4.8	Data Collection	25
5	Power Management	29
5.1	Introduction	29

5.2	System Architecture	29
5.3	Virtual Battery System	30
5.4	Adaptive Learning Algorithm	32
5.5	Power Distribution Algorithms	35
5.6	System-Wide Load Line Coordination	38
5.7	Utility Power Export Distribution	39
5.8	System Integration and Real-Time Control	40
5.9	Performance Monitoring and Data Collection	42
6	Dashboard	45
6.1	Introduction	45
6.2	Remote Object Architecture	45
6.3	Dashboard Application Framework	46
6.4	Houses Simulation Interface	47
6.5	Summary and Monitoring Views	47
6.6	Real-time Update Architecture	47
6.7	Data Collection and Analysis Support	47
7	Raspberry Pi Controller	49
7.1	Introduction	49
7.2	System Architecture Overview	49
7.3	Hardware Configuration and GPIO Mapping	50
7.4	GPIOController Implementation	50
7.5	Remote Communication Interface	52
7.6	Real-Time Control Loop Implementation	53
7.7	Hardware Integration and Deployment	53
8	Final Results	57
8.1	System Design Validation	57
8.2	Simulation Results and Analysis	60
9	Conclusions and Suggestions	61
9.1	Conclusions	61
9.2	Suggestions	61
	Thanks and Appreciation	63
	References	65
A	Documentation Workflow	67
A.1	Introduction	67
A.2	Collaborative Development Approach	67
A.3	Documentation Development Phases	68
A.4	Sphinx	69
A.5	Automated API Reference Generation	70
A.6	Matplotlib Plot Integration	70
B	Notebooks Workflow	73
B.1	Introduction	73

B.2	marimo Framework Overview	73
B.3	Notebook Applications	74
B.4	Data Integration and Workflow	78
C	API Reference	79
C.1	RSGP	79
C.2	Other Packages	107

LIST OF FIGURES

2.1	RSGP system architecture overview	4
2.2	RSGP system component interconnection diagram	5
3.1	Houses simulation architecture	10
3.2	Houses simulator coordination and load aggregation workflow	11
3.3	ADSR envelope showing Attack-Decay-Sustain-Release phases with device state transitions	12
3.4	ADSR parameter impact comparison	13
3.5	Some regular devices simulated load profiles	14
3.6	Real-time houses simulation demonstration	15
4.1	Solar system simulation architecture	18
4.2	Photovoltaic panel modeling workflow and data processing pipeline	19
4.3	Components of solar radiation (Source: ResearchGate)	20
4.4	Solar irradiance components over a typical clear day	20
4.5	Solar zenith angle (Source: ScienceDirect Topics)	21
4.6	DC-to-AC conversion characteristics	22
4.7	Inverter operational modes	24
4.8	Snapshot of the NSRDB dataset used	25
4.9	Real-time solar system simulation demonstration	26
5.1	Power management architecture	30
5.2	Virtual battery weight evolution and capacity allocation	32
5.3	Learning algorithm convergence analysis	34
5.4	Weight variance and convergence metrics analysis	35
5.5	Power distribution waterfall visualization	36
5.6	Virtual battery state-of-charge tracking	38
5.7	Utility export distribution fairness demonstration	40
5.8	Power manager real-time update cycle	41
6.1	Dashboard snapshot	46

6.2	Dashboard component layout	46
7.1	GPIO controller state machine	51
7.2	Raspberry Pi used in the project	53
7.3	Hardware parts required by the system	54
7.4	Raspberry Pi 5 GPIO pinout diagram	55
8.1	RSGP ADSR refrigerator model validation against real power consumption data	58
8.2	RSGP solar system simulation validation against real SolarMax inverter data . .	59
2.1	Graph editor notebook interface	74
2.2	NSRDB visualization notebook interface	75
2.3	Inverter logs visualization notebook interface	76
2.4	RSGP logs visualization notebook interface	77

LIST OF TABLES

7.1	GPIO pin mappings for hardware controls	54
-----	---	----

CHAPTER

1

INTRODUCTION

Solar energy adoption in residential areas has created a significant inefficiency: houses with solar panels often generate more energy than they need while neighboring houses lack sufficient power. This excess energy typically goes to waste or gets sold back to the grid at low rates, while houses needing energy must purchase it from the utility at higher costs. The problem stems from the individual nature of current residential solar installations, where each house operates independently without coordination with nearby houses.

This inefficiency motivated the development of the Residential Smart Grid Prototype (RSGP). Neighborhoods could achieve better energy utilization and reduced grid dependence by intelligently sharing energy resources among houses. Instead of wasting excess solar generation, RSGP enables houses to share their surplus with neighbors who need it, creating a collaborative energy ecosystem at the residential level.

RSGP solves this problem through an intelligent power management system that coordinates energy distribution among multiple houses sharing a common battery. The system uses virtual battery allocation, where each house receives a portion of the shared physical battery based on its consumption patterns. A learning algorithm continuously adjusts these allocations by analyzing each house's energy usage, ensuring fair distribution while maximizing the benefits of shared solar generation. Houses that consistently use less energy receive smaller battery allocations but greater utility export benefits, while houses with higher demands get larger shares but less utility export benefits, creating a dynamic and fair system.

The project implements this solution through a comprehensive simulation framework that models realistic house energy consumption and solar power generation, and implements intelligent power management algorithms. This simulation environment allows testing and validating before real-world deployment, ensuring the system works effectively under various scenarios and conditions.

This document presents a comprehensive examination of the Residential Smart Grid Prototype through nine core chapters and three supporting appendices. Chapter 2 provides a detailed

project preview that establishes system architecture and technology stack. The preview chapter presents the overall vision and scope of the project.

Chapters 3 through 5 constitute the technical core of the documentation, examining each simulation component in comprehensive detail. Chapter 3 explores houses simulation architecture, ADSR envelope modeling, and integration with power management. Chapter 4 presents solar system simulation implementation, including photovoltaic modeling using established libraries, battery system design, inverter control algorithms, and weather data integration from authoritative sources. Chapter 5 examines power management system architecture, virtual battery mathematical foundation, and the adaptive learning algorithm.

Chapters 6 and 7 address interactive user interfaces implementations. Chapter 6 details dashboard system architecture, remote object communication, and real-time monitoring. Chapter 7 presents hardware integration through Raspberry Pi, GPIO mapping, and physical interaction mechanisms.

Chapter 8 presents results from simulation validation, performance analysis, and system effectiveness evaluation. The results demonstrate correlation between simulation models and real-world data and quantifies system performance improvements in grid independence.

Chapter 9 concludes with analysis of project outcomes, identification of future development opportunities, and recommendations for continued research and real-world deployment.

The appendices provide essential supporting documentation for project understanding and continuation. Appendix A details documentation workflow, including mathematical notation standards and diagram creation methodologies that ensure consistency. Appendix B presents the notebook-based analysis tool and data visualization framework that supports ongoing research and system optimization efforts. Appendix C provides API reference documentation that enables future development and integration with external systems.

CHAPTER

2

PROJECT PREVIEW

The Residential Smart Grid Prototype (RSGP) aims to solve the following problem: in residential buildings where each house has its own solar system and battery, some houses have excess generated energy where others lack energy. That excess could have gone to the houses lacking energy at no cost of the houses with excess. As such, RSGP comes in to provide a smart solution that tracks the houses' needs and consumption patterns to allocate to each house the amount it needs.

The current status of RSGP is a prototype operating on a simulation of houses and a solar system with one shared battery, with a power management algorithm that allocates and alters virtual batteries allocated to the houses.

2.1 System Architecture Overview

RSGP system consists of three core simulation modules, supporting infrastructure, and analysis tools that collectively provide a foundation for analysis to improve RSGP and validate its algorithms before it is applied in the real world.

The time simulation engine provides synchronized timestep coordination across all system components, ensuring that houses simulation, solar system simulation, and power management operate consistently. The configurable time acceleration factor enables for simulating the results of applying RSGP over larger periods of time to facilitate studying and analyzing the results before applying RSGP to the real world.

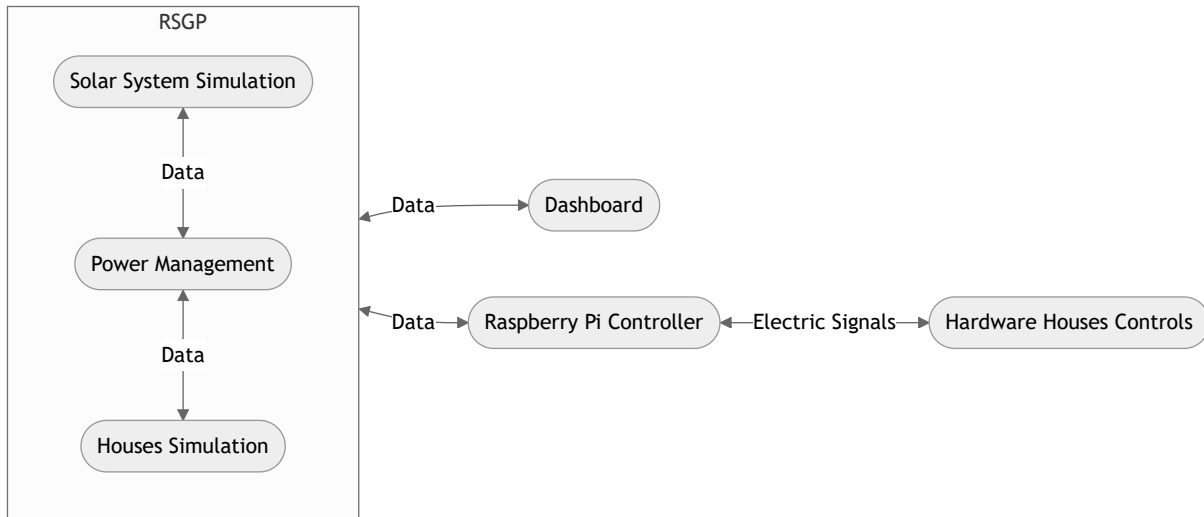


Fig. 2.1: RSGP system architecture overview

2.2 Core Simulation Components

The core components are the houses simulation, solar system simulation, and power management.

- **Houses Simulation** consists of a collection of simulated houses, each containing simulated devices that can be activated and deactivated per user input or using scripts. The ability to script activation and deactivation of devices allow for scripting scenarios and running them to collect data and use it to analyze both the simulation and the power management algorithm.
- **Solar System Simulation** consists of simulated solar panels, battery, and inverter. This simulation uses a dataset provided for research purposes from the National Solar Radiation Database (NSRDB). As for the rest of the components, their simulation use the standard libraries pvlib and pvwatts.
- **Power Management** consists of virtual batteries and a power manager that serves as the decision-making unit and connector between all the other components. The smart algorithm implemented is based on statistics to assign a weight to each house that represents its share of the shared battery. The algorithm implements a minimum weight and a maximum weight to ensure no house is left without an adequate share; so that only the excess is shared.

All components log data into csv files for later analysis to improve the simulation and the algorithms implemented.

2.3 System Integration and Communication

The system uses Pyro5 remote objects to provide access to component methods and properties while maintaining network transparency.

The primary data exchange patterns include:

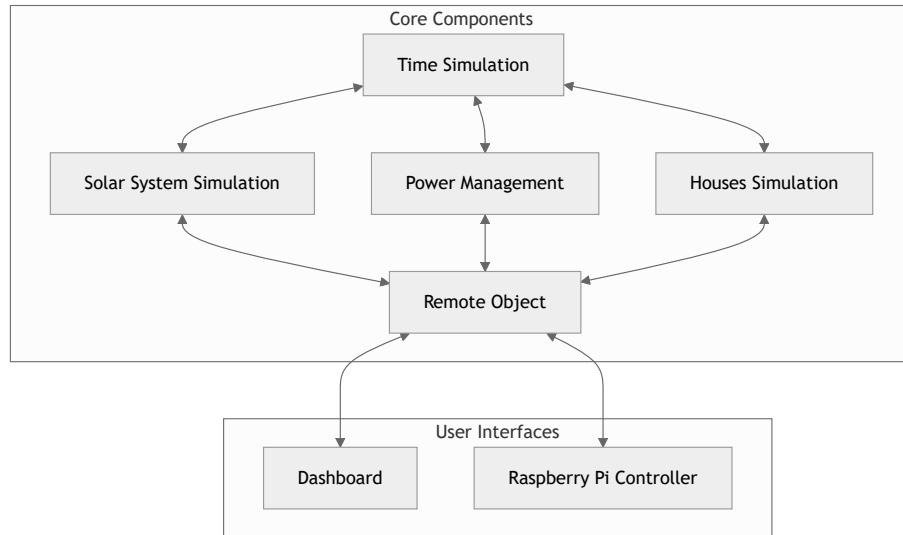


Fig. 2.2: RSGP system component interconnection diagram

Power Generation Data: Solar system simulation provides power generation, battery state information, and inverter operational status to power management for decision-making.

Load Demand Data: Houses simulation reports individual house loads, device states, and line connection status to power management for decision-making.

Control Commands: Power management issues load shedding commands, utility line controls, and battery discharge instructions based on the decisions of the algorithm.

2.4 Technology Stack and Development Tools

The system foundation uses **Python** as the primary programming language, providing access to extensive scientific computing libraries.

- **NumPy** serves as the foundation for mathematical operations, enabling efficient array operations essential for power management calculations.
- **Pandas** provides data manipulation, particularly valuable for processing NSRDB datasets and managing simulation logs.
- **PVLib** provides photovoltaic system modeling, including solar position calculations and irradiance computations.
- **PVWatts** inverter models from the National Renewable Energy Laboratory (NREL) provide realistic DC-AC conversion efficiency curves and power electronic behavior modeling.
- **NSRDB** (National Solar Radiation Database) serves as the primary data source, providing hourly solar irradiance measurements required for realistic solar system modeling.
- **timezonefinder** provides geographic timezone detection.
- **Pyro5** (Python Remote Objects) enables the distributed simulation architecture by providing remote method invocation.

- **Tkinter** provides the graphical user interfaces.
- **ttkbootstrap** extends the interface with modern themes and enhanced widget styling.
- **Raspberry Pi** serves as the embedded computing platform for hardware integration.
- **lgpio** library provides modern GPIO control with improved performance and reliability compared to legacy alternatives.

Physical prototyping utilizes standard **breadboards**, **wires**, **LEDs**, **resistors**, and **toggle buttons** to create tangible representations of simulated devices and system states.

2.4.1 Data Analysis and Notebook Environment

- **Marimo** provides notebooks for interactive data analysis and system exploration.
- **Altair** extends the visualization through statistical plotting and interactive charting features.
- **Matplotlib** generates visualizations for system analysis.
- **Mermaid** creates system architecture diagrams and workflow visualizations integrated directly into documentation.
- **WebPlotDigitizer** extracts data from plots and images, useful for comparing simulation results with published figures.

2.4.2 Documentation

- **Sphinx** generates documentation with support for mathematical notation, code highlighting, and multi-format output generation. The system produces both HTML and PDF documentation.
- **LaTeX** integration enables mathematical notation and document formatting.
- **sphinxcontrib-mermaid** integrates diagram generation directly into the documentation.

2.4.3 Development and Deployment Tools

- **Git** version control manages collaborative development and maintains complete project history.
- **Make** automates builds and maintains the consistency of the process across each development environment.
- **pip** package management handles dependency resolution and virtual environment management.
- **python-dotenv** provides environment variable management for configuration settings for deployment flexibility without code modification.

2.5 Conclusion and Future Development Potential

The system serves as both a practical solution to a real problem and a research tool that collects data and visualizes it to facilitate research and development of its algorithms. Through its usage of an extensive list of tools and libraries, RSGP aims to utilize the full range of available resources to tackle the problem at hand: that of efficient distribution of generated solar power.

CHAPTER

3

HOUSES SIMULATION

3.1 Introduction

The houses simulation serves as the primary load generator within the system. This chapter examines the implementation of the residential home model, the device modeling framework, and the coordination mechanisms between the components of the house simulation.

The significance of the houses simulation extends beyond simple load modeling. It provides the testing environment to evaluate power management algorithms under varying consumption scenarios. The simulation tracks N houses implementing the same model. Each house contains multiple devices with different consumption patterns. This aims to mirror real world residential demand.

3.2 Architecture Overview

The architecture separates simulation coordination, individual house modeling, and device-level power consumption calculations.

- The `HousesSimulator` serves as the coordination layer; it manages the house instances and coordinates their execution.
- The `House` serves as the individual unit; it maintains state information about its devices and utility and load lines.
- The `DeviceClass` serves as the base for the loads; it does the calculations per each device's parameters to simulate realistic load consumption patterns.

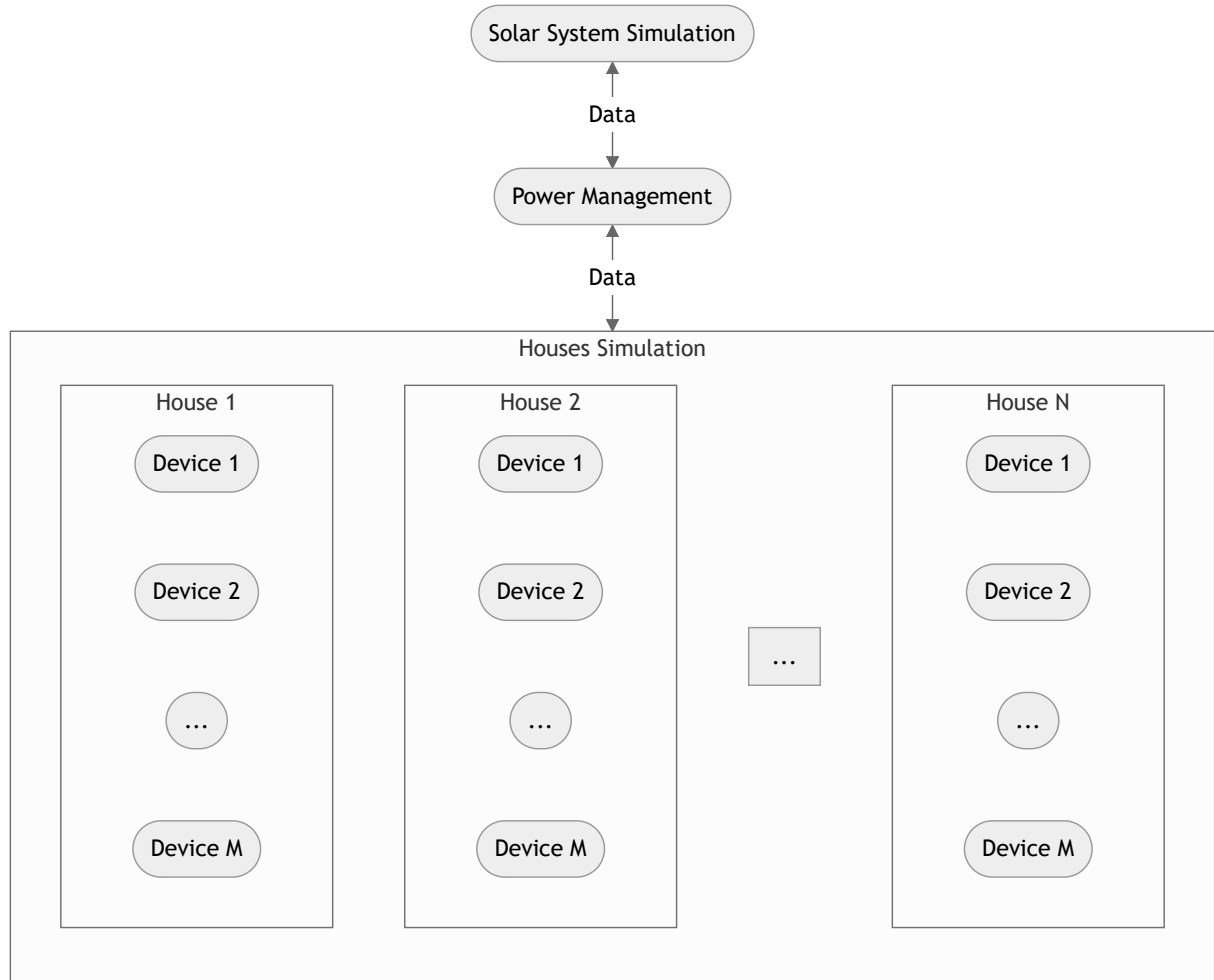


Fig. 3.1: Houses simulation architecture

Note

The design prioritizes modularity and allows for dynamic addition of new device types without modification to the core simulation. This makes it easier to craft testing scenarios.

3.3 The Houses Simulator

The `HousesSimulator` coordinates the execution of individual house instances; it aggregates system-wide load calculations and provides the interface between the residential demand and the power management system. The simulator operates on its own thread to ensure that house load calculations proceed independently of the other components. The system-wide load aggregation follows:

$$L_{system} = \sum_{i=1}^N L_i \times LL_i$$

where N represents the number of the houses, L_i the load of the i -th house, and LL_i the binary connection state for the i -th house.

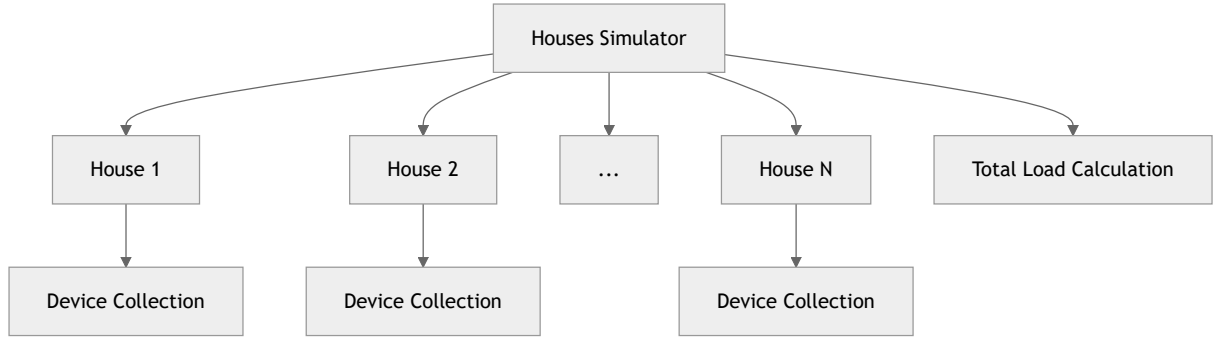


Fig. 3.2: Houses simulator coordination and load aggregation workflow

The simulator implements a configurable update cycle usually set to 100-millisecond intervals. During each update cycle, the simulator queries all house instances for their current power consumption and aggregates these values. The system-wide total load is then made available to the power management system.

Warning

Load line disconnection affects only the connectivity between houses and the power providers; internal house simulations continue to maintain realistic behavior on reconnection.

3.4 Individual House Modeling

The House model maintains state information about connectivity and devices. Multiple device instances coexist and contribute to the overall house load. Each house contains a predefined set of device types, which include heating and cooling systems, lighting, major appliances, and miscellaneous electrical loads.

The model aggregates individual device loads and tracks its energy exchange with the utility. This supports both scenarios where the house exports excess renewable energy or imports power when that renewable energy is insufficient. The house load aggregation follows:

$$L_{house} = \sum_{d=1}^{N_{devices}} L_{device,d}$$

where utility exchange power is calculated as the difference between available renewable energy and house demand, with positive values indicating export and negative values indicating import.

3.5 Device Modeling Framework

The Device modeling framework implements a mathematical model that captures the dynamic behavior of residential electrical appliances. This framework borrows the **Attack-Decay-Sustain-Release envelope (ADSR)** from sound synthesis and uses it along wave modulation techniques to approximate the load profiles. These techniques generate realistic power consumption patterns that mimic real electrical devices.

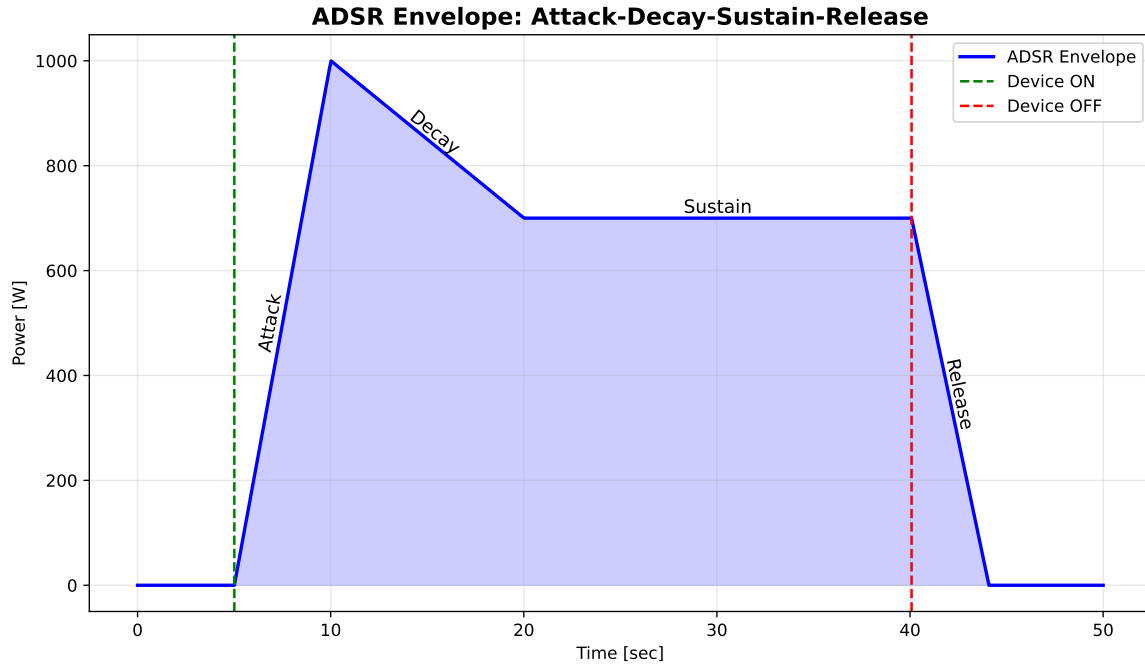


Fig. 3.3: ADSR envelope showing Attack-Decay-Sustain-Release phases with device state transitions

The **ADSR envelope** provides the temporal structure for the device's power consumption. It models the startup spike, steady-state operation, and shutdown characteristics of the device. The mathematical representation of the envelope uses piecewise linear functions with configurable timing parameters:

- The **Attack** phase models the initial power spike that occurs when the device activates. This typically involves inrush currents for motor-driven appliances or heating element activation for thermal devices. The mathematical representation follows: $P(t) = \frac{1}{a} \times t$ for $0 \leq t \leq a$.
- The **Decay** phase models the transition from that startup spike to normal operation as the device's components stabilize. The decay function implements: $P(t) = \frac{s-1}{d} \times t + 1 - a \times \frac{s-1}{d}$ for $a < t \leq a + d$.
- The **Sustain** phase models the steady-state operation where the device maintains its power consumption level. This phase also uses wave modulation to model load variations due to changing conditions. The sustain level maintains: $P(t) = s$ for $t > a + d$.
- The **Release** phase models the shutdown. This includes braking effects in motor systems and thermal cool-down periods that continue to consume power for a short while until complete shutdown. The release function follows: $P(t) = -\frac{s}{r} \times t + P_{toggle}$ where P_{toggle} represents the power level at device deactivation.

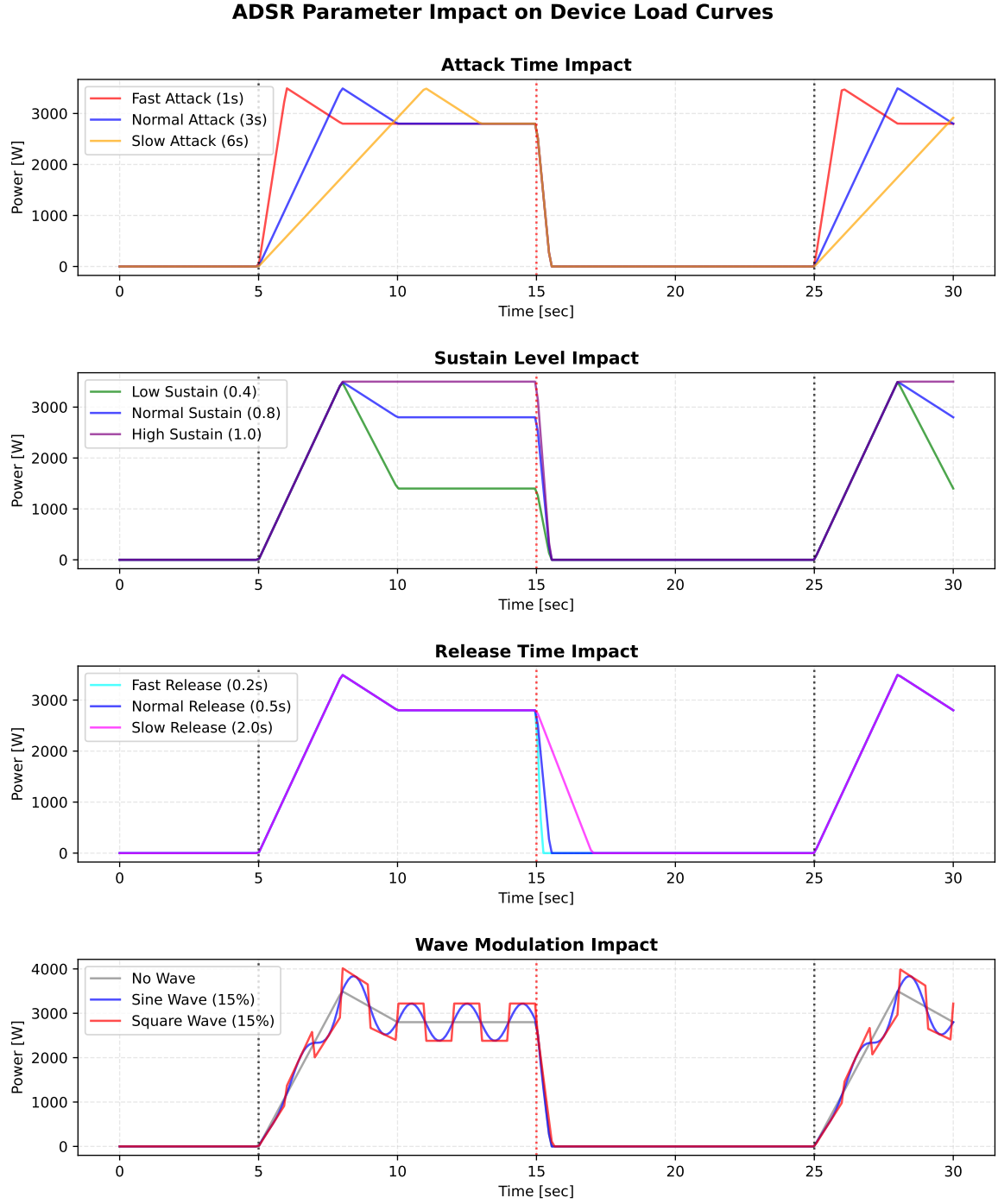


Fig. 3.4: ADSR parameter impact comparison

Wave modulation introduces realistic variations in power consumption over the ADSR envelope. The framework supports multiple modulation types: sine waves, square waves, and random modulation. These modulation patterns operate at different frequencies and amplitudes to create complex power signatures that more closely match measured residential devices' behavior.

The wave modulation mathematical functions include:

- **Sine wave modulation:** $W(t) = 1 + w_a \times \sin\left(\frac{2\pi t}{w_p}\right)$

- **Square wave modulation:** $W(t) = 1 + w_a \times \text{sgn} \left(\sin \left(\frac{2\pi t}{w_p} \right) \right)$
- **Random modulation:** $W(t) = 1 + w_a \times \mathcal{U}(-1, 1)$

where w_a represents wave amplitude, w_p represents wave period, and $\mathcal{U}(-1, 1)$ represents uniform random distribution.

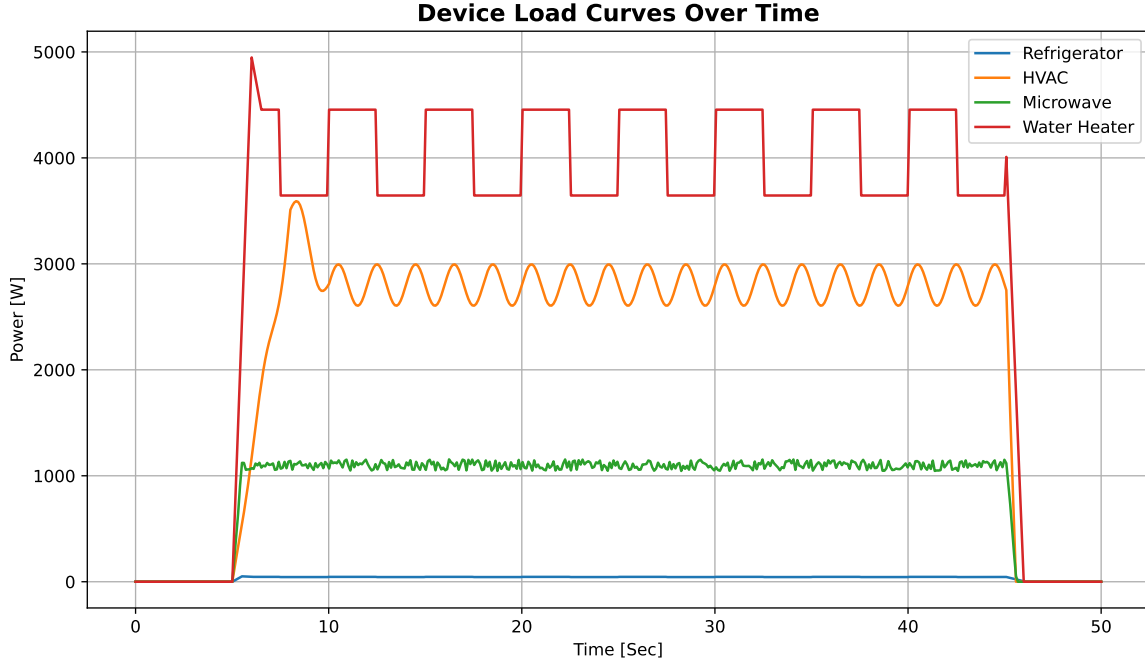


Fig. 3.5: Some regular devices simulated load profiles

The device modeling framework organizes electrical devices into categories. Each category implements specialized ADSR parameters and wave modulation to capture the unique behavior of the device. The total device class load calculation combines all active device instances:

$$L_{total} = \sum_{i=1}^{N_{active}} P_{base} \times W(t) \times ADSR_i(t)$$

where P_{base} represents the base wattage, $W(t)$ represents the wave modulation function, and $ADSR_i(t)$ represents the envelope multiplier for device instance i .

3.6 Integration with Power Management

The `HousesSimulator` provides the aggregated system load to the `PowerManager`, along the specific load for each house in the simulation. The `PowerManager` then does its calculation and may communicate back to the simulation, connecting or disconnecting some of the houses' load or utility lines. The `HouseSimulator` thus offers an interface for the `PowerManager` to read the data it needs and modify the simulation per its judgment.

Note

The interface maintains loose coupling between the `HouseSimulator` and the `PowerManager` to enable independent development and testing.

3.7 Data Collection

The houses simulation captures device operation statistics and system performance metrics for later analysis. The data is stored as a time-series dataset (in a CSV file) to enable examination of the effectiveness of power management solutions.

CSV logging operates in the update cycle (usually once every 100-milliseconds). It records total house loads, individual device loads, connectivity states, and utility exchange values for each house throughout the simulation. The goal is to track the utility exchange power and minimize it through an effective power management solution.

Houses Simulation: Real-time Device Control Scenario

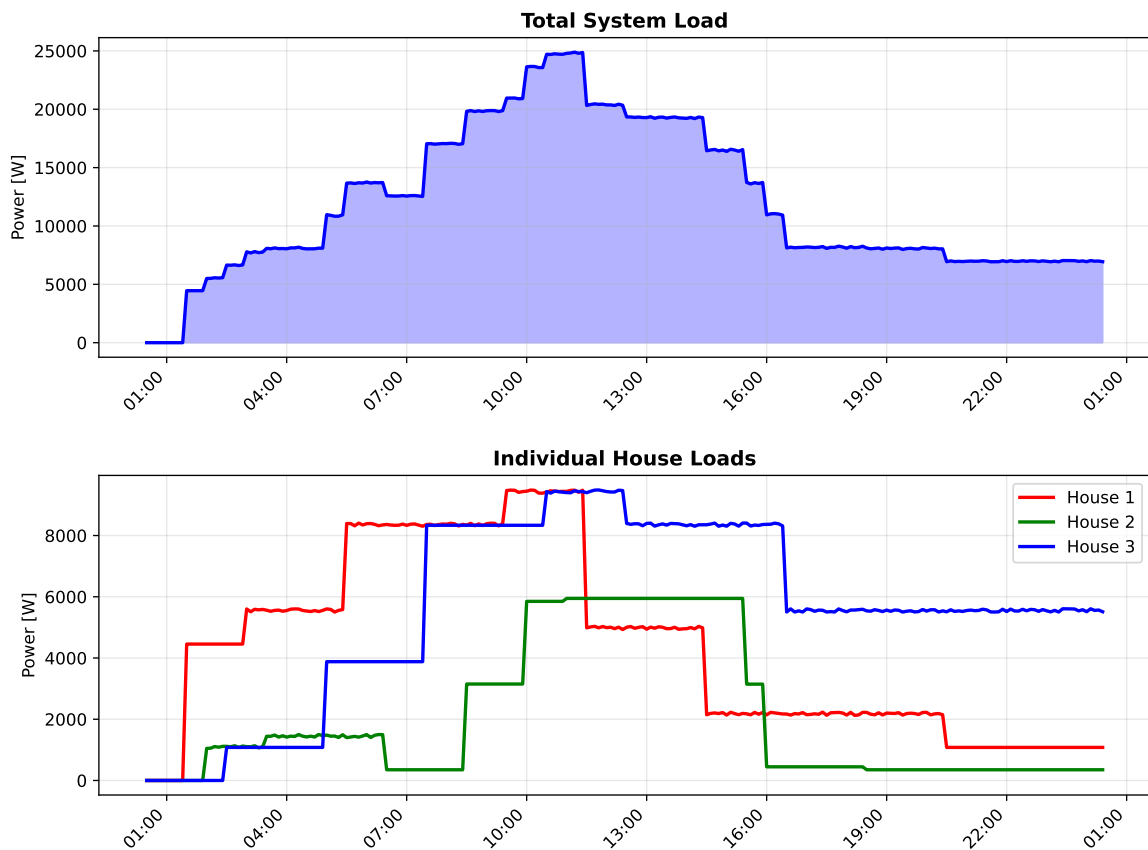


Fig. 3.6: Real-time houses simulation demonstration

CHAPTER

4

SOLAR SYSTEM SIMULATION

4.1 Introduction

The solar system simulation serves as the source of power within the system. This chapter examines the implementation of photovoltaic panel modeling, battery energy storage systems, and inverter control mechanisms that enable realistic solar power generation and management.

While the houses simulation models energy consumption, the solar system simulation models energy production and storage. Together they form the fundamental supply-demand dynamic that defines smart grids. The power management system, discussed in later chapters, coordinates between these two to optimize energy usage.

4.2 Solar System Architecture Overview

The solar system simulation architecture consists of photovoltaic panels, one battery, and an inverter connecting the elements. The inverter is then connected to the power manager through a load line and a utility line. This design separates concerns between energy generation, storage, and power conversion and enables more realistic modeling of each component.

The **Panels** handle solar irradiance calculations and power generation based on the data collected from the National Solar Radiation Database (NSRDB). The **Battery** manages charge state tracking. The **Inverter** adheres to its operational mode control and manages power distribution from and to the solar system components and the demands of the power manager.

Note

The three-component architecture enables independent development and testing of solar generation, energy storage, and power distribution.

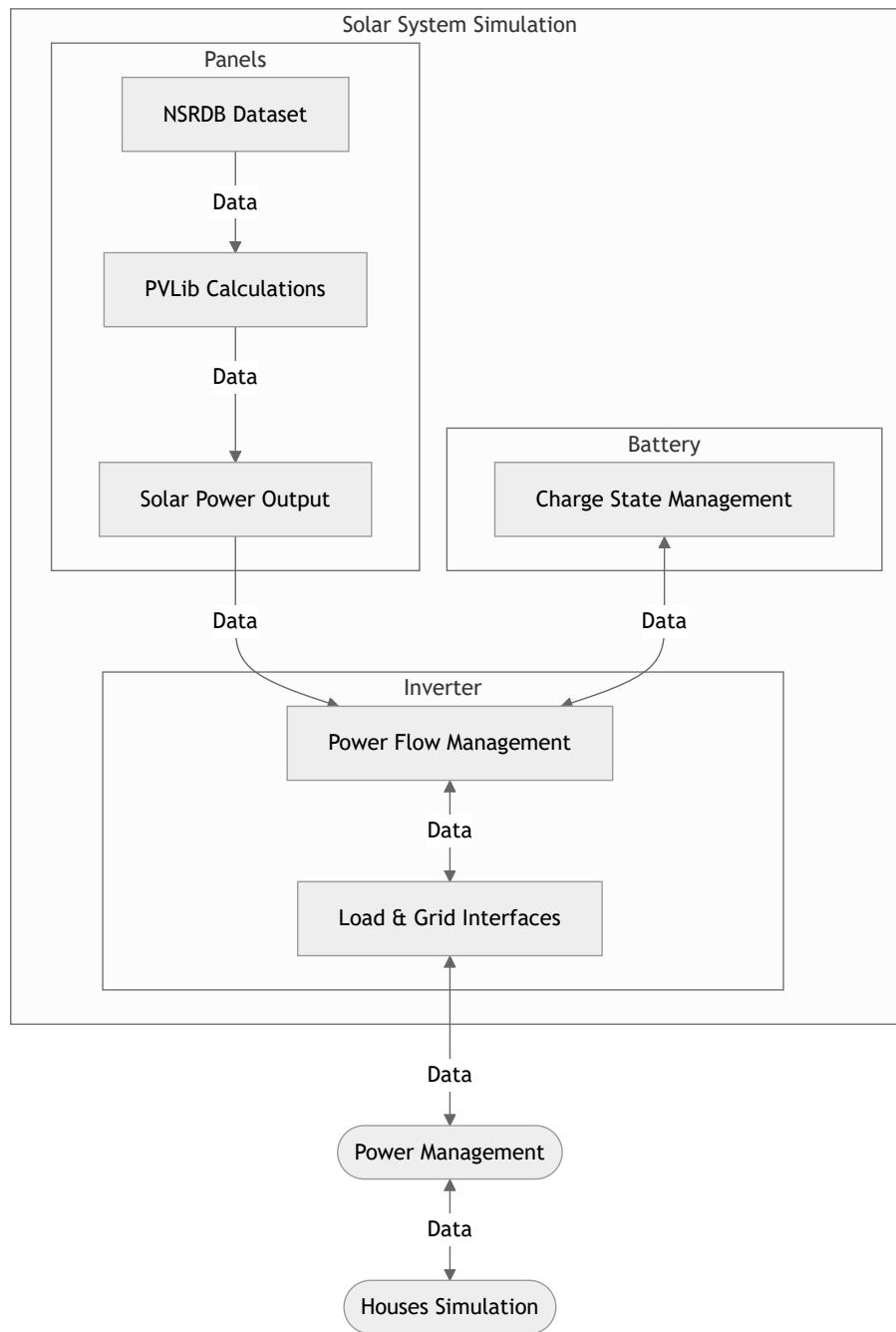


Fig. 4.1: Solar system simulation architecture

4.3 Photovoltaic Panel Modeling

The photovoltaic panels integrate with the National Solar Radiation Database to obtain weather information and employs the PVLlib photovoltaic modeling library to perform accurate solar calculations. The panel model accounts for solar position, atmospheric conditions, and system configuration parameters to generate power output that mirror actual photovoltaic system performance.

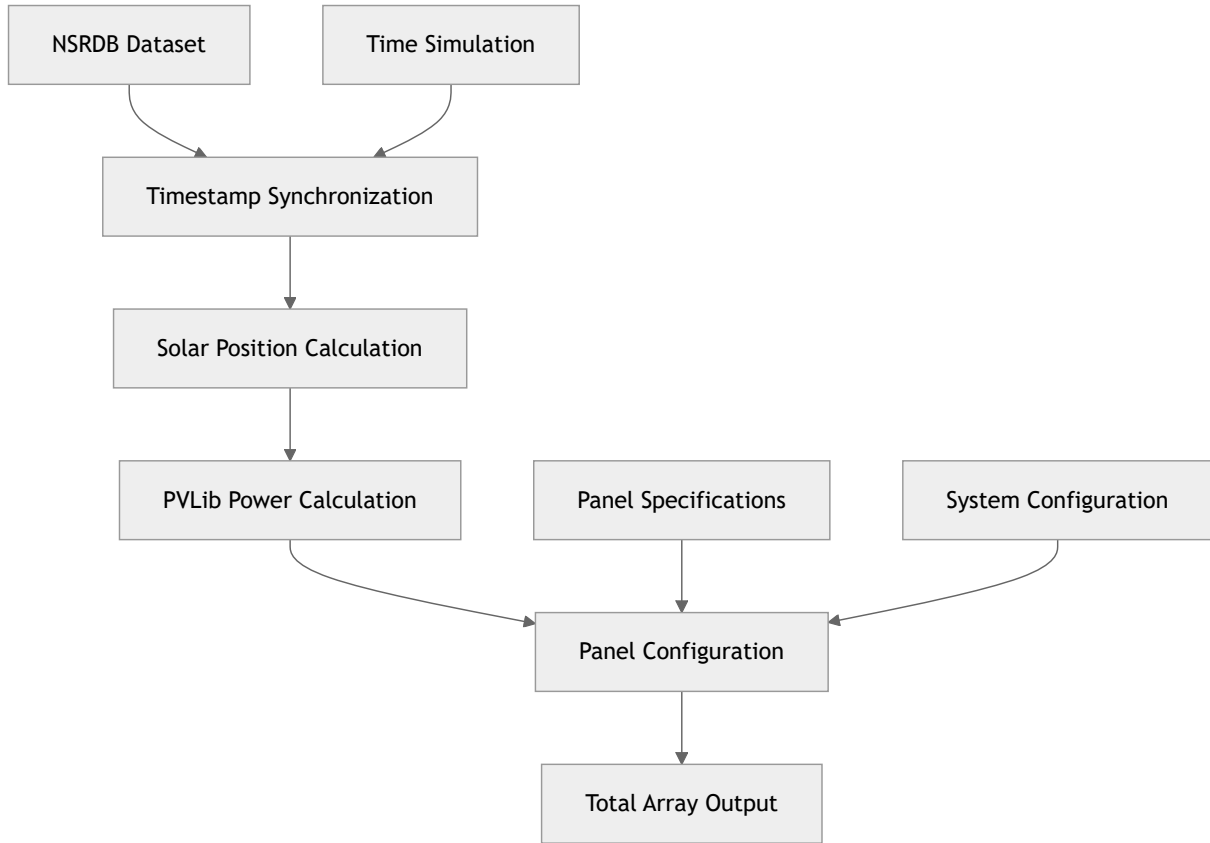


Fig. 4.2: Photovoltaic panel modeling workflow and data processing pipeline

The NSRDB contains hourly solar irradiance measurements, temperature data, and atmospheric conditions for locations across the United States. In the simulation, only data local to Phoenix/Arizona was used. The simulation system processes this data to extract direct normal irradiance (DNI), diffuse horizontal irradiance (DHI), and global horizontal irradiance (GHI) values necessary for photovoltaic calculations. The database integration includes automatic timestamp conversion, timezone handling, and data interpolation to support accelerated simulation time.

Solar position calculations account for latitude, longitude, time of year, and time of day to determine solar azimuth and elevation angles. The solar position information combines with irradiance data (DNI, DHI, and GHI) to calculate the effective solar energy. The PVLib library performs these calculations using established solar position algorithms. Below is an overview of the parameters of that calculation.

- **Direct Normal Irradiance (DNI)** represents the irradiance directly from the sun to the surface of the photovoltaic panels.
- **Diffuse Horizontal Irradiance(DHI)** represents the irradiance indirectly reaching the surface of the photovoltaic panels through the rays scattered by the sky dome.
- **Global Horizontal Irradiance(GHI)** represents the total irradiance, calculated by the following formula:

$$GHI = DNI \times \cos(\theta) + DHI$$

where θ is the solar zenith angle.

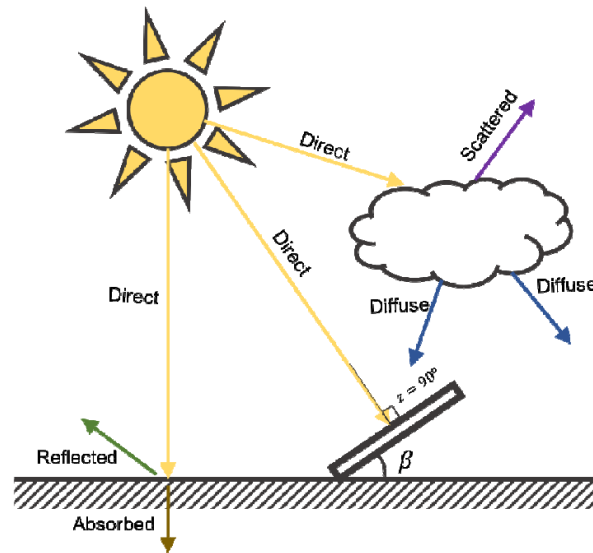


Fig. 4.3: Components of solar radiation (Source: ResearchGate)

Solar Irradiance and Solar Position for a Clear Day

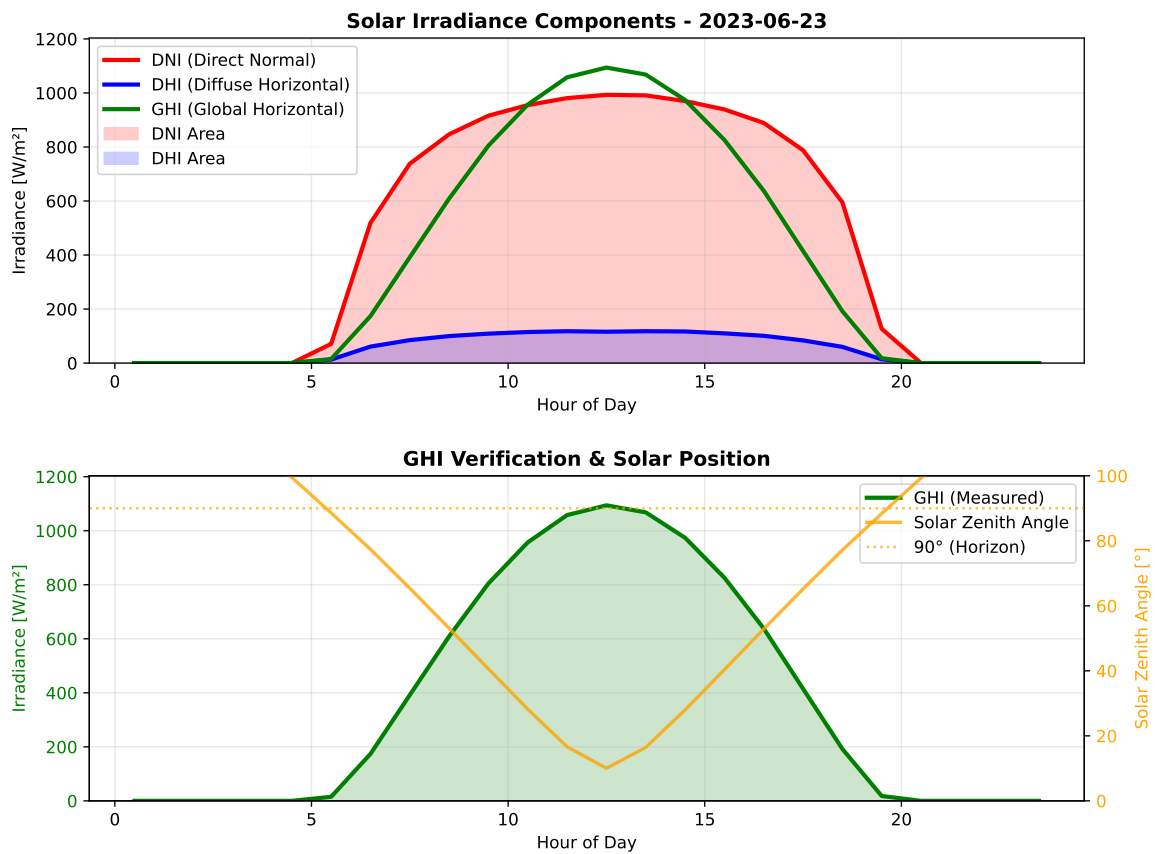


Fig. 4.4: Solar irradiance components over a typical clear day

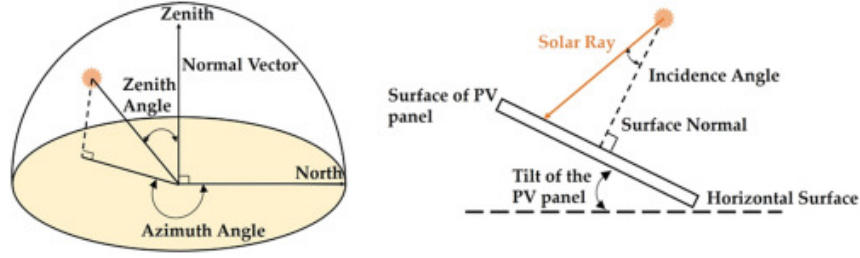


Fig. 4.5: Solar zenith angle (Source: ScienceDirect Topics)

The simulation supports configurable panel count, individual panel area, and conversion efficiency specifications that determine total array capacity. The total photovoltaic power generation follows:

$$P_{total} = POA_{irradiance} \times A_{panel} \times \eta_{panel} \times N_{panels}$$

where $POA_{irradiance}$ represents plane-of-array irradiance, A_{panel} represents individual panel area, η_{panel} represents panel efficiency, and N_{panels} represents the number of panels.

4.4 Battery Energy Storage System

The battery system implement lithium-ion battery characteristics including charge acceptance, storage capacity, and charge efficiency. The model provides an interface to the inverter to charge and discharge from the battery as per its calculations. As such, the battery component itself is devoid of any calculations beside implementing its charge efficiency and limitations.

The battery charge operation follows:

$$P_{charge,actual} = \min(P_{charge,requested}, P_{max,charge}) \times \eta_{charge}$$

$$E_{charge} = P_{charge,actual} \times \Delta t$$

$$SoC_{new} = \min(SoC_{current} + E_{charge}, C_{total})$$

where η_{charge} represents charging efficiency, Δt time interval in hours ($\Delta t = \frac{\Delta t_s}{3600}$ where Δt_s is the interval in seconds), and SoC state-of-charge.

The battery can also be configured on the maximum charge current it can accept, and the maximum discharge. This models the safety considerations present in actual batteries as they reject currents above a certain level. The discharge operation implements:

$$P_{discharge,actual} = \min(P_{discharge,requested}, P_{max,discharge}) / \eta_{charge}$$

$$E_{discharge} = \min(P_{discharge,actual} \times \Delta t, SoC_{current})$$

$$SoC_{new} = SoC_{current} - E_{discharge}$$

4.5 Inverter Control System

The inverter manages power flow between the photovoltaic panels, battery, and the power manager. It implements DC to AC conversion modeling to account for power losses that occur when converting direct current from the photovoltaic panels and the battery to alternating current for load consumption and utility export. The conversion is handled by the PVWatts library that follows established algorithms to accurately model the conversion.

The DC to AC conversion follows the PVWatts model:

$$P_{ac} = \min \left(\eta_{inv} \times P_{dc} \times \left(1 - \frac{P_{dc}}{P_{dc0}} \right), P_{ac0} \right)$$

where η_{inv} represents inverter efficiency, P_{dc0} represents DC power rating, and P_{ac0} represents AC power rating. The reverse calculation for required DC power uses:

$$P_{dc,required} = \frac{P_{ac,target}}{\eta_{inv,overall}}$$

Inverter DC-AC Conversion and Efficiency

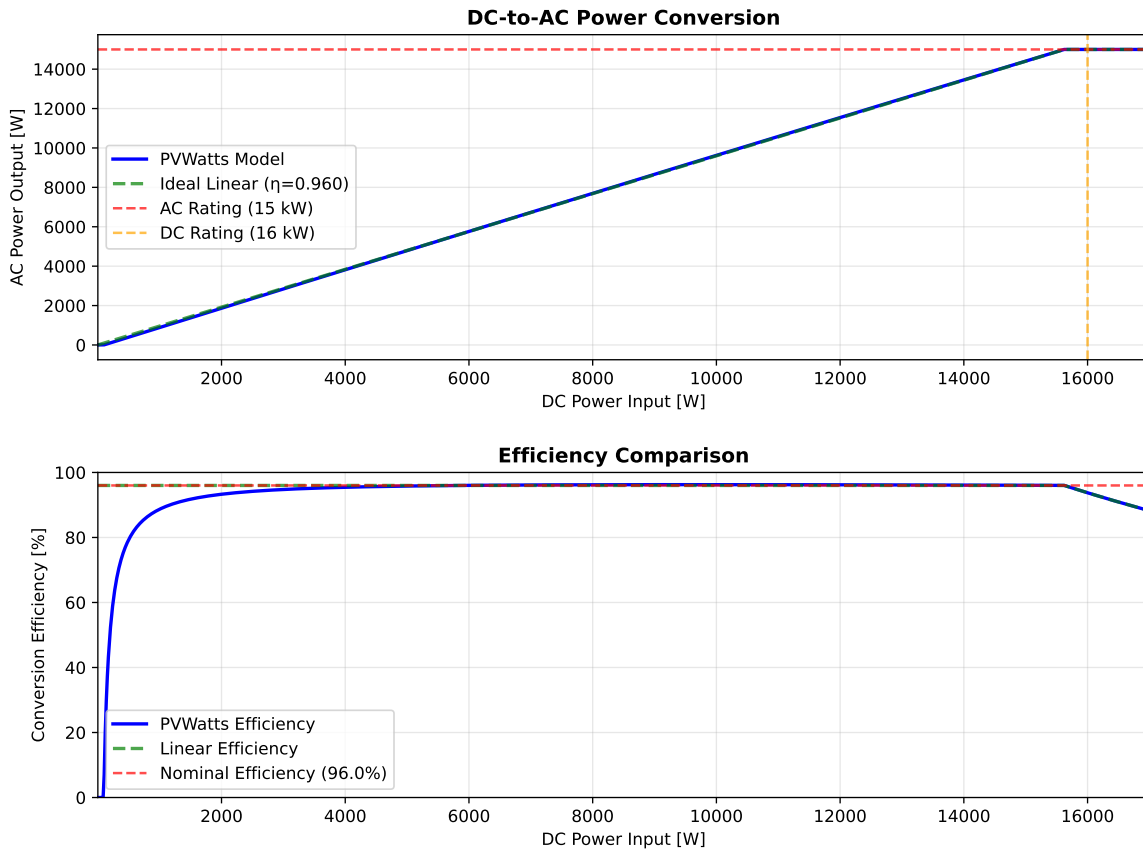


Fig. 4.6: DC-to-AC conversion characteristics

The inverter implements three operating modes for power flow priority.

- **Solar-Battery-Utility (SBU) mode** prioritizes using the power available from the photovoltaic panels if present, and only when the panels by themselves do not meet the load that it draws from the battery. When the battery is empty, only then would the inverter draw from utility. This mode aims to reduce grid-dependency to a minimum.
- **Solar-Utility-Battery (SUB) mode** prioritizes using the power available from the photovoltaic panels—just like the previous mode. However, when the panels power prove insufficient, it draws from utility to meet the demand and keeps the battery as only a last resort when no other source of energy exists. This mode aims to provide security and prepare to meet emergencies.
- **Utility-Solar-Battery (USB) mode** prioritizes drawing from utility. It uses the solar system as only a back-up in case the utility is out of service.

Note

The power manager can switch the operation mode of the inverter. This may prove useful in scenarios where the battery should be reserved for a foreseen emergency, thus switching the operation mode to SUB to use the utility now and keep the battery to be used when neither the solar panels nor the utility can provide power. While the capability for this currently exists in the system, the power manager carries out no such switches as it currently lacks the ability to predict emergencies.

Additionally, the inverter implements a charge priority, which determines how excess solar power gets allocated between battery charging and utility export. The system supports multiple charge priority settings including solar-only charging, solar-first charging, and utility-assisted charging modes.

- **Solar-only** directs all excess solar power to battery charging before any utility export occurs.
- **Solar-first** uses the excess solar power to charge the battery when present. When not present, it uses the utility to charge the battery.
- **Utility-assisted** enables battery charging from utility power even when some excess solar power is present.

In all charge priorities, the inverter will export to utility only when the battery is full. The power flow priority algorithms implement sequential decision trees:

$$P_{remaining} = P_{load} - \sum_{source} P_{source,allocated}$$

where power sources are allocated according to the configured priority mode, and any remaining unmet load triggers load line disconnection.

The inverter implements load shedding as a protective measure that prevents system overload. It disconnects the load when available power sources prove insufficient. The load is then automatically reconnected after a set period of time, during which the power manager should have taken care of the issue.

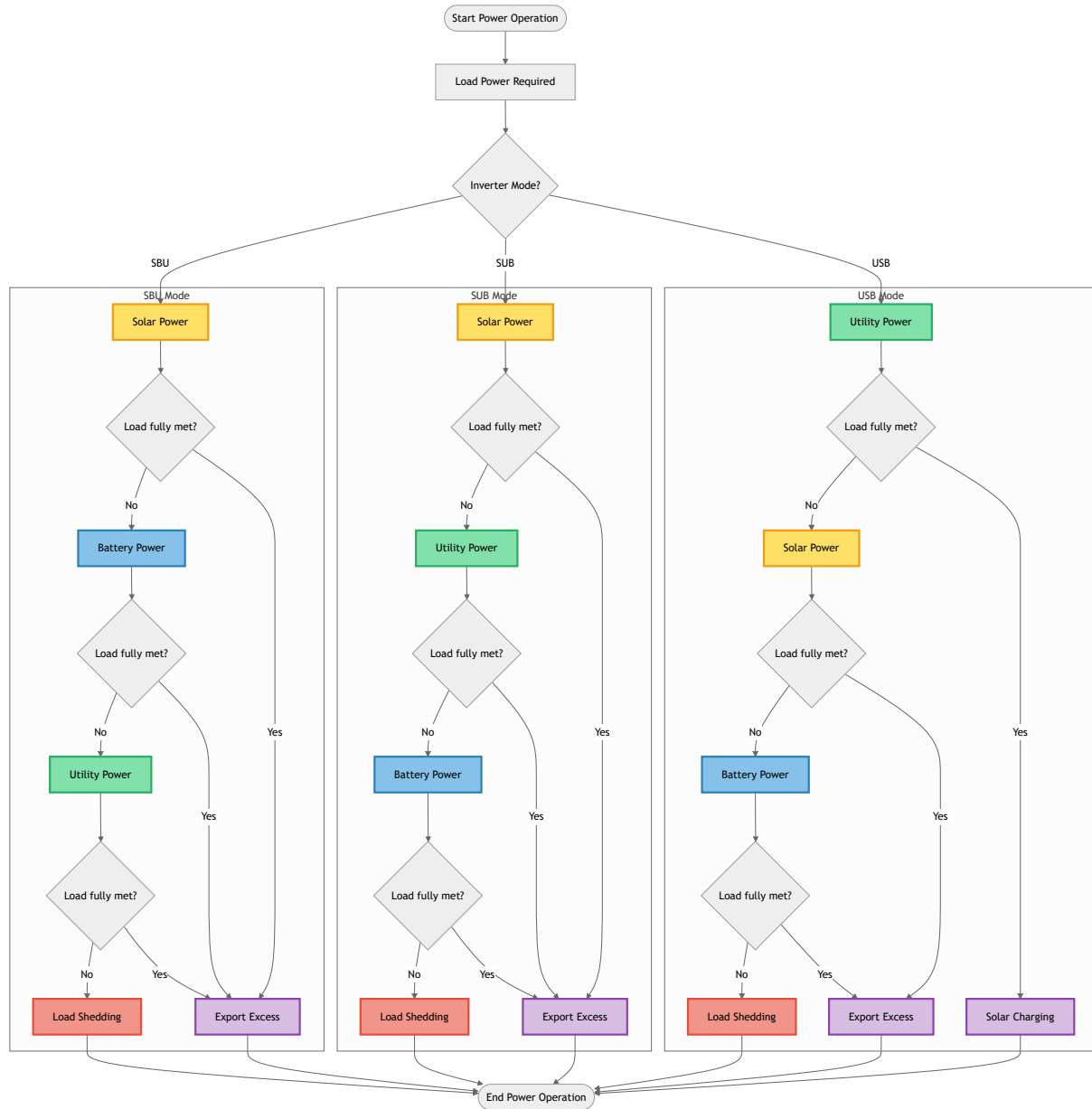


Fig. 4.7: Inverter operational modes

Note

The power manager ensures that load shedding may not happen as it automatically disconnects loads from the specific houses that are overusing. As such, load shedding is only ever triggered in the extreme situations where the power manager fails.

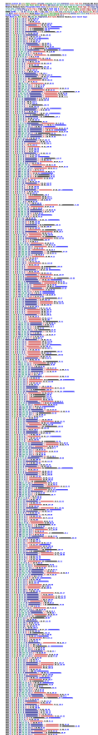
4.6 Weather Data Integration

This component manages the interface between the simulation environment and the external weather database, ensuring that solar calculations reflect actual atmospheric conditions.

The National Solar Radiation Database is the primary data source for solar irradiance and atmo-

spheric conditions. The NSRDB provides hourly measurements of solar radiation components (DNI, DHI, and GHI), ambient temperature, wind speed, and other meteorological parameters collected at numerous locations across the United States. This database is one of the most comprehensive sources of solar data available for renewable energy applications, providing the data necessary for detailed photovoltaic system modeling.

The dataset used from NSRDB covers the aforementioned measurements taken in the Phoenix/Arizona for one full year; one record each half an hour. Data is extracted using the simulation time; the record with the timestamp closest to the current simulation timestamp is used for the calculations of the solar energy in that specific moment in the simulation.



```

1 Source,Location ID,City,State,Country,Latitude,Longitude,Time Zone,Elevation,Local Time Zone,Clearsky DHI Units,Clearsky DNI
Units,Clearsky GHI Units,Dew Point Units,DHI Units,DNI Units,GHI Units,Solar Zenith Angle Units,Temperature Units,Pressure Units,
Relative Humidity Units,Precipitable Water Units,Wind Direction Units,Wind Speed Units,Cloud Type -15,Cloud Type 0,Cloud Type 1,
Cloud Type 2,Cloud Type 3,Cloud Type 4,Cloud Type 5,Cloud Type 6,Cloud Type 7,Cloud Type 8,Cloud Type 9,Cloud Type 10,Cloud Type
11,Cloud Type 12,Fill Flag 0,Fill Flag 1,Fill Flag 2,Fill Flag 3,Fill Flag 4,Fill Flag 5,Surface Albedo Units,Version
2 NSRDB,323705,-,,-,33.45,-112.06,0,334,-7,w/m2,w/m2,w/m2,c,w/m2,w/m2,w/m2,Degree,c,mbar,%,cm,Degrees,m/s,N/A,Clear,Probably Clear,
Fog,Water,Super-Cooled Water,Mixed,Opaque Ice,Cirrus,Overlapping,Overshooting,Unknown,Dust,Smoke,N/A,Missing Image,Low Irradiance,
Exceeds Clearsky,Missing Cloud Properties,Rayleigh Violation,N/A,4.0.1
3 Year,Month,Day,Hour,Minute,DNI,DHI,GHI,Temperature,Wind Speed,Relative Humidity,Solar Zenith Angle
4 2023,1,1,0,30,0,0,3,15.4,0.9,63.83,90.14
5 2023,1,1,1,30,0,0,0,14.5,1,65.28,102.17
6 2023,1,1,2,30,0,0,0,13.9,1.1,66.59,114.18
7 2023,1,1,3,30,0,0,0,13.3,1.2000000000000002,68.26,126.52
8 2023,1,1,4,30,0,0,0,12.700000000000001,1.3,70.33,139.02
9 2023,1,1,5,30,0,0,0,12.4,1.4000000000000001,71.38,151.39000000000001
10 2023,1,1,6,30,0,0,0,12.3,1.5,71.57000000000001,162.92000000000002
11 2023,1,1,7,30,0,0,0,12.3,1.6,71.71000000000001,169.55
12 2023,1,1,8,30,0,0,0,12.200000000000001,1.8,72.21000000000001,163.47
13 2023,1,1,9,30,0,0,0,12.3,2,71.62,152.04
14 2023,1,1,10,30,0,0,0,12.600000000000001,2,70.05,139.68
15 2023,1,1,11,30,0,0,0,12.8,2.1,69.12,127.18
16 2023,1,1,12,30,0,0,0,13.2,5,68.79,114.82000000000001
17 2023,1,1,13,30,0,0,0,13.3,3,6,70.74,102.78
18 2023,1,1,14,30,0,0,0,13.3,4,5,77.45,91.28
19 2023,1,1,15,30,0,4,4,13.200000000000001,5.2,83.76,80.5
20 2023,1,1,16,30,0,11,13.600000000000001,6,85.65,71.08
21 2023,1,1,17,30,0,14,14.5,6.600000000000005,83.95,63.43
22 2023,1,1,18,30,0,33,33,15.3,7,81.71000000000001,58.300000000000004
23 2023,1,1,19,30,0,56,56,15.700000000000001,6,7,80.60000000000001,56.410000000000004
24 2023,1,1,20,30,12,107,113,15.9,6.300000000000001,79.2,58.08
25 2023,1,1,21,30,11,59,64,15.5,6,2,75.38,63.02
26 2023,1,1,22,30,148,120,169,14.5,5.300000000000001,70.10000000000001,70.53
27 2023,1,1,23,30,9,46,48,13.3,5,70.3,79.86
28 2023,1,2,0,30,0,3,3,11.5,2.4000000000000004,76.2,90.03
29 2023,1,2,1,30,0,0,0,10.700000000000001,2.3000000000000003,79.26,102.03
30 2023,1,2,2,30,0,0,0,10.3,2.6,80.31,114.04
31 2023,1,2,3,30,0,0,0,9.8,2.9000000000000004,82.66,126.39

```

Fig. 4.8: Snapshot of the NSRDB dataset used

4.7 Power Management Interface

The solar system takes in the load from the power manager and gives back the solar panel power used, battery discharged, and grid import or export. The power manager then uses these data to take decisions regarding the load. The interface is two-way; the power manager provides the load, and the solar system provides metrics specifying how the load is being met.

Currently, the interface consists purely of the exchange of data; there is no exchange of commands or requests between the components.

4.8 Data Collection

Data collection operates continuously throughout simulation to capture time-series data about solar generation, battery operation, and system-wide energy flows. The collected data can then be analyzed to pinpoint issues in the model and help refine both the simulation and the power

management solutions. Utility import and export represent the most important metrics collected, as they are direct metrics of the effectiveness of the power management solution.

The system captures data from all the components to enable any form of analysis. The data collection is extensive: state-of-charge of the battery, power generated from the panels, the system-wide load, utility import and export, among other metrics. The data is logged to a CSV file continuously throughout the simulation.

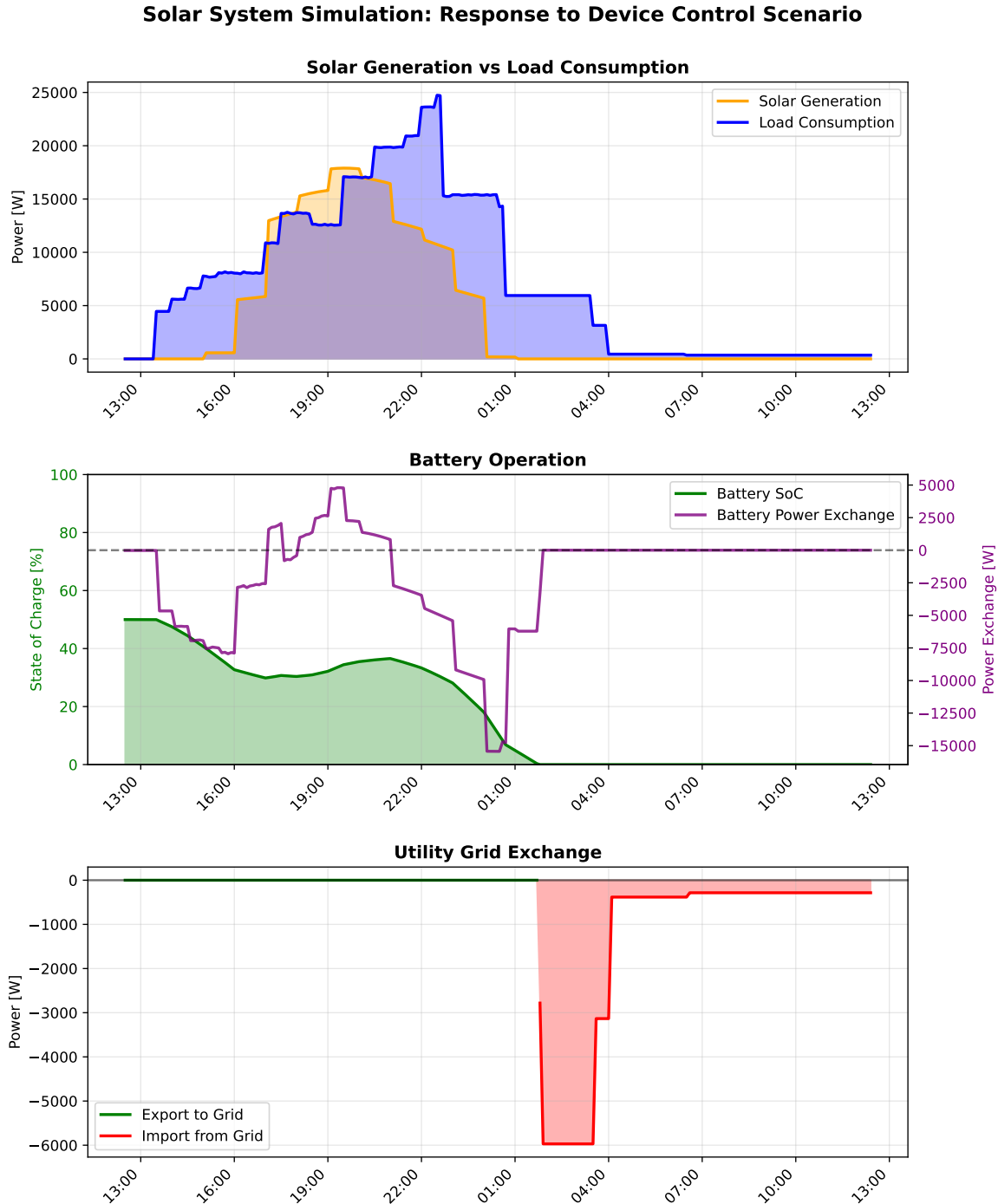


Fig. 4.9: Real-time solar system simulation demonstration

Along the data from NSRDB, the logged CSV files provide all the data needed to carry out analysis on the trends of power generation and consumption. Trend analysis might lead to improvements on the power management solutions. Additionally, AI techniques can be used to benefit from the data and extend the power manager with AI capabilities. This could be especially useful to predict shortages of power generation and thus switch the operation mode of the inverter to import from utility to keep the battery for that shortage and thus maintain long-term system stability.

CHAPTER

5

POWER MANAGEMENT

5.1 Introduction

The power management system serves as the intelligent coordination layer. This chapter examines the algorithms and mathematical foundation that enable the power manager to achieve efficient energy utilization.

The power manager implements an adaptive algorithm that learns from the consumption patterns and adjusts distribution weights as to minimize utility grid dependence. The system operates through two primary components: the central `PowerManager` that coordinates system-wide decisions, and individual `VirtualBattery` instances that represent each house's allocation within the shared battery.

5.2 System Architecture

The power management system integrates with both the houses simulation and solar system simulation through well-defined interfaces that enable real-time coordination and control.

The power manager operates on a configurable update cycle, typically set to match the houses simulation update interval of 100 milliseconds. During each update cycle, the manager executes a sequence of coordinated algorithms that process current system state, update virtual battery parameters, and make power distribution decisions.

The coordination interface provides bidirectional communication between components. The houses simulation provides current load demands and accepts load line control commands. The solar system simulation provides generation data, battery state information, and accepts load requirements while returning actual power delivery.

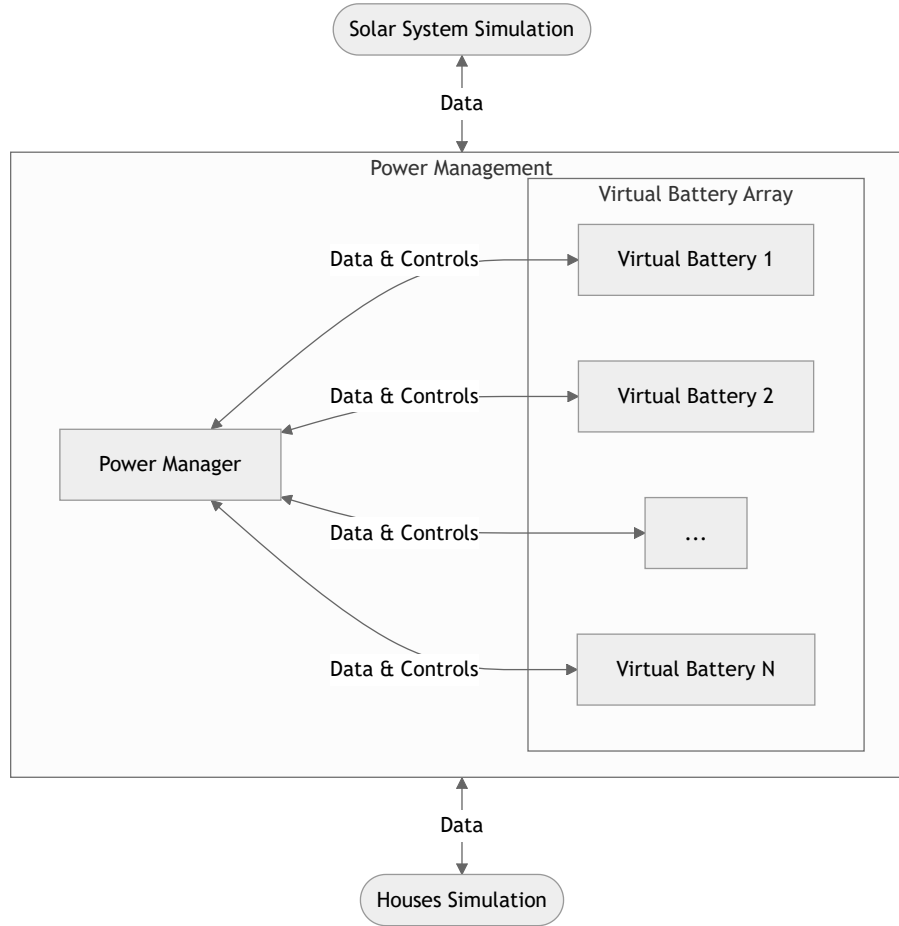


Fig. 5.1: Power management architecture

5.3 Virtual Battery System

The virtual battery system partitions the shared physical battery among individual houses based on their consumption patterns. Each house receives a dedicated `VirtualBattery` instance that tracks its allocated capacity, manages charge and discharge, and tracks the house's weight that determines its share of the total system capacity.

5.3.1 Mathematical Foundation

The virtual battery allocation system employs several mathematical relationships to ensure fair and efficient energy distribution. The fundamental relationship establishes the connection between virtual battery capacity and system-wide resources:

$$C_{vb,i} = \frac{C_{total}}{N} \times w_i$$

where:

- $C_{vb,i}$ represents the total capacity of the i -th virtual battery
- C_{total} represents the total capacity of the physical battery
- N represents the number of houses in the system

- w_i represents the weight assigned to the i -th house

The weight constraint ensures that the sum of all virtual capacities equals the total physical capacity:

$$\sum_{i=1}^N w_i = N$$

where N represents the number of houses in the system. This constraint ensures energy conservation between the physical battery and the virtual batteries.

The system implements weight boundaries to ensure fairness and prevent extreme allocations:

$$w_{min} \leq w_i \leq w_{max}$$

where:

- $w_{min} = \alpha$ (guaranteed minimum weight, typically 0.8)
- $w_{max} = 1 + (1 - \alpha) \times (N - 1)$

5.3.2 Virtual Battery Operations

The virtual battery implements charge and discharge operations that maintain energy conservation across the distributed system. The charge operation accounts for charging efficiency and capacity constraints:

$$P_{charge,actual} = \min(P_{charge} \times \eta_{charge} \times \Delta t, C_{total} - C_{residual}) \times \frac{1}{\Delta t \times \eta_{charge}}$$

where:

- $P_{charge,actual}$ represents the actual power consumed for charging
- P_{charge} represents the requested charging power
- η_{charge} represents the charging efficiency
- Δt represents the time interval in hours: $\Delta t = \frac{\Delta t_s}{3600}$ where Δt_s represents the time interval in seconds
- C_{total} represents the maximum capacity
- $C_{residual}$ represents the current residual capacity

The discharge operation implements efficiency losses and capacity limitations:

$$P_{discharge,actual} = \min\left(\frac{P_{discharge} \times \Delta t}{\eta_{charge}}, C_{residual}\right) \times \frac{\eta_{charge}}{\Delta t}$$

The energy conversion between power and capacity uses the relationship:

$$E = P \times \Delta t$$

where energy is measured in watt-hours and power in watts.

5.3.3 Weight Adjustment Algorithm

The adjustment function modifies weights while maintaining system constraints:

$$w_i^{t+1} = \text{clamp}(w_i^t + \Delta w_i, w_{\min}, w_{\max})$$

After adjusting the weights, the new capacity for the virtual battery is calculated:

$$C_{vb,i}^{t+1} = \frac{C_{\text{total}}}{N} \times w_i^{t+1}$$

When weight adjustment reduces total capacity $C_{vb,i}^{t+1}$ below current residual capacity $C_{vb,i}^{\text{residual}}$, the excess energy is calculated and returned to the system:

$$E_{\text{excess}} = \max(0, C_{vb,i}^{\text{residual}} - C_{vb,i}^{t+1})$$

This excess energy is then redistributed among the other virtual batteries to conserve the energy during system optimization.

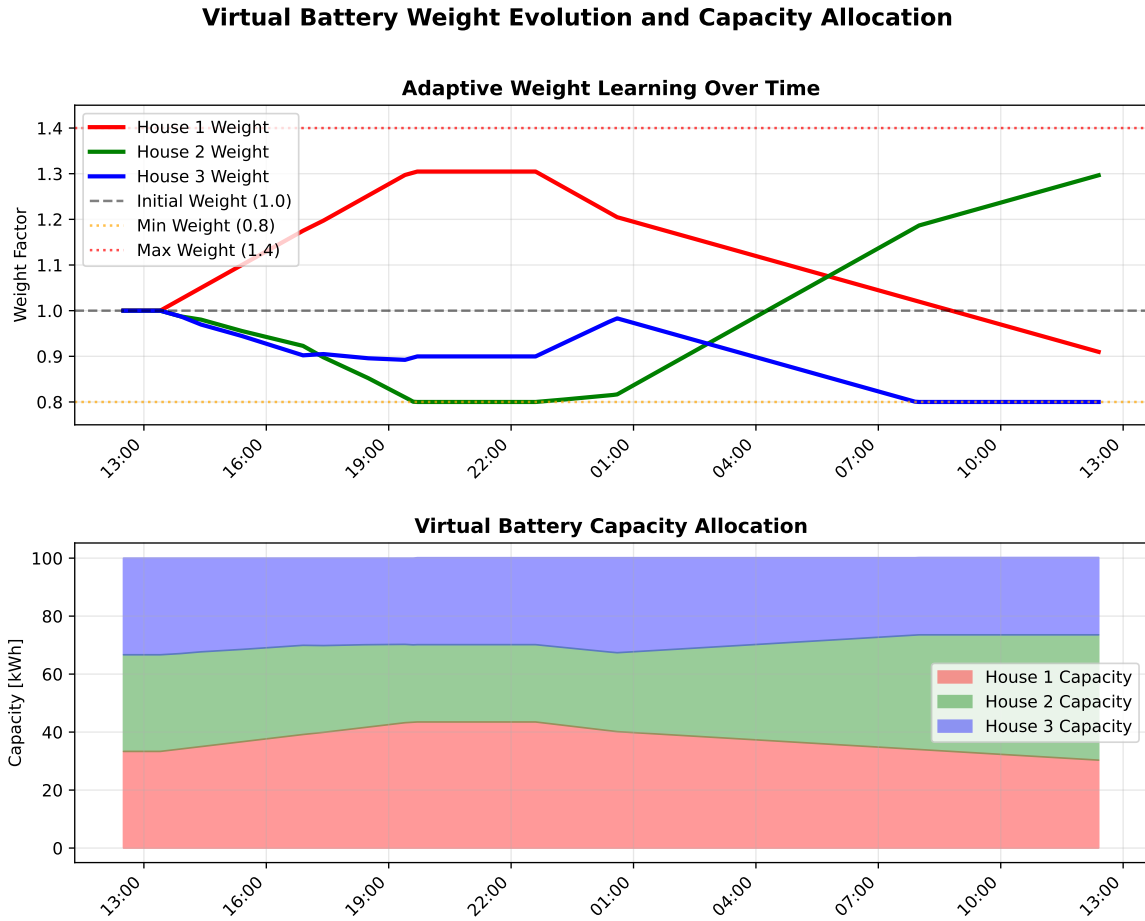


Fig. 5.2: Virtual battery weight evolution and capacity allocation

5.4 Adaptive Learning Algorithm

The power management system implements an adaptive learning algorithm that continuously adjusts virtual battery weights based on consumption. This algorithm enables the system to

optimize energy allocation over time, improving efficiency as it learns each house's consumption patterns.

5.4.1 Load Deviation Calculation

The learning algorithm employs statistical analysis of houses consumption to make decisions regarding their weights. It begins by calculating deviations from the mean consumption to identify houses with above-average and below-average demand:

$$\mu_L = \frac{1}{N} \sum_{i=1}^N L_i$$

$$\sigma_i = L_i - \mu_L$$

where:

- L_i represents the current load for the i -th house

The deviation values undergo normalization to ensure consistent scaling across different load magnitudes:

$$\sigma_{norm,i} = \frac{\sigma_i}{\max_j |\sigma_j|}$$

This normalization prevents large load values from overwhelming the learning algorithm and ensures that weight adjustments respond proportionally to relative consumption differences rather than absolute values.

5.4.2 Load Redistribution for Minimum Weight Houses

The algorithm implements a redistribution mechanism for houses operating at a minimum weight. When a house reaches its minimum weight and exhibits below-average consumption, the algorithm redistributes its negative deviation to houses with above-average consumption:

$$S_{pos} = \{i \mid \sigma_i > 0\}$$

$$S_{neg} = \{i \mid \sigma_i < 0\}$$

$$S_{min} = \{i \mid w_i = w_{min}\}$$

$$\sigma_{norm,j}^{adjusted} = \sigma_{norm,j} + \sigma_{norm,i} \times \frac{\sigma_{norm,j}}{\sum_{k \in S_{pos}} \sigma_{norm,k}}, \quad i \in S_{min} \cap S_{neg}, \quad j \in S_{pos}$$

This redistribution maintains energy conservation and ensures that houses at minimum weight do not continue to lose capacity allocation.

5.4.3 Weight Update and Learning Step

The calculated adjustments are then scaled by the learning rate and sent to the virtual battery to calculate its new weight:

$$\Delta w_i^{t+1} = \sigma_{norm,i}^{adjusted} \times \alpha_{learning}$$

where $\alpha_{learning}$ represents the learning rate (typically 0.002). The learning rate parameter controls the convergence speed and system stability. Smaller values provide more stable convergence at the cost of slower adaptation, while larger values enable rapid adaptation but may introduce oscillatory behavior.

Learning Algorithm Convergence Analysis: Weight Evolution

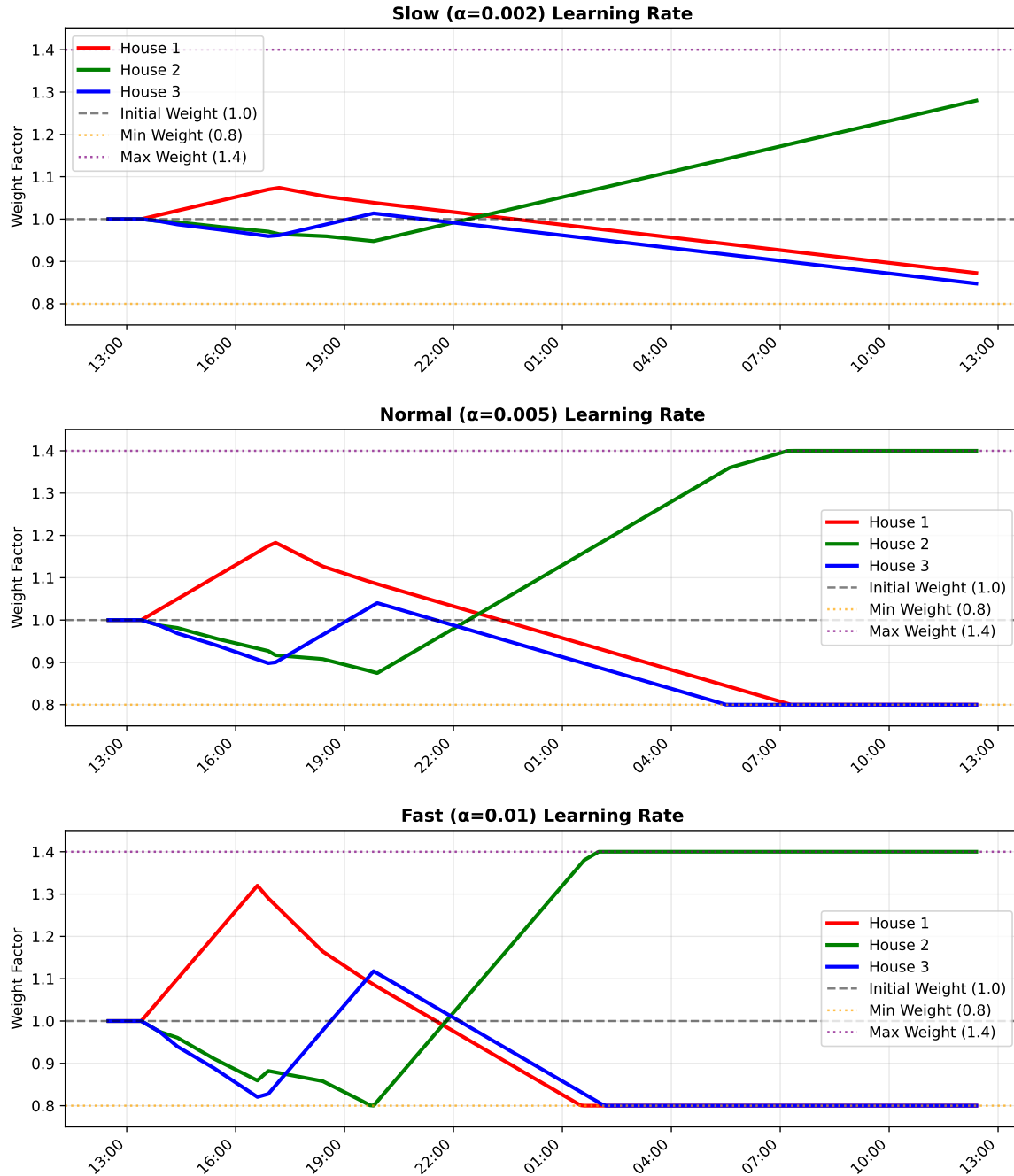


Fig. 5.3: Learning algorithm convergence analysis

Learning Algorithm Convergence Metrics

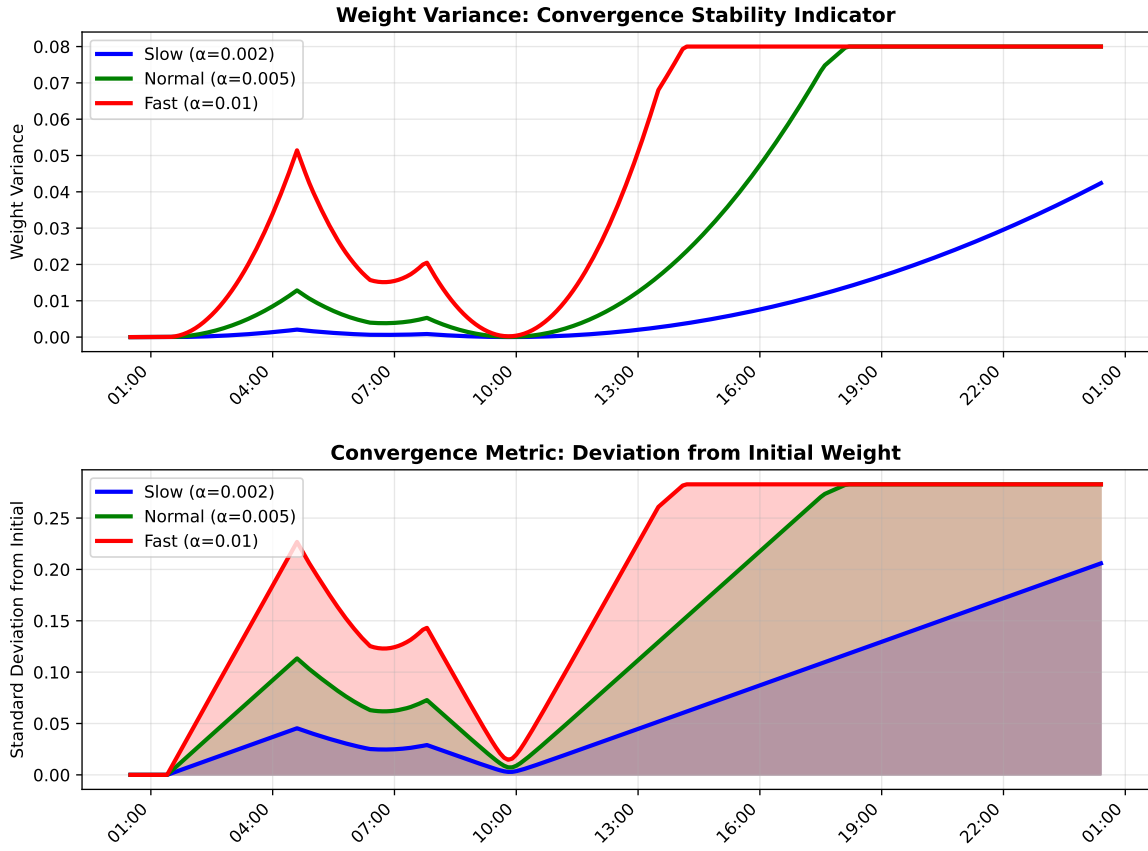


Fig. 5.4: Weight variance and convergence metrics analysis

The total excess capacity released from all weight adjustments is then aggregated:

$$E_{excess} = \sum_{i=1}^N E_{excess,i}$$

This excess capacity becomes available for redistribution during the charging phase, ensuring that energy released through weight reductions does not disappear, thus maintaining energy conservation.

5.5 Power Distribution Algorithms

The power management system implements algorithms for distributing available power sources among connected houses while maintaining fairness and system efficiency. These algorithms coordinate solar panel output, utility grid power, and virtual battery discharge to meet residential demand.

5.5.1 Houses Virtual Battery Usage Calculation

The virtual battery usage calculation determines how much energy each house should draw from their virtual battery after accounting for the solar and utility power available. The algorithm implements a priority-based allocation system that processes houses in ascending order of load demand to ensure fair distribution:

$$P_{panels,used,i} = \min \left(L_i, \frac{P_{panels,available}}{N_{remaining}} \right)$$

$$P_{utility,used,i} = \min \left(L_i - P_{panels,used,i}, \frac{P_{utility,available}}{N_{remaining}} \right)$$

$$P_{vb,i} = L_i - P_{panels,used,i} - P_{utility,used,i}$$

where:

- L_i represents the load of the i-th house
- $P_{panels,available}$ represents remaining solar panel power
- $P_{utility,available}$ represents remaining utility power
- $N_{remaining}$ represents the number of houses not yet processed
- $P_{vb,i}$ represents the i-th house power demand from its virtual battery

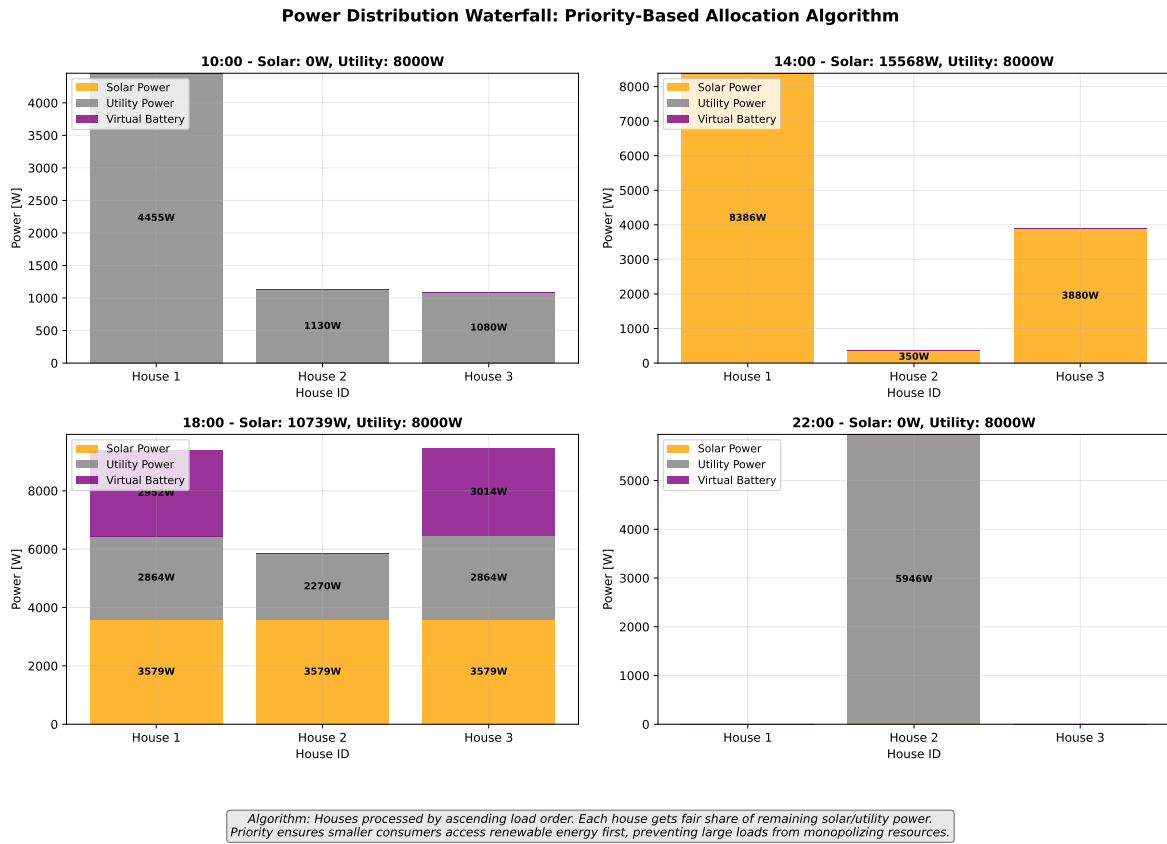


Fig. 5.5: Power distribution waterfall visualization

This sequential allocation ensures that houses with lower demand receive priority access to available solar and utility power, as they might not use their full share, thus allowing for efficient redistribution of the remainder equally among the remaining houses.

5.5.2 Virtual Battery Charging Algorithm

When excess power becomes available from solar generation, the charging algorithm distributes this power among virtual batteries based on their remaining capacity and charging capabilities. The algorithm implements priority charging for batteries with the lowest capacity deficit:

$$\text{Charging order: } \arg \min_i (C_{total,i} - C_{residual,i})$$

The charging power allocation follows:

$$P_{charge,i} = \frac{P_{charge,available}}{N_{remaining}}$$

where virtual batteries are processed in order of their capacity deficit, ensuring that batteries with the least available space receive charging priority as they might not need their full share, thus allowing for efficient redistribution of the remainder of their share equally among the remaining virtual batteries.

5.5.3 Virtual Battery Discharging with Weighted Distribution

The discharging algorithm implements weighted power distribution based on the normalized power demands from the virtual batteries. The algorithm calculates discharge weights using sum normalization:

$$w_{discharge,i} = \begin{cases} \frac{P_{vb,i}}{\sum_{j=1}^N P_{vb,j}} & \sum_{j=1}^N P_{vb,j} > \epsilon \\ \frac{1}{N} & \text{otherwise} \end{cases}$$

When the denominator approaches zero (no house demands discharging of its virtual battery), the algorithm defaults to equal distribution.

The actual discharge power for each virtual battery follows:

$$P_{discharge,i} = P_{discharge,total} \times w_{discharge,i}$$

The algorithm tracks whether each virtual battery can fully meet its required discharge:

$$\text{Usage Met} = |P_{discharge,i} - P_{actual,i}| < \epsilon$$

Houses whose virtual batteries cannot meet their full energy requirements trigger load line disconnection to prevent system instability.

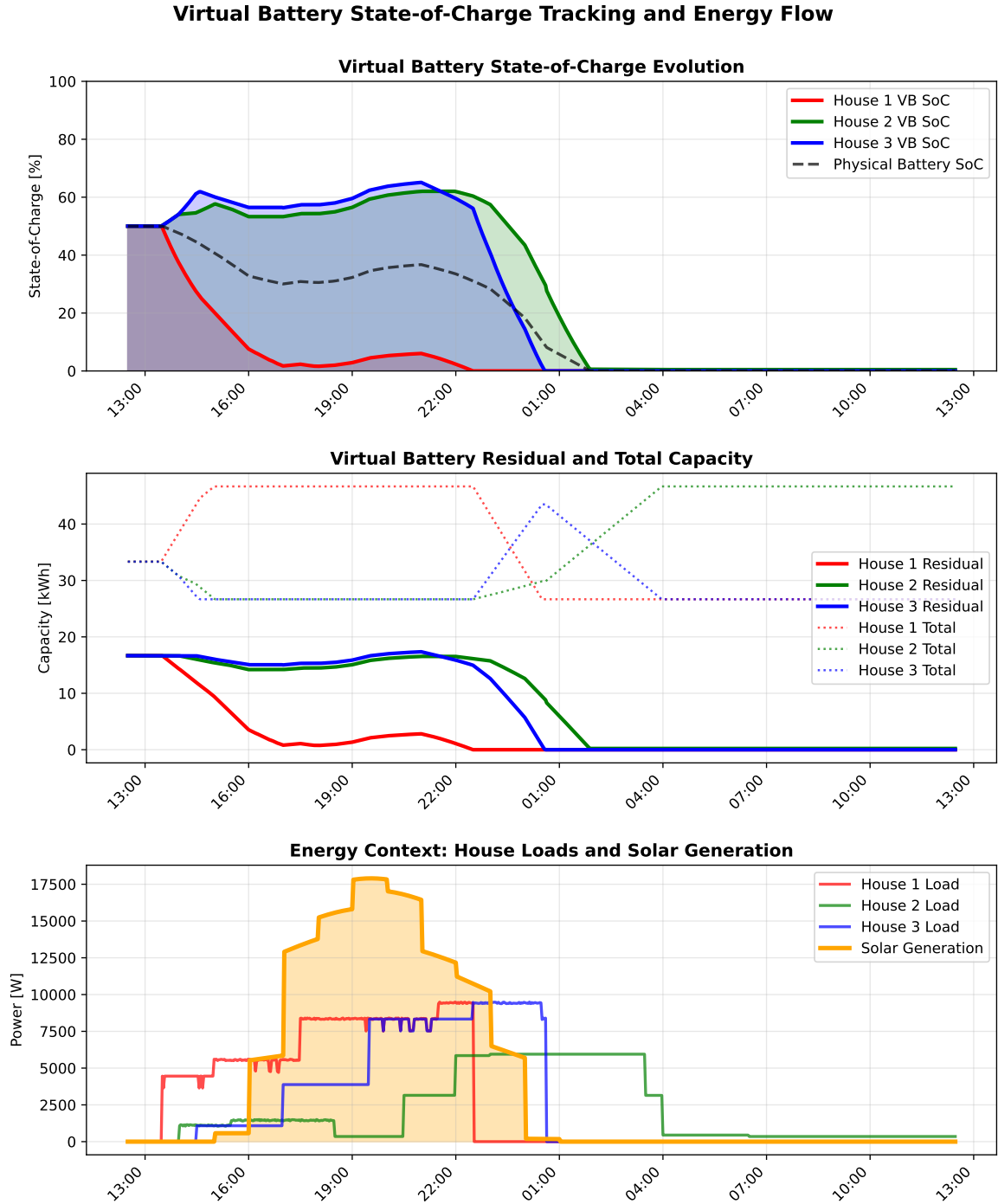


Fig. 5.6: Virtual battery state-of-charge tracking

5.6 System-Wide Load Line Coordination

The power manager coordinates load line states between individual houses and the solar system inverter. When the inverter implements load shedding due to extreme power shortage, the power manager propagates this state to all connected houses:

$$\neg LL_{inverter} \rightarrow \neg LL_i, \quad \forall i$$

where:

- $LL_{inverter}$ represents the state of the inverter load line connection
- LL_i represents the state of the load line connection of the i-th house

This ensures individual house load line states remain consistent with overall system capacity and prevents conflicts between local and system-wide load management decisions.

Restoration operates automatically after a configurable set interval (typically 30 seconds).

5.7 Utility Power Export Distribution

The power management system implements an algorithm for distributing utility export power among houses when excess energy becomes available for grid export. The algorithm ensures fair distribution of export benefits while accounting for individual house contributions to system efficiency.

5.7.1 Export Eligibility and Weight Calculation

The utility export distribution algorithm operates exclusively on houses that maintain active utility line connections:

$$S_{export} = \{i \mid UL_i\}$$

where:

- UL_i represents the utility line connection state of the i-th house

The export weight calculation employs inverse virtual battery weights to provide higher export allocation to houses with lower virtual battery weights, creating incentives for efficient energy usage:

$$w_{export,i} = \frac{1}{w_{vb,i}}$$

The normalized export distribution calculates each house's share of total export power using sum normalization:

$$w_{export,norm,i} = \frac{w_{export,i}}{\sum_{j \in S_{export}} w_{export,j}}, \quad \forall i \in S_{export}$$

5.7.2 Export Power Allocation

The final export power allocation for each eligible house follows:

$$P_{export,i} = P_{export,total} \times w_{export,norm,i}$$

where $P_{export,total}$ represents the total power available for utility export from the solar system.

This ensures that houses with more efficient virtual battery utilization (lower weights) receive proportionally higher export benefits, creating economic incentives for optimal energy usage.

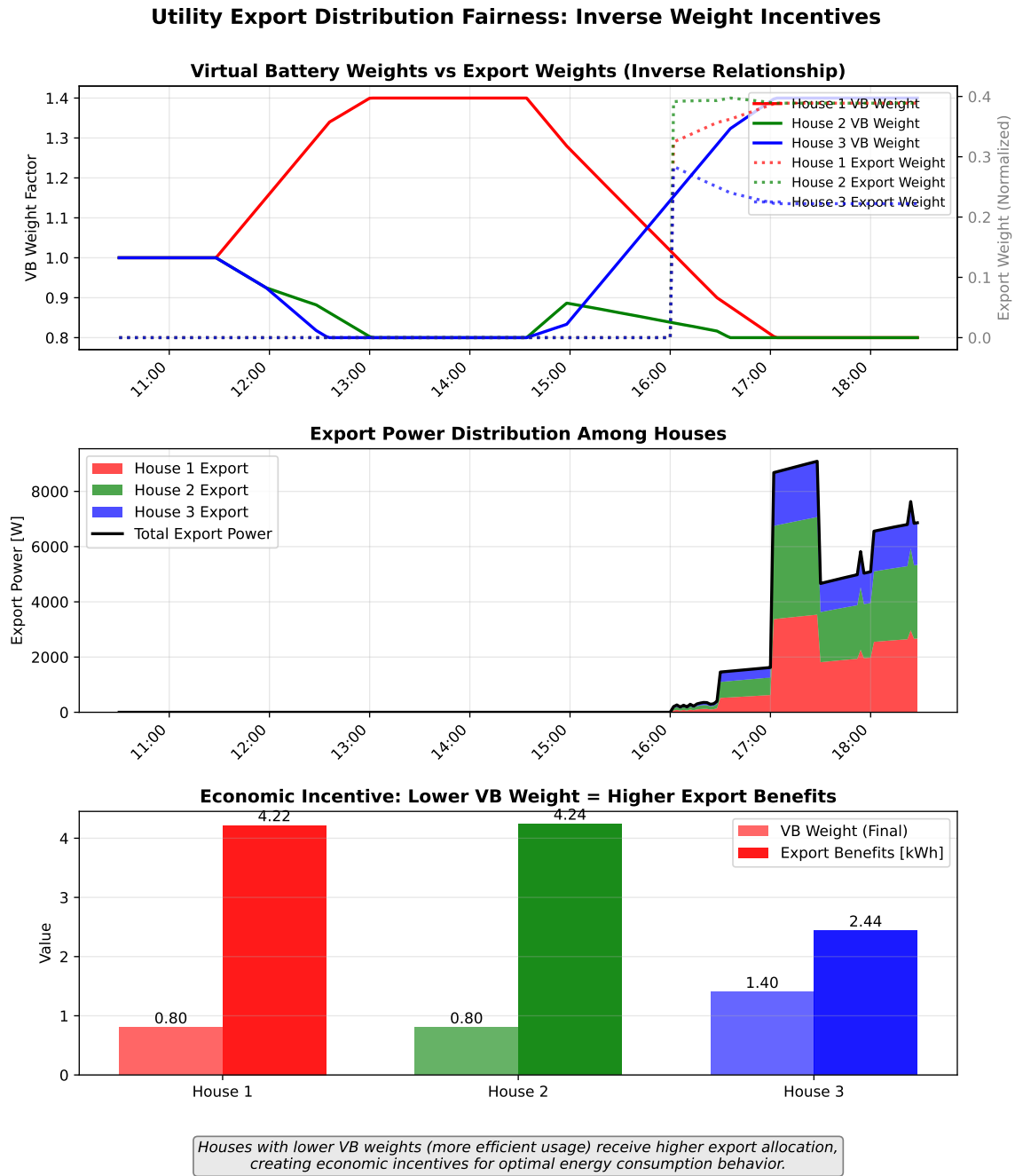


Fig. 5.7: Utility export distribution fairness demonstration

5.8 System Integration and Real-Time Control

The power management system maintains continuous coordination with simulation components through real-time interfaces. The integration architecture ensures that power management decisions reflect current system state.

5.8.1 Real-Time Update Cycle Implementation

The power manager executes its core algorithm within a threaded update loop that operates independently of other system components. The update cycle duration matches the simulation time step, typically configured to 100-millisecond intervals. This synchronization ensures that power management decisions occur at the same time as houses simulation and solar system simulation updates.

The update sequence implements a structured workflow that processes system state, executes learning algorithms, performs power distribution calculations, and updates component states within each cycle:

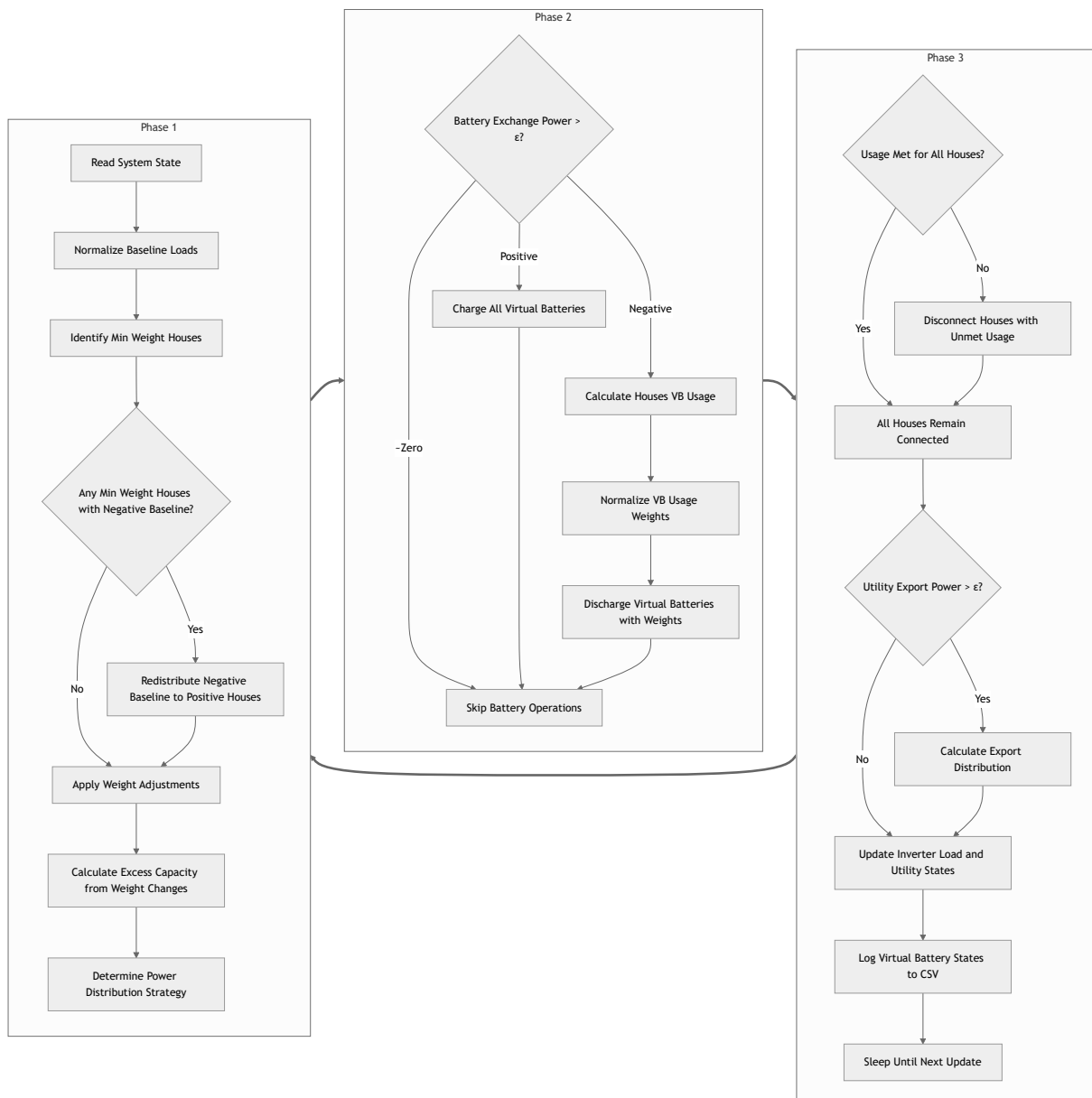


Fig. 5.8: Power manager real-time update cycle

5.8.2 Inverter Synchronization

The power manager maintains synchronization with the solar system inverter through bidirectional communication that coordinates load requirements and power delivery capabilities. The synchronization algorithm updates inverter state based on aggregated house demands:

$$P_{load,inverter} = \sum_{i=1}^N L_i \times LL_i$$

where LL_i represents the binary load line state for the i -th house.

The utility line state coordination ensures that the inverter maintains proper grid connection based on house connectivity:

$$UL_{inverter} = \bigvee_{i=1}^N UL_i$$

where UL_i represents the binary utility line state for the i -th house. This coordination maintains consistency between individual house utility connections and overall system grid interface requirements.

Note

The current model for setting the inverter's utility line is not representative of the real world and thus must be changed to one more closely reflective of reality. It was set as this for now for the lack of a better idea, and further analysis of the problem was precluded by approaching deadlines.

5.9 Performance Monitoring and Data Collection

The power management system implements data collection that captures system performance metrics and algorithm behavior. The monitoring system enables both real-time performance assessment and historical analysis of power management effectiveness.

The system collects time-series data for all virtual battery states, capturing capacity utilization, weight evolution, and energy flow throughout simulation execution. The collected data enables statistical analysis of algorithm convergence, weight distribution evolution, and system efficiency improvements over time. Performance metrics derived from this data include average utility grid dependence, battery utilization efficiency, and load balancing effectiveness.

The collected data allows for the calculation of key performance metrics that measure power management effectiveness and system optimization success. Primary metrics include utility grid import minimization and battery capacity utilization:

$$\text{Grid Dependence Ratio} = \frac{\sum_t P_{utility,import,t}}{\sum_t P_{load,total,t}}$$

$$\text{Battery Utilization} = \frac{\sum_t C_{discharge,t}}{\sum_t C_{residual,t}}$$

These metrics provide quantitative assessment of power management performance and enable comparison between different algorithm configurations and parameter settings.

 Note

The extensive data collection enables future integration of machine learning for predictive algorithms and advanced optimization techniques.

CHAPTER

6

DASHBOARD

6.1 Introduction

The dashboard serves as the human-machine interface for the residential smart grid prototype, providing real-time monitoring and control. This chapter examines the implementation of the graphical user interface and the remote object communication architecture that enable operators to interact with the simulation.

6.2 Remote Object Architecture

The dashboard implements a distributed architecture using Pyro5 remote objects to communicate with the simulation components. This design separates the user interface from the computation-intensive simulation processes, enabling the dashboard to run independently while maintaining synchronized access to system state and control functions.

The system uses environment variables to configure connection parameters, allowing deployment flexibility across different network configurations and development environments.

Note

The Pyro5 remote object framework provides transparent network communication, enabling the dashboard to treat remote simulation components as local objects while maintaining network resilience and error handling.

6.3 Dashboard Application Framework

The dashboard application builds upon the Tkinter GUI framework enhanced with tkbootstrap styling to provide a modern, responsive user interface.

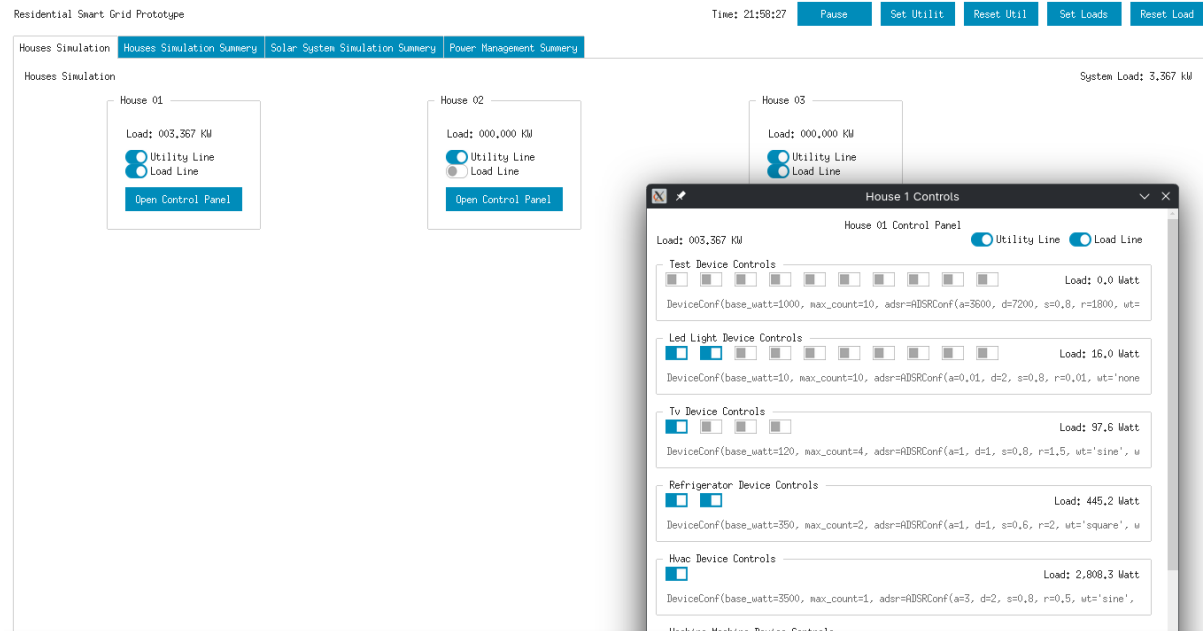


Fig. 6.1: Dashboard snapshot

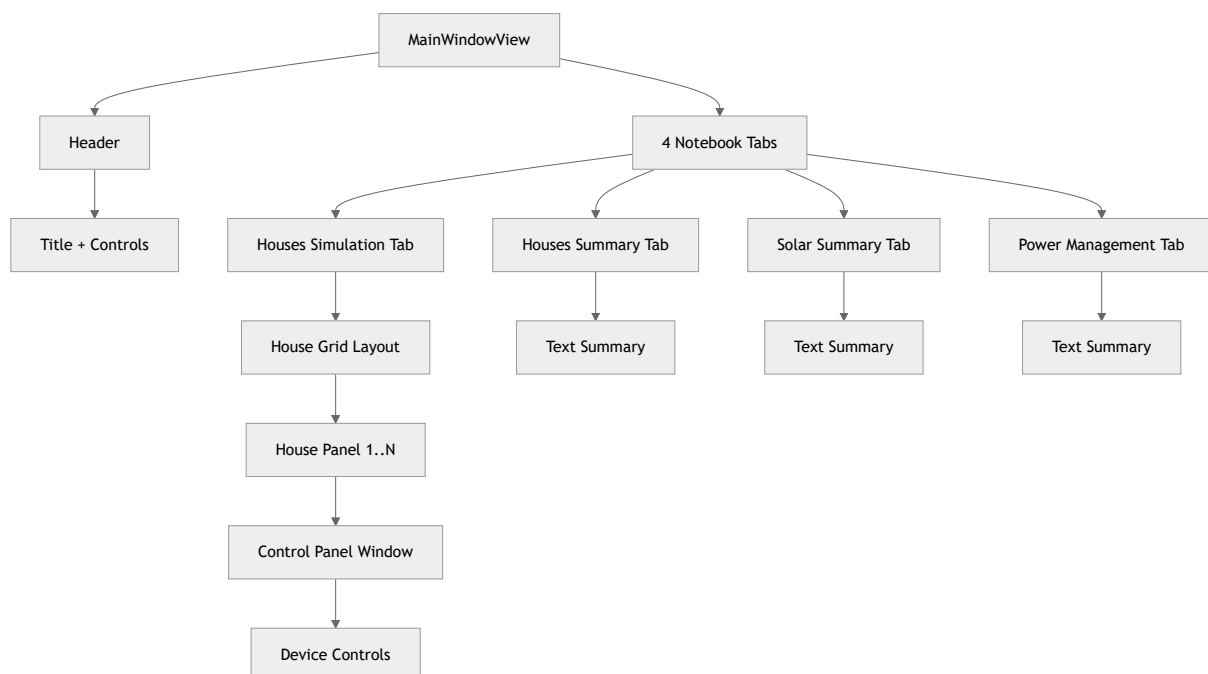


Fig. 6.2: Dashboard component layout

The application implements a hierarchical view structure that scales to accommodate varying numbers of houses and devices within the simulation. The framework provides scrollable inter-

faces for scenarios with large numbers of components, ensuring usability regardless of simulation scale.

6.4 Houses Simulation Interface

The houses simulation interface provides monitoring and control capabilities for individual houses. Each house panel displays current power consumption, connectivity status, and provides access to control the house's load. The utility line and load line toggles enable operators to simulate grid disconnections to support testing of power management response mechanisms.

The house control window provides access to individual device operations within each house. These windows display device-specific information including current load and configuration parameters.

6.5 Summary and Monitoring Views

The dashboard implements dedicated summary views for each major simulation component, providing comprehensive real-time status information and operational metrics. These views serve as primary monitoring interfaces for system analysis and troubleshooting activities.

- The houses simulation summary displays system-wide statistics including total load, individual house contributions, and connectivity states. The information updates continuously during simulation execution and provides formatted output that facilitates rapid assessment of system state.
- The solar system simulation summary provides detailed information about photovoltaic generation, battery state, and inverter operations.
- The power management summary displays algorithm performance metrics and decision-making status. This information proves essential for evaluating power management effectiveness and identifying areas for algorithm improvement.

6.6 Real-time Update Architecture

The dashboard implements a real-time update system that maintains synchronization between the user interface and the underlying simulation. The length of the update cycle can be set to balance between maintaining real-time connection and minimizing overhead. As such, the default of 100-milliseconds was chosen as a middle between responsiveness and low overhead.

6.7 Data Collection and Analysis Support

The dashboard provides access to simulation data through its summary views and component monitoring interfaces. This information supports both real-time system analysis and post-simulation evaluation of system performance and algorithm effectiveness.

CHAPTER

7

RASPBERRY PI CONTROLLER

7.1 Introduction

The Raspberry Pi controller serves as the physical hardware interface for the residential smart grid Prototype. This chapter examines the implementation of the GPIO controller system that bridges between the software simulation and physical hardware controls. The controller enables real-time interaction with the RSGP simulation through physical buttons and visual feedback via LEDs; thus providing an intuitive testing environment.

7.2 System Architecture Overview

The controller architecture separates hardware abstraction, GPIO management, and simulation integration into distinct layers. This modular design ensures that hardware-specific code remains isolated from simulation logic; thus maintaining system flexibility and enabling future expansion to different hardware platforms.

The architecture implements a three-layer approach: the hardware layer manages physical GPIO operations; the controller layer provides abstraction and mapping logic; and the communication layer enables remote interaction with the RSGP simulation. This separation ensures that each layer operates independently; thus allowing for individual component testing and modification without affecting other system elements.

The architecture is a variation on the popular Model-View-Controller architecture, but the Views are swapped here with a communication system that synchronizes with the simulation, albeit a minimal View represented by the state of the LEDs still remains.

7.3 Hardware Configuration and GPIO Mapping

The hardware configuration system manages the mapping between physical GPIO pins and logical device controls.

7.3.1 GPIO Pin Allocation Strategy

The pin allocation follows a systematic approach that groups related controls and maintains logical separation between houses and device types. The allocation strategy ensures that each house receives dedicated GPIO pins for its device controls; while global controls such as the utility line receive separate pin assignments.

The GPIO mapping employs a paired button-LED configuration where each control function receives both an input button and a corresponding output LED. The button enables user interaction; while the LED provides visual feedback about the current state of the load line of the associated house in the simulation. This pairing ensures that users receive immediate confirmation of their actions and continuous state information.

7.3.2 Mathematical GPIO Pin Assignment

The GPIO pin assignment follows a mathematical pattern that ensures systematic allocation across houses and device types. The button pin allocation for house devices follows:

$$P_{button,h,d} = (h - 1) \times 4 + d + 1$$

where h represents the house number (1-3), d represents the device index (0-3), and P_{button} represents the GPIO pin assigned to the button.

The LED pin allocation maintains a consistent offset from button pins:

$$P_{LED,h,d} = P_{button,h,d} + 13$$

This mathematical relationship ensures predictable pin assignments and simplifies hardware layout planning. The utility line receives dedicated pin assignments outside the house-specific allocation range to prevent conflicts with house-level controls.

7.4 GPIOController Implementation

The `GPIOController` serves as the primary interface between the Raspberry Pi hardware and the RSGP simulation system. The controller implements comprehensive GPIO management, including input monitoring, output control, and bidirectional communication with simulation components.

7.4.1 Controller Initialization and Setup

The controller initialization process establishes GPIO chip access; configures pin modes; and sets up callback mechanisms for user interactions. The initialization sequence ensures that all hardware resources are properly allocated before beginning the main control loop.

The GPIO setup employs pull-up resistors for button inputs to ensure reliable signal detection; while LED outputs initialize to the off state to provide a consistent starting configuration. The controller maintains internal state tracking for both buttons and LEDs to enable efficient change detection and minimize unnecessary GPIO operations.

7.4.2 Button State Monitoring Algorithm

The button monitoring system implements edge detection to identify button press events to prevent multiple triggers from single user actions. The algorithm maintains previous button states and compares them with current readings to detect falling edge transitions that indicate button presses.

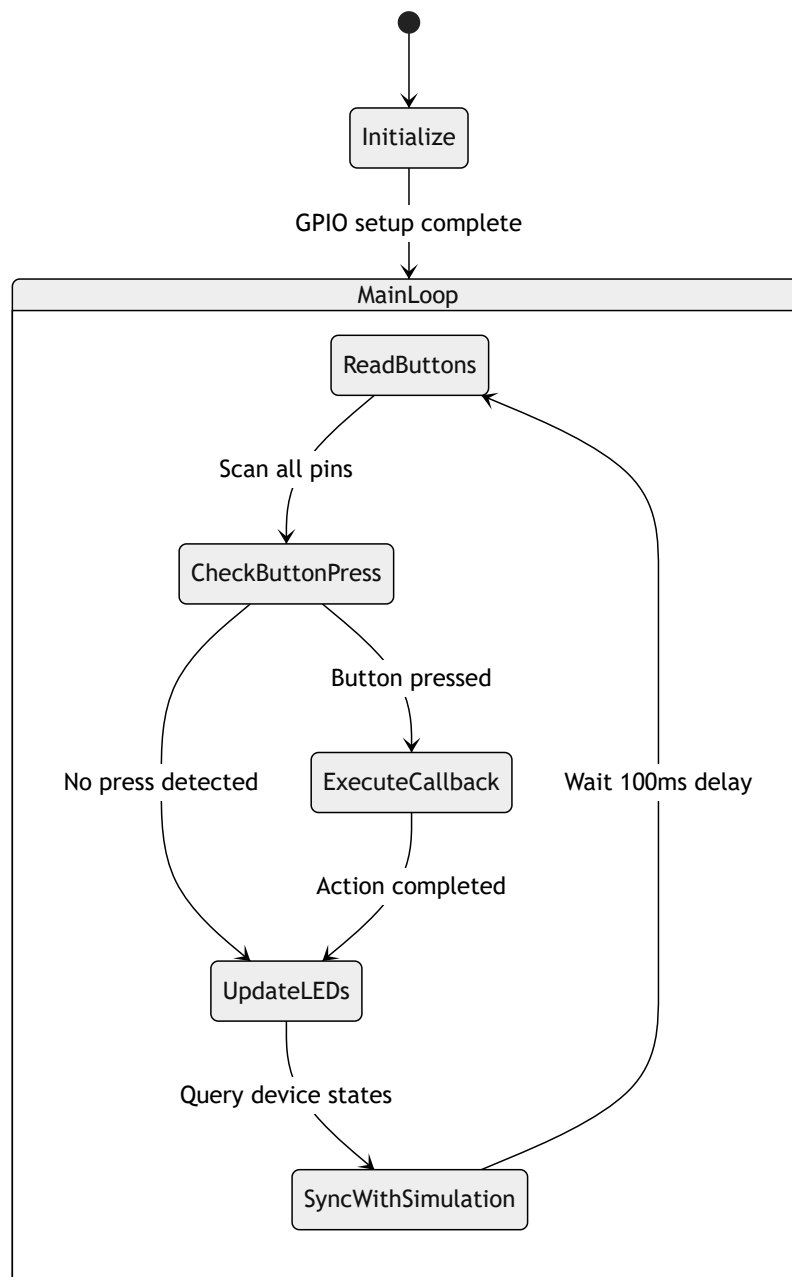


Fig. 7.1: GPIO controller state machine

$$\text{Button Press} = S_{previous} \wedge \neg S_{current}$$

where $S_{previous}$ represents the previous button state and $S_{current}$ represents the current button state. This logical operation identifies the transition from high to low that occurs when a button is pressed with pull-up configuration.

The button reading cycle operates continuously within the main update loop:

7.4.3 LED State Synchronization

The LED state synchronization system maintains visual feedback that accurately reflects the current state of simulation components. The algorithm queries simulation components to determine their current status then updates LED states accordingly to provide real-time visual feedback.

The LED update process implements efficient change detection to minimize GPIO write operations:

$$\text{LED Update Required} = S_{LED, cached} \oplus S_{simulation, current}$$

where $S_{LED, cached}$ represents the cached LED state and $S_{simulation, current}$ represents the current simulation component state. This comparison prevents unnecessary GPIO operations when states remain unchanged.

7.4.4 Device Control Callback Functions

The controller maps GPIO pins to callback functions.

- **Utility Line Control:** The utility line control callback affects all houses simultaneously; reflecting the global nature of utility grid connection.
- **Individual Device Control:** Individual device controls target specific house devices; enabling fine-grained control over simulation behavior.

7.5 Remote Communication Interface

The controller implements remote communication with the simulation through the Pyro5 distributed object system. This communication enables the controller to operate independently of the main simulation while maintaining synchronized state information.

7.5.1 Pyro5 Proxy Configuration

The proxy configuration establishes connection parameters. The system employs environment variable configuration to enable flexible deployment across different network configurations.

```
HOST = os.getenv('RSGP_REMOTE_OBJECT_HOST', '0.0.0.0')
PORT = int(os.getenv('RSGP_REMOTE_OBJECT_PORT', 41991))
BASE = f'PYRO:{{name}}@{HOST}:{PORT}'
```

The proxy configuration supports both local and networked deployment scenarios. Local deployment enables single-machine testing; while networked deployment enables distributed system evaluation.

7.6 Real-Time Control Loop Implementation

The controller employs a fixed update cycle timing that balances responsiveness with computational efficiency. The default update interval of 100 milliseconds provides adequate responsiveness for human interaction while preventing excessive GPIO polling that could impact system performance.

$$f_{update} = \frac{1}{\Delta t_{update}} = \frac{1}{0.1} = 10 \text{ Hz}$$

The update frequency of 10 Hz ensures that button presses receive prompt recognition and LED updates occur frequently enough to provide smooth visual feedback.

7.7 Hardware Integration and Deployment

The controller requires specific hardware configuration and deployment considerations to ensure reliable operation within the residential smart grid testing environment.

7.7.1 Physical Hardware Requirements



Fig. 7.2: Raspberry Pi used in the project

The controller implementation requires a Raspberry Pi single-board computer with sufficient GPIO pins to support the defined control mappings. The system requires:

- Raspberry Pi 4 or equivalent with 40-pin GPIO header
- 13 push-button switches with pull-up configuration
- 13 LED indicators with appropriate current-limiting resistors
- Breadboard or custom PCB for component mounting
- Power supply suitable for Raspberry Pi and connected components



Fig. 7.3: Hardware parts required by the system

7.7.2 Physical Hardware Layout and Wiring Specifications

The hardware configuration implements a comprehensive physical layout that maps logical device controls to specific GPIO pins.

Table 7.1: GPIO pin mappings for hardware controls

Control Function	House ID	Button GPIO Pin	LED GPIO Pin
H1 Refrigerator	1	2	15
H1 HVAC	1	3	16
H1 Water Heater	1	4	17
H1 Load Line	1	5	18
H2 Refrigerator	2	6	19
H2 HVAC	2	7	20
H2 Water Heater	2	8	21
H2 Load Line	2	9	22
H3 Refrigerator	3	10	23
H3 HVAC	3	11	24
H3 Water Heater	3	12	25
H3 Load Line	3	13	26
Utility Line	Global (0)	14	27

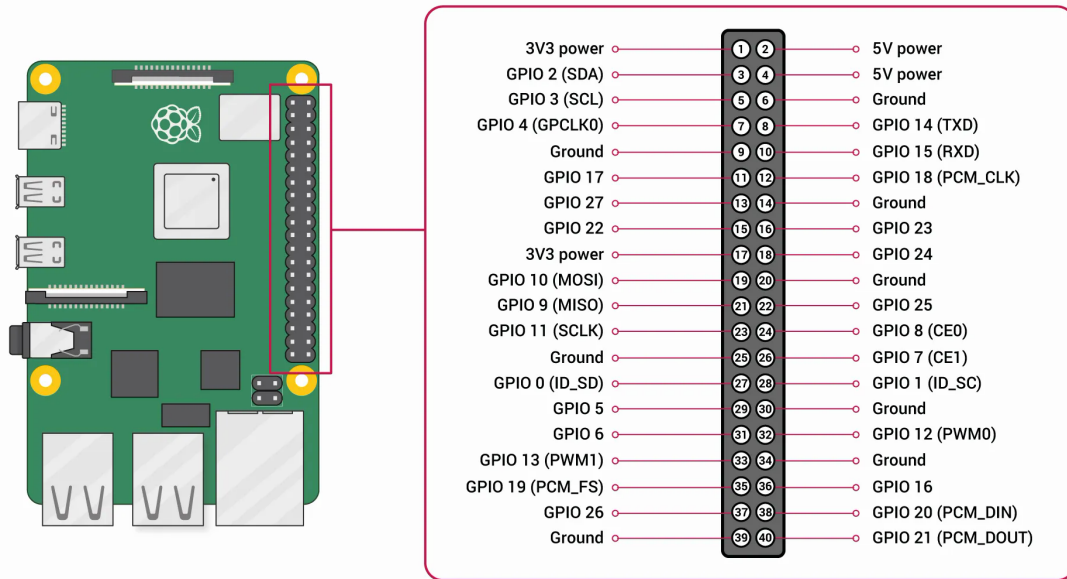


Fig. 7.4: Raspberry Pi 5 GPIO pinout diagram

The GPIO pin allocation follows the mathematical assignment pattern defined in the hardware configuration system. The complete hardware mapping is shown in the following table:

Button inputs employ internal pull-up resistors provided by the Raspberry Pi GPIO controller; thus simplifying external wiring requirements and ensuring reliable signal detection.

LED outputs require current-limiting resistors to prevent excessive current flow that could damage the GPIO pins or LED components. The recommended resistor values range from 220Ω to 470Ω depending on the LED specifications and desired brightness level. The GPIO outputs operate at 3.3V logic levels with a maximum current capacity of 16mA per pin; necessitating proper current limiting for reliable operation.

Note

The current controller is implemented as a proof of concept for demonstration purposes. For real-world applications, a new system must be built using the results of testing this proof of concept.

CHAPTER

8

FINAL RESULTS

This chapter presents the results obtained from simulation runs, performance analysis, and validation testing conducted during the development and evaluation of the project.

To restate the purpose of this project, it is this: to create a residential smart grid application that would minimize excess energy and grid dependence using a smart algorithm. The results are as follows.

8.1 System Design Validation

The system uses three models as the base of its simulation: the ADSR Envelope for device modeling, pvwatts for inverter modeling, and pvlib for solar panels modeling. The results of validating the models are as follows:

ADSR Device Modeling: The ADSR envelope model demonstrated good correlation with real power consumption data extracted from scientific literature. Below is a comparison between data collected from a real refrigerator versus that collected from the ADSR envelope for a refrigerator.

RSGP ADSR Refrigerator Model vs Real Load Profile Comparison

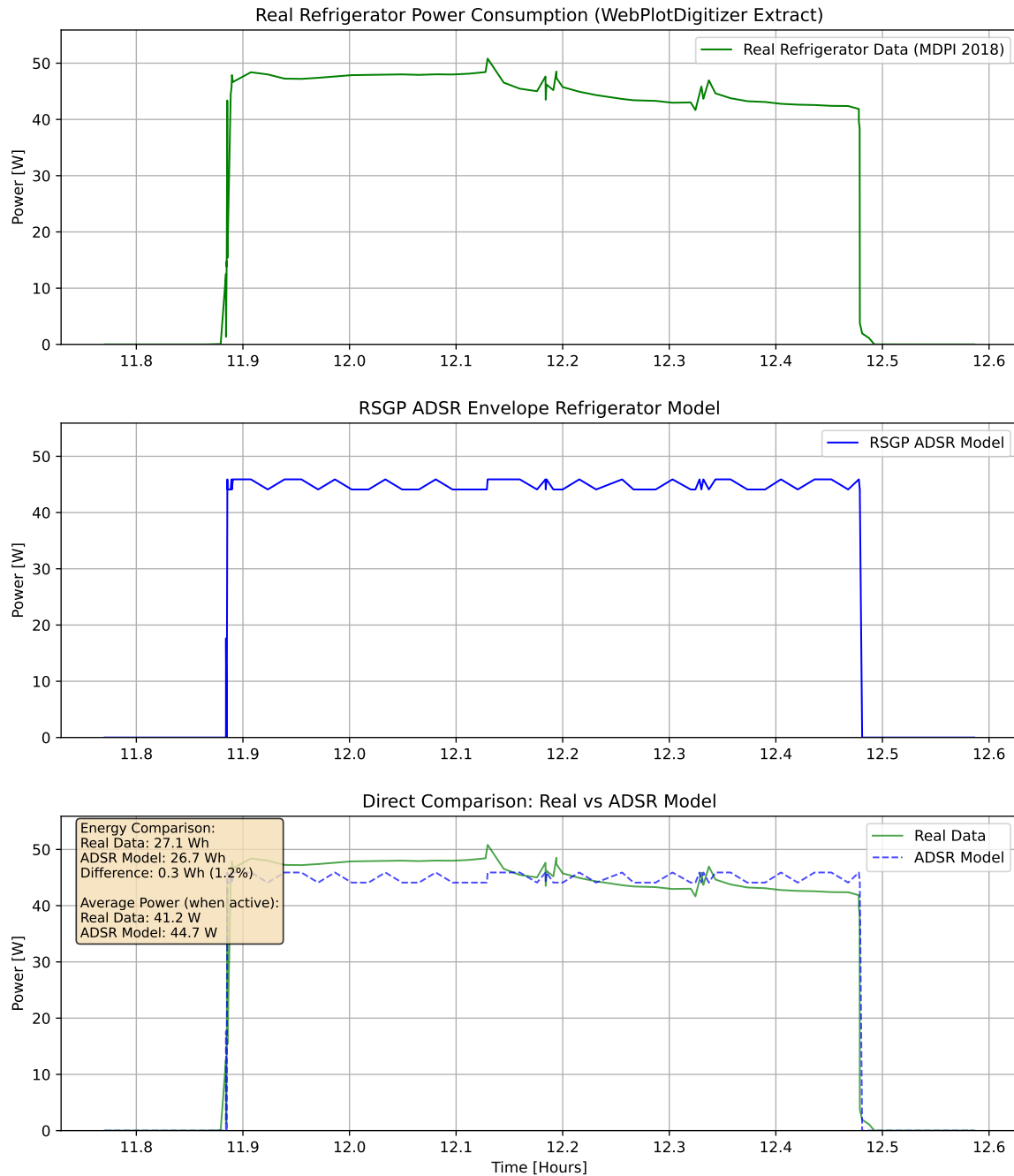


Fig. 8.1: RSGP ADSR refrigerator model validation against real power consumption data

Solar System Modeling Accuracy: Due to usage of the standard libraries pvlib and pwwatts, and the usage of a dataset from the National Solar Radiation Database, the solar system simulation, including the inverter and panels, closely match reality.

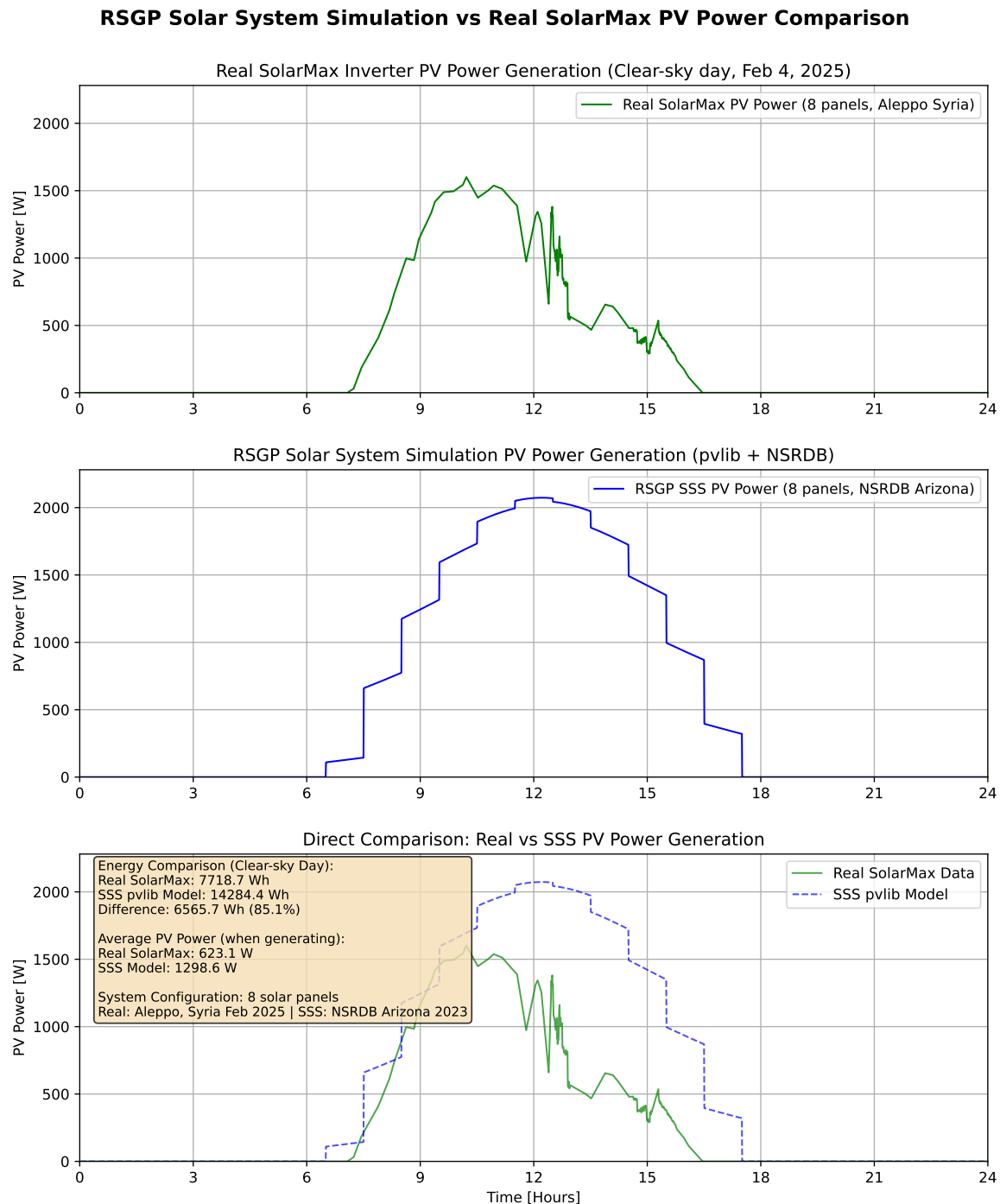


Fig. 8.2: RSGP solar system simulation validation against real SolarMax inverter data

Note

The SSS data represents theoretical maximum power generation capacity calculated by pvlib based on solar irradiance and panel specifications, while the real inverter data shows practical power extraction limited by actual load demand. Solar panels only deliver the power that is actively consumed by the connected loads, which explains potential differences between

theoretical generation capacity and measured inverter output.

8.2 Simulation Results and Analysis

The smart power management algorithm implemented in RSGP demonstrates adaptive learning capabilities through dynamic weight adjustment and virtual battery allocation. The system continuously optimizes power distribution among houses based on load patterns and solar generation availability, leading to improved grid independence and efficient energy utilization.

Note

Further validation of the project can happen only through a real-world prototype and data from real usage of the project.

CHAPTER

9

CONCLUSIONS AND SUGGESTIONS

9.1 Conclusions

The Residential Smart Grid Prototype serves as both a practical solution to a real problem and a research tool that collects data and visualizes it to facilitate research and development of its algorithms. Using standard libraries like pvlib and pwwatts and international data sources like NSRDB, and implementing smart algorithms that are based on statistics, RSGP has demonstrated great results and showed potential for real world application to fairly distribute the generated solar power without wasting resources and efficiently manage the shared battery to meet each house's demand.

9.2 Suggestions

- **Machine Learning Integration:** Machine learning algorithms could greatly enhance power management when implemented for solar power generation prediction, demand forecasting, and virtual battery allocation optimizations.
- **Enhanced Visualization Capabilities:** Research productivity could improve through more sophisticated visualization tools, including real-time performance dashboards.
- **Commercial Building Integration:** RSGP could see broader application if extended to include commercial and industrial loads.
- **Economic Modeling and Market Simulation:** The current implementation of demand response incentive could improve through adding economic models for electricity pricing and energy trading.

These suggestions are based on what we currently see RSGP to be lacking. However, employing RSGP in the real world could provide insights into the human response to it, and thus provide feedback on the weak points of RSGP and allow for a clearer view on how to further develop

the project. As such, the current priority of the project should be real-world prototyping, after which the aforementioned suggestions should be re-evaluated and possibly integrated.

THANKS AND APPRECIATION

We extend our deepest gratitude to all those who supported and guided us throughout this journey, making the Residential Smart Grid Prototype a meaningful and enriching experience.

First and foremost, we express our sincere appreciation to **Dr. Fadi Farha**, our project supervisor, whose invaluable guidance, expertise, and unwavering support made this project possible.

We are grateful to the **Faculty of Informatics Engineering** at the **University of Aleppo** for providing the academic environment for conducting this research.

Our appreciation extends to the faculty members of the **Department of Systems and Computer Networks** who contributed to our education that enabled us to understand and implement system architecture principles that form the backbone of this project.

We acknowledge the invaluable contributions of open-source communities and research institutions whose tools and datasets made this work possible. Special recognition goes to the developers of **pvlb**, **pwwatts**, and the **National Renewable Energy Laboratory (NREL)**. We thank the **National Solar Radiation Database (NSRDB)** team for making comprehensive solar irradiance data freely available for research purposes.

REFERENCES

- [1] **National Renewable Energy Laboratory (NREL)**
PVLib Python Photovoltaic Modeling Library, pvlib-python.readthedocs.io
- [2] **National Renewable Energy Laboratory (NREL)**
PVWatts Calculator and API, pvwatts.nrel.gov
- [3] **National Solar Radiation Database (NSRDB)**
NSRDB Dataset 323705_33.45_-112.06_2023, Phoenix, Arizona Solar Irradiance Data
- [4] **Raspberry Pi Foundation**
Raspberry Pi GPIO Programming Documentation, [raspberrypi.org](https://www.raspberrypi.org)
- [5] **Raspberry Pi Foundation**
Raspberry Pi 5 GPIO Pinout Diagram, Raspberry Pi Documentation
- [6] **ResearchGate**
Components of Solar Radiation, [researchgate.net](https://www.researchgate.net)
- [7] **ScienceDirect Topics**
Solar Zenith Angle, [sciencedirect.com/topics](https://www.sciencedirect.com/topics)
- [8] **MDPI**
Real Refrigerator Power Consumption Data, *Energies* 2018, 11, 607
- [9] **SolarMax**
Inverter Data Log DataLog_929321041053717, SolarMax Systems
- [10] **Holmgren, W.F., Hansen, C.W., Mikofski, M.A.**
pvlib python: a python package for modeling solar energy systems, *Journal of Open Source Software*
- [11] **Irmen, I.**
Pyro5 - Python Remote Objects, pyro5.readthedocs.io

[12] Brandl, G.

Sphinx - Python Documentation Generator, sphinx-doc.org

[13] Rohatgi, A.

WebPlotDigitizer, automeris.io/WebPlotDigitizer

APPENDIX

A

DOCUMENTATION WORKFLOW

A.1 Introduction

The workflow implements a multi-phase approach that ensures content accuracy, visual consistency, and mathematical precision while supporting both web-based development documentation and academic publication formats. The system uses version control to allow for collaborative development and it separates the work into writing, editing, enrichment, and validation phases.

A.2 Collaborative Development Approach

The collaborative approach implements clear role separation to optimize the development process:

- **Content Authors:** Focus on technical accuracy and comprehensive coverage of system components without concern for formatting or presentation details.
- **Technical Editors:** Verify mathematical formulas and algorithm representations, and cross-reference accuracy between documentation and implementation.
- **Content Enrichers:** Add diagrams, plots, figures, and visual elements that support the text without modifying core technical information.
- **Language Reviewers:** Ensure consistency in technical terminology, writing style, and adherence to academic standards.

This separation enables parallel development where multiple team members can contribute simultaneously without conflicts or redundant effort.

A.3 Documentation Development Phases

The documentation development follows four phases that ensure content accuracy and presentation quality.

A.3.1 Phase 1: Initial Writing

The initial writing phase focuses on content creation without concern for formatting, diagrams, or presentation elements. Authors concentrate on technical accuracy, comprehensive coverage, and clear explanation of system components.

During this phase, content authors:

- Document system architecture, algorithms, and mathematical formulas
- Create comprehensive technical explanations using plain reStructuredText markup
- Create placeholders for diagrams and plots to be added later
- Focus on content completeness rather than presentation quality

The writing phase employs minimal markup to avoid distraction from content development.

A.3.2 Phase 2: Technical Editing

The technical editing phase goes through two rounds of reviews: one for mathematical formulas and one for algorithms.

Mathematical Formula Verification: All mathematical formulas undergo consistency checking between related equations.

$$P_{total} = POA_{irradiance} \times A_{panel} \times \eta_{panel} \times N_{panels}$$

Each formula receives validation for:

- **Implementation correlation:** Verification that documented formulations match actual code implementations
- **Cross-reference accuracy:** Ensuring mathematical symbols maintain consistent meaning throughout documentation
- **Consistence formatting:** Ensuring the mathematical formulas all follow the same conventions

Algorithm Representation Review: All algorithm descriptions undergo verification to ensure accurate representation of the actual implementation. The review process examines:

- Flowchart accuracy against actual code execution paths
- Decision tree completeness for all conditional branches
- Consistency between flowcharts in the conventions they use

A.3.3 Phase 3: Content Enrichment

The content enrichment phase adds visual elements, diagrams, and plots that support the text without modifying core technical information.

Mermaid Diagram Integration: System architecture and workflow diagrams use Mermaid markup for consistency and maintainability:

```
graph TD
  SSS[Solar System Simulation]
  PM[Power Management]
  HS[Houses Simulation]
  SSS <--Data--> PM <--Data--> HS
```

Diagrams undergo iterative refinement to ensure visual clarity and technical accuracy. The diagram development process includes validation against actual system architecture to prevent documentation drift from implementation reality.

Matplotlib Plot Generation: Technical visualizations use Python scripts that generate plots dynamically during documentation builds:

Plot generation scripts exist as separate Python files in `docs/_static/plots/` to maintain separation between content and visualization code. This approach enables independent development and testing of visualizations without affecting documentation content.

A.3.4 Phase 4: Final Validation

The final validation phase implements quality assurance to verify documentation integrity across all dimensions.

Flowchart Integrity Verification: All diagrams undergo systematic verification against actual system behavior through:

- Execution path tracing to ensure flowcharts represent actual code paths
- State transition validation for simulation components
- Interface verification between system components

Build Verification: Final validation includes verification of compilation success for all output formats to ensure that content renders correctly.

A.4 Sphinx

The documentation uses Sphinx as the primary documentation generator and reStructuredText as the primary markup language, as it proves extensibility for technical content:

```
.. math:: C_{vb,i} = \frac{C_{total}}{N} \times w_i

.. mermaid:: ../_static/diagrams/C5_pm_arch.mmd
   :caption: Power management architecture
```

The documentation integrates multiple Sphinx extensions that provide specialized functionality for technical documentation.

Autodoc Extension: Automatic API documentation generation from Python docstrings:

The autodoc extension maintains synchronization between code documentation and generated API reference, ensuring that documentation remains current with implementation changes.

Napoleon Extension: Support for Google and NumPy docstring formats that provide structured parameter and return value documentation within source code.

A.5 Automated API Reference Generation

All source code employs Google-style docstrings that provide structured, machine-readable documentation directly within the codebase:

```
def example_function(param1: int, param2: str) -> bool:
    """
    This is an example function demonstrating Google-style docstrings.

    Args:
        param1 (int): The first parameter.
        param2 (str): The second parameter.

    Returns:
        bool: The return value. True if successful, False otherwise.

    Raises:
        ValueError: If param1 is negative.
    """
    if param1 < 0:
        raise ValueError("param1 cannot be negative")
    return param2 == "success"
```

This structured approach provides API documentation directly within source code, ensuring that documentation matches implementation behavior.

A.6 Matplotlib Plot Integration

The documentation uses matplotlib for technical visualizations, mathematical concepts, and simulation results.

Technical plots exist as executable Python scripts within docs/_static/plots/ that generate visualizations during documentation builds:

```
import matplotlib.pyplot as plt
from rsgp.utils.nsrdb_data import nsrdb_data
```

(continues on next page)

(continued from previous page)

```
plt.plot(nsrd_data['Timestamp'], nsrd_data['GHI'], label='Global_
→Horizontal Irradiance')
plt.show()
```

This architecture provides several advantages:

- **Live data integration:** Plots use actual simulation data and code
- **Automatic updates:** Visualizations automatically reflect code changes
- **Reproducibility:** Plot generation is completely automated and reproducible
- **Version control:** Plot scripts maintain change history independently

A.6.1 Scientific Data Extraction and Validation

The documentation workflow includes extracting real-world data from scientific literature to validate simulation models against established research findings.

WebPlotDigitizer Integration: For model validation scenarios where raw data is not available, the workflow uses WebPlotDigitizer (<https://automeris.io/WebPlotDigitizer/>) to extract numerical data points from published scientific figures:

1. Identify peer-reviewed papers with relevant experimental data
2. Use WebPlotDigitizer to extract data points from figures
3. Save extracted data as CSV with proper source attribution
4. Create comparison plots showing model vs. real data
5. Include statistical analysis and validation metrics

A.6.2 Scenario-Based Plot Generation

Plots use a scenario-based system where events are defined with their time of arrival and the corresponding lambda function that changes the state of the simulation. The Python script runs the simulation and calls those lambda functions at the times specified in the scenario, then it plots the results for visualization. Events are not arbitrary and are meant to reflect typical usage patterns in residential houses:

```
SCENARIO_EVENTS = [
    # Morning routine (6-9 AM)
    (1.0, lambda hs: hs.get_house(0).get_device('WATER_HEATER').toggle_
→envelope_state(0, 1.0)),
    (1.5, lambda hs: hs.get_house(1).get_device('MICROWAVE').toggle_
→envelope_state(0, 1.5)),
    (2.0, lambda hs: hs.get_house(2).get_device('DISHWASHER').toggle_
→envelope_state(0, 2.0)),

    # Mid-day activity (10 AM - 2 PM)
```

(continues on next page)

(continued from previous page)

```
(3.0, lambda hs: hs.get_house(1).get_device('WASHING_MACHINE').
↪toggle_envelope_state(0, 3.0)),
(4.5, lambda hs: hs.get_house(2).get_device('HVAC').toggle_envelope_
↪state(0, 4.5)),

# Evening routine (6-10 PM)
(10.0, lambda hs: hs.get_house(2).get_device('MICROWAVE').toggle_
↪envelope_state(0, 10.0)),
(11.0, lambda hs: hs.get_house(0).get_device('WATER_HEATER').toggle_
↪envelope_state(0, 11.0)),

# Night shutdowns (10 PM - 12 AM)
(14.0, lambda hs: hs.get_house(0).get_device('HVAC').toggle_
↪envelope_state(0, 14.0)),
(15.0, lambda hs: hs.get_house(1).get_device('HVAC').toggle_
↪envelope_state(0, 15.0)),
]
```

Scenario files enable:

- **Realistic Load Patterns:** Multi-house coordination that mimics actual residential behavior
- **Sequencing:** Time-based event orchestration for morning, afternoon, and evening routines
- **System Stress Testing:** Peak demand scenarios and load distribution analysis
- **Demonstrations:** Clear examples of power management algorithm responses

A.6.3 Build System Integration

Matplotlib plots integrate with Sphinx through the `matplotlib.sphinxext.plot_directive`:

```
.. plot:: _static/plots/C3_adsr_envelope_basic.py
   :align: center
```

ADSR envelope showing Attack-Decay-Sustain-Release phases

The plot directive executes Python scripts during documentation builds and embeds the generated visualizations directly into the output.

APPENDIX

B

NOTEBOOKS WORKFLOW

B.1 Introduction

The notebooks provide comprehensive data visualization through marimo-based web applications. Each notebook targets a specific aspect of the system data, including the NSRDB dataset. This approach ensures ease of examination as each notebook contains related data only. Additionally, data can be downloaded as csv, JSON, or parquet to be imported into 3rd party analysis tools for mathematical analysis.

B.2 marimo Framework Overview

The notebooks employ marimo, a Python notebook framework that stores notebooks as pure Python files rather than JSON, enabling proper collaborative development and version control. The framework automatically manages cell dependencies; when variables change in one cell, all dependent cells execute automatically to maintain consistency.

The reactive execution model ensures that notebook state remains synchronized with code changes. This eliminates the common problem of stale outputs that occurs when cells execute out of order in traditional notebook environments. Additionally, marimo provides built-in support for modern Python tooling, including code formatting and interactive components that enable interactive analysis.

The framework supports deployment of notebooks as standalone web applications. This enables sharing the notebooks with researchers who need not set up the development environment to analyze the data.

B.3 Notebook Applications

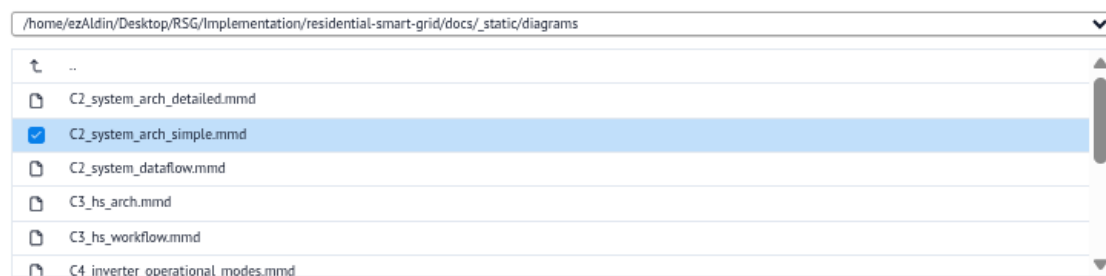
The project includes four specialized notebook applications.

B.3.1 Graph Editor

The graph editor notebook provides an interactive environment for creating and modifying Mermaid diagrams used throughout the project documentation. The live preview this notebook provides expedites the process of creating the diagrams.

Graphs Editor

Mermaid File Browser



1 file selected clear all
/home/ezAldin/Desktop/RSG/Implementation/residential-smart-grid/docs/_static/diagrams/C2_system_arch_simple.mmd

Mermaid Graph

Graph Rendering Graph Code Editor

Graph Rendering

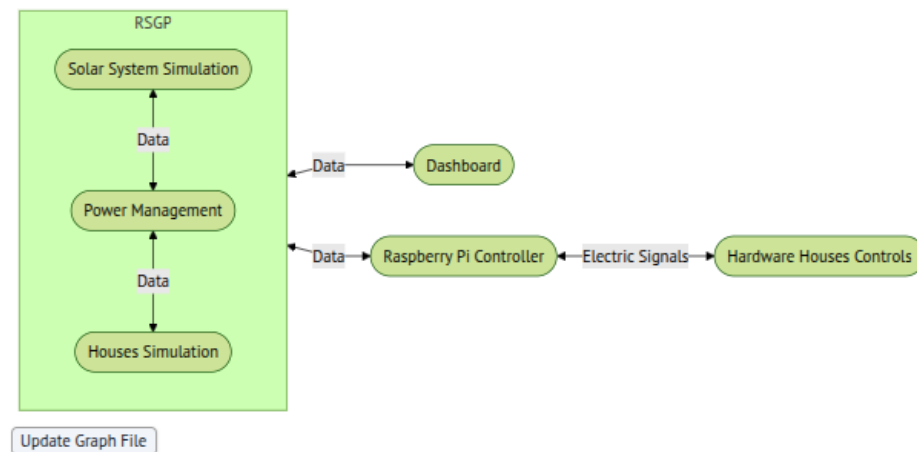


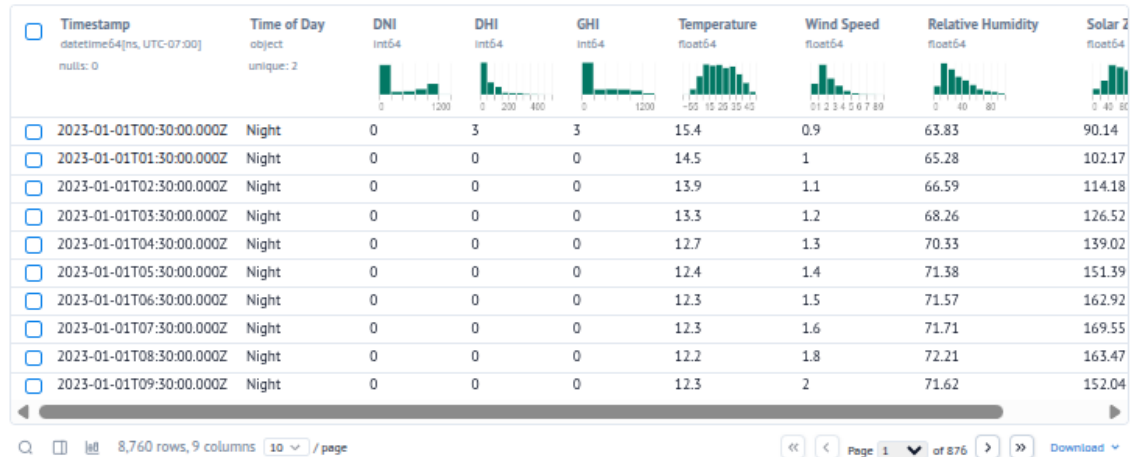
Fig. 2.1: Graph editor notebook interface

The live preview renders Mermaid diagrams immediately as code changes occur, providing instant visual feedback during the design process. This reactive behavior eliminates the traditional edit-compile-view cycle commonly associated with diagram creation. The application also allows for file persistence, allowing users to save modifications directly to the project files.

B.3.2 NSRDB Visualization

The NSRDB visualization notebook provides visualization for the National Solar Radiation Database weather dataset. This application enables researchers to examine solar irradiance patterns, atmospheric conditions, and seasonal variations that affect photovoltaic system performance.

NSRDB Data



NSRDB Chart

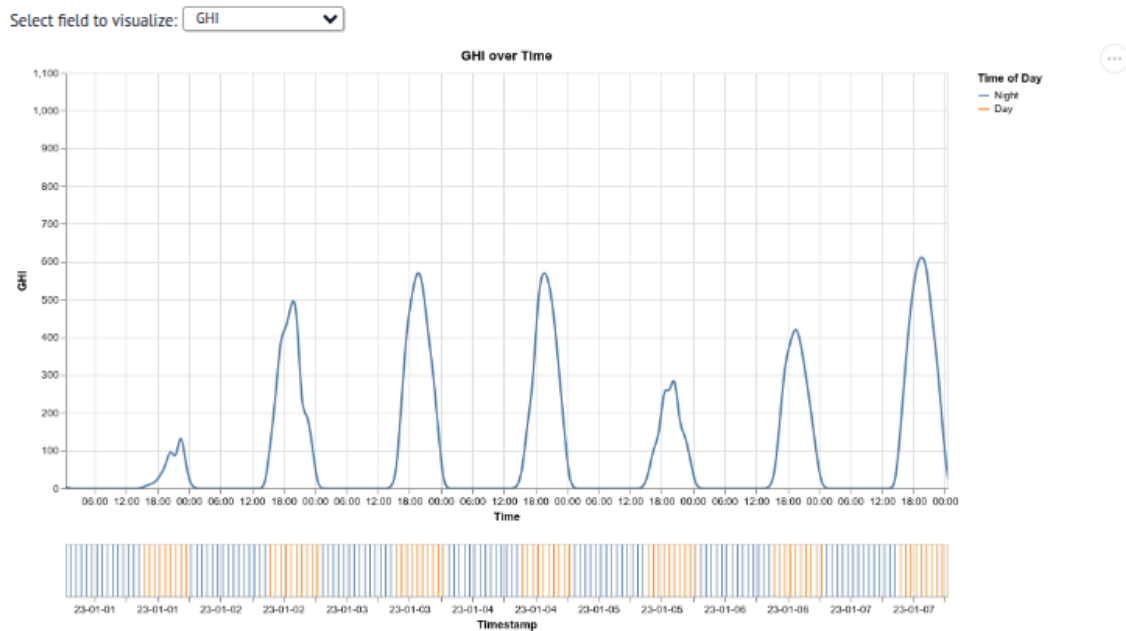


Fig. 2.2: NSRDB visualization notebook interface

The application visualizes CSV files containing hourly solar radiation measurements from Phoenix, Arizona. The visualization provides interactive field selection, allowing users to examine different atmospheric parameters including Direct Normal Irradiance (DNI), Diffuse Horizontal Irradiance (DHI), Global Horizontal Irradiance (GHI), temperature, and wind conditions. Users can focus on specific time periods and zoom in or out to cover them, supporting visualization of both short-term and long-term changes.

B.3.3 Inverter Logs Visualization

The inverter logs visualization notebook visualizes data from a real, physical inverter outside of the simulation. This application enables analysis of the simulation against real-world hardware performance, and thus enables validation of the theoretical models against actual device behavior.

Inverter Logs Visualization

Inverter Logs XLS File Browser

☒	DataLog_929321041053717_20250203-20250209.xls
☐	DataLog_929321041053717_20250217.xls

1 file selected clear all

/home/ezAldin/Desktop/RSG/Implementation/residential-smart-grid/docs/_static/data/inverter_logs/DataLog_929321041053717_20250203-20250209.xls

Inverter Logs Chart

Select field to visualize: PV1 input power (Watt)

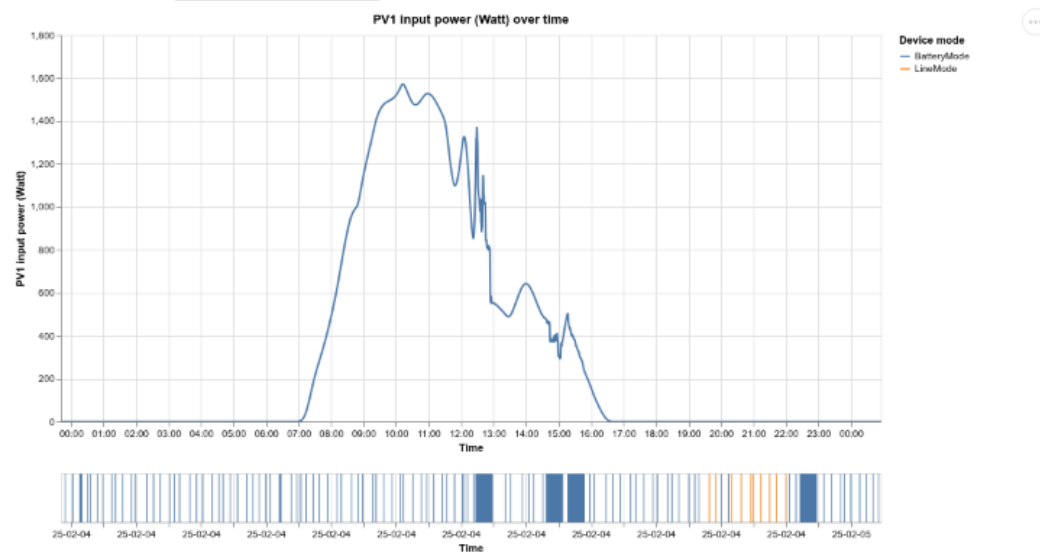


Fig. 2.3: Inverter logs visualization notebook interface

The application reads Excel files containing inverter telemetry data captured through the Watch-Power monitoring system. Users can select from the available fields to examine different aspects of the inverter operation. The fields include power generation levels, battery charge status, load consumption measurements, and the operational mode of the inverter.

Note

The inverter data represents actual hardware measurements rather than simulation outputs, providing ground truth for model validation and system performance assessment.

B.3.4 RSGP Logs Visualization

The RSGP logs visualization notebook visualizes the simulation output data generated by the residential smart grid simulation. This application enables researchers to examine system-wide behavior as well as the behavior of each individual component.

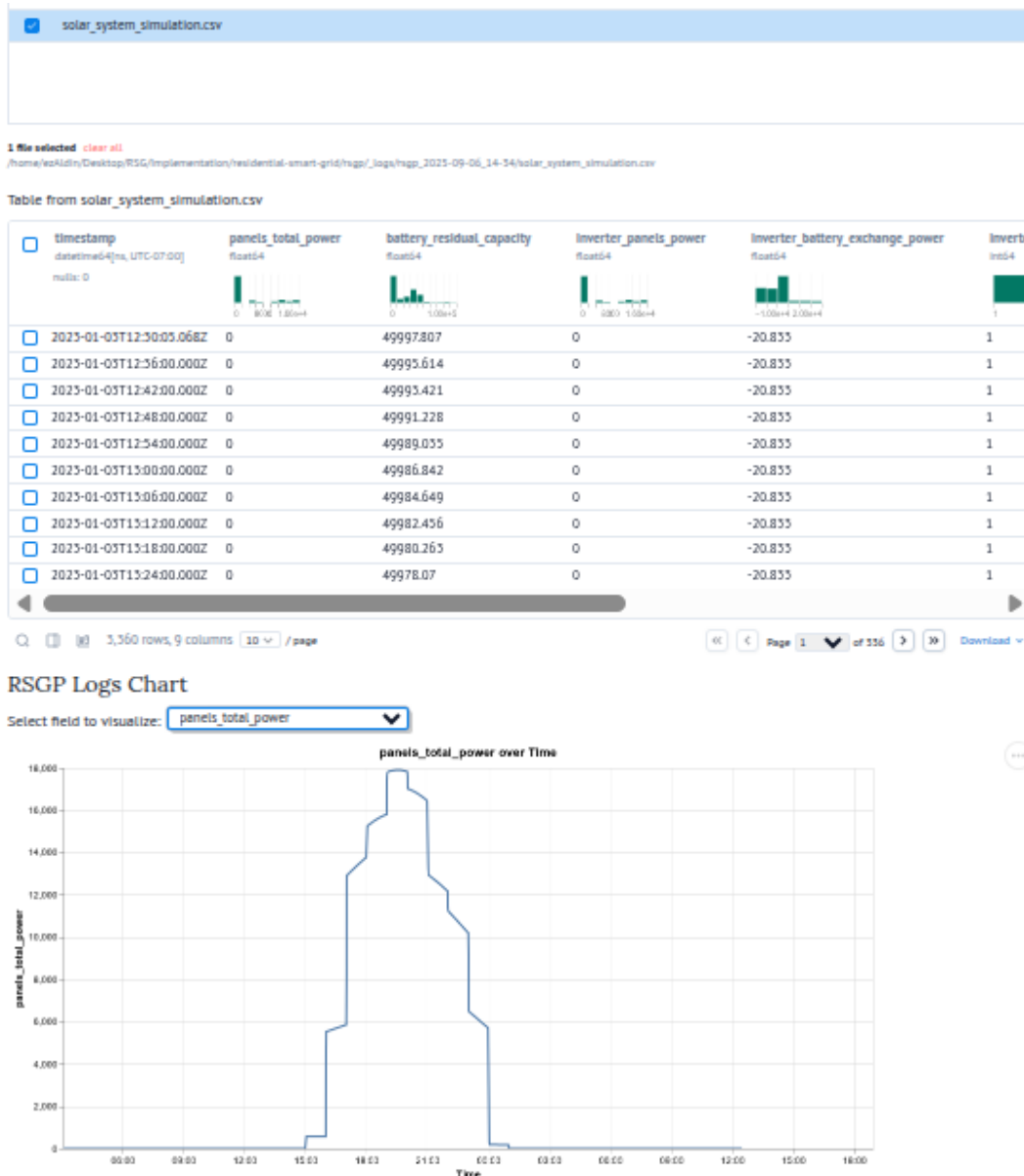


Fig. 2.4: RSGP logs visualization notebook interface

The application processes CSV files containing time-series data from simulation runs, including house-level load profiles, solar power generation, battery charge cycles, and power management decisions. The data processing pipeline converts timestamps to proper datetime objects and provides field selection to examine different system metrics.

The application provides essential feedback for power management algorithm development; researchers can examine how control decisions affect system stability, efficiency, and grid import/export. This supports the iterative refinement of power management strategies and validation of simulation accuracy.

B.4 Data Integration and Workflow

The notebook applications work together to provide comprehensive data visualization that spans from simulation results to actual hardware performance. This integrated approach enables correlation analysis across multiple data sources and validation of simulation accuracy against real-world measurements.

The workflow typically begins with NSRDB data analysis to understand environmental conditions during specific time periods. Researchers then examine simulation results using the RSGP logs visualization to understand how the system responds to those conditions. Finally, inverter logs provide validation data to confirm that physical hardware behavior aligns with simulation predictions.

Tip

The reactive nature of marimo notebooks enables researchers to modify analysis parameters in one notebook and immediately observe corresponding changes in dependent visualizations, supporting iterative hypothesis testing and parameter optimization.

The graph editor supports this analytical workflow by providing tools for creating visual documentation that communicates findings to broader audiences. Diagrams created through the graph editor integrate directly into project documentation, ensuring that visual representations remain synchronized with analytical discoveries.

APPENDIX

C

API REFERENCE

C.1 RSGP

C.1.1 Houses Simulation

Simulator

Houses simulator.

class `rsgp.houses_sim.simulator.HousesSimulator`

Bases: `object`

Houses simulator.

get_house(*idx*)

Get a specific house in the system.

Parameters

idx (*int*) – House index.

Returns

House object.

Return type

[House](#) (page 81)

get_houses()

Get houses in the system.

Returns

Houses in the system.

Return type

list[[House](#) (page 81)]

get_num_houses()

Get number of houses in the system.

Returns

Number of houses in the system.

Return type

int

get_system_load()

Get total load for the system.

Returns

Total load for the system.

Return type

float

get_time_sim_elapsed()

Get elapsed time in the simulation.

Returns

Elapsed time in the simulation.

Return type

float

houses: list[[House](#) (page 81)]

Houses in the system.

Type

list[[House](#) (page 81)]

is_running()

Check if the houses simulation is running.

Returns

True if the houses simulation is running, False otherwise.

Return type

bool

pause()

Pause the houses simulation.

Return type

None

resume()

Resume the houses simulation.

Return type

None

start(*dt*)

Start the houses simulation.

Parameters**dt** (*int*) – Simulation time step in milliseconds.**Return type**

None

summary()

Get a summary of the houses simulation.

Returns

Summary of the houses simulation.

Return type

str

system_load: float

Total load for the system.

Type

float

House

Houses simulation house.

class rsgp.houses_sim.house.**House**(*idx*)

Bases: object

House.

Parameters**idx** (*int*) – House index.**devices: dict[str, [DeviceClass](#) (page 83)]**

Devices in the house.

Type

dict[str, Device]

get_device(*dn*)

Get a specific device in the house.

Parameters**dn** (*str*) – Device name.**Returns**

Device object.

Return type

Device

get_devices()

Get devices in the house.

Returns

Devices in the house.

Return type

dict[str, Device]

get_idx()

Get house index.

Returns

House index.

Return type

int

get_load()

Get total load for the house.

Returns

Total load for the house.

Return type

float

get_load_line()

Get load line state.

Returns

Load line state (connected=1, disconnected=0).

Return type

bool

get_utility_line()

Get utility line state.

Returns

Utility line state (connected=1, disconnected=0).

Return type

bool

idx: int

House index.

Type

int

load_line: bool = True

Flag for load line state (connected=1, disconnected=0).

Type

bool

load_power: float

Total load for the house.

Type

float

set_load_line(*new_value*)

Set load line state.

Parameters

new_value (*bool*) – New load line state.

Return type

None

set_utility_line(*new_value*)

Set utility line state.

Parameters

new_value (*bool*) – New utility line state.

Return type

None

toggle_load_line()

Toggle the load line state.

Return type

None

toggle_utility_line()

Toggle the utility line state.

Return type

None

utility_exchange_power: float

Total power from/to the utility for the house (+ === export, - === import)

Type

float

utility_line: bool = True

Flag for utility line state (connected=1, disconnected=0).

Type

bool

Device

Houses simulation device class.

class rsgp.houses_sim.device.**DeviceClass**(*device_name*)

Bases: object

Device class.

Parameters

device_name (*str*)

calc_adsr_multiplier(*elapsed*, *envelope*)

Calculate the ADSR (Attack-Decay-Sustain-Release) multiplier.

Parameters

- **elapsed** (*float*) – Current elapsed time in the simulation.
- **envelope** (*tuple[*float*, *float*, *bool*]*) – A tuple containing the state toggle time, the load at the state toggle, and the active status of the envelope.

Returns

ADSR multiplier.

Return type

float

calc_load(*elapsed*)

Calculate the total load for the device class based on the ADSR envelope and wave type.

Parameters

elapsed (*float*) – Current elapsed time in the simulation.

Returns

Total load for the device class.

Return type

float

calc_wave_multiplier(*elapsed*)

Calculate the wave multiplier based on the wave type and elapsed time.

Parameters

elapsed (*float*) – Current elapsed time in the simulation.

Returns

Wave multiplier.

Return type

float

conf: [*DeviceConf*](#) (page 87)

Device class static configuration.

Type

[*DeviceConf*](#) (page 87)

filter_idle_envelopes(*elapsed*)

Filter out idle envelopes.

Parameters

elapsed (*float*) – Current elapsed time in the simulation.

Returns

List of non-idle envelopes.

Return type

list[tuple[float, float, bool]]

get_conf_base_watt()

Get device class base wattage.

Returns

Device class base wattage.

Return type

float

get_conf_max_count()

Get maximum count for the device class.

Returns

Maximum count for the device class.

Return type

int

get_conf_summary()

Get device class configuration summary.

Returns

Device class configuration summary.

Return type

str

get_envelopes()

Get device class envelopes.

Returns

Device class envelopes.

Return type

list[tuple[float, float, bool]]

get_load()

Get total load for the device class.

Returns

Total load for the device class.

Return type

float

get_name()

Get device name.

Returns

Device name.

Return type

str

load: float

Total load for all device instances.

Type

float

name: str

Device class name.

Type

str

set_envelopes(*envelopes*)

Set device class envelopes.

Parameters

envelopes (*list[tuple[float, float, bool]]*) – Device class envelopes.

Return type

None

toggle_envelope_state(*idx, elapsed*)

Toggle the state of a specific device instance envelope.

Parameters

- **idx** (*int*) – Index of the device instance envelope to toggle.
- **elapsed** (*float*) – Current elapsed time in the simulation.

Return type

None

Data

Houses simulation data and data types.

class rsgp.houses_sim.data.**ADSRConf**(*a, d, s, r, wt='none', wp=1, wa=0*)

Bases: object

ADSR (Attack-Decay-Sustain-Release) envelope configuration.

Parameters

- **a** (*float*)
- **d** (*float*)
- **s** (*float*)
- **r** (*float*)
- **wt** (*Literal['none', 'sine', 'square', 'random']*)

- **wp** (*float*)

- **wa** (*float*)

a: float

Attack time [sec].

Type

float

d: float

Decay time [sec].

Type

float

r: float

Release time [sec].

Type

float

s: float

Sustain level multiplier.

Type

float

wa: float = 0

Wave amplitude multiplier.

Type

float

wp: float = 1

Wave period [sec].

Type

float

wt: Literal['none', 'sine', 'square', 'random'] = 'none'

Wave type.

Type

Literal['none', 'sine', 'square', 'random']

class rsgp.houses_sim.data.**DeviceConf**(*base_watt, max_count, adsr*)

Bases: object

Device configuration.

Parameters

- **base_watt** (*float*)

- **max_count** (*int*)

- **adsr** ([ADSRConf](#) (page 86))

adsr: [ADSRConf](#) (page 86)

Device ADSR configuration.

Type

[ADSRConf](#) (page 86)

base_watt: **float**

Device base wattage (maximum amplitude) [Watt].

Type

float

max_count: **int**

Device maximum count a regular house could have.

Type

int

class rsgp.houses_sim.data.**RegularDevices**(*values)

Bases: Enum

Regular household devices with their configurations.

DISHWASHER = **DeviceConf**(base_watt=1800, max_count=1,
adsr=**ADSRConf**(a=3, d=1.5, s=0.6, r=2, wt='sine', wp=1, wa=0.05))

Dish washer.

Type

[DeviceConf](#) (page 87)

DRYER = **DeviceConf**(base_watt=3000, max_count=1, adsr=**ADSRConf**(a=2,
d=1, s=0.9, r=2, wt='sine', wp=2, wa=0.03))

Dryer.

Type

[DeviceConf](#) (page 87)

HVAC = **DeviceConf**(base_watt=3500, max_count=1, adsr=**ADSRConf**(a=3,
d=2, s=0.8, r=0.5, wt='sine', wp=2, wa=0.07))

DeviceConf: HVAC (Heating, Ventilation, and Air Conditioning).

Type

[DeviceConf](#) (page 87)

Type

#

LED_LIGHT = **DeviceConf**(base_watt=10, max_count=10,
adsr=**ADSRConf**(a=0.01, d=2, s=0.8, r=0.01, wt='none', wp=1, wa=0))

LED Light.

Type

[DeviceConf](#) (page 87)

```
MICROWAVE = DeviceConf(base_watt=1100, max_count=1,  
adsr=ADSRConf(a=0.5, d=0.2, s=1, r=0.5, wt='random', wp=0.5,  
wa=0.05))
```

Micro-wave.

Type

DeviceConf (page 87)

```
REFRIGERATOR = DeviceConf(base_watt=50, max_count=2,  
adsr=ADSRConf(a=0.5, d=0.5, s=0.9, r=1, wt='square', wp=5,  
wa=0.02))
```

Refrigerator.

Type

DeviceConf (page 87)

```
TEST = DeviceConf(base_watt=1000, max_count=10,  
adsr=ADSRConf(a=3600, d=7200, s=0.8, r=1800, wt='none', wp=1,  
wa=0))
```

Test Device.

Type

DeviceConf (page 87)

```
TV = DeviceConf(base_watt=120, max_count=4, adsr=ADSRConf(a=1, d=1,  
s=0.8, r=1.5, wt='sine', wp=0.5, wa=0.1))
```

TV.

Type

DeviceConf (page 87)

```
WASHING_MACHINE = DeviceConf(base_watt=500, max_count=1,  
adsr=ADSRConf(a=2, d=2, s=0.7, r=0.5, wt='sine', wp=0.5, wa=0.04))
```

Washing machine.

Type

DeviceConf (page 87)

```
WATER_HEATER = DeviceConf(base_watt=4500, max_count=1,  
adsr=ADSRConf(a=1, d=0.5, s=0.9, r=1, wt='square', wp=5, wa=0.1))
```

Water Heater.

Type

DeviceConf (page 87)

C.1.2 Solar System Simulation

Simulator

Solar system simulator.

class rsgp.solar_system_sim.simulator.SolarSystemSimulator

Bases: object

Solar system simulator.

battery: [Battery](#) (page 92)

The battery.

Type

[Battery](#) (page 92)

inverter: [Inverter](#) (page 93)

The inverter.

Type

[Inverter](#) (page 93)

is_running()

Check if the solar system simulation is running.

Returns

True if the solar system simulation is running, False otherwise.

Return type

bool

panels: [Panels](#) (page 91)

The solar panels.

Type

[Panels](#) (page 91)

pause()

Pause the solar system simulation.

Return type

None

resume()

Resume the solar system simulation.

Return type

None

start(dt)

Start the solar system simulation.

Parameters

dt (*int*) – Simulation time step in milliseconds.

Return type

None

summary()

Get a summary of the solar system simulation.

Returns

Summary of the solar system simulation.

Return type

str

Panels

Solar system simulation panels.

class `rsgp.solar_system_sim.panels.Panels`

Bases: object

Panels.

calc_total_power(*timestamp*)

Calculate and return the total power that could be produced by the panels.

Parameters

timestamp (*datetime*) – The current timestamp in the same timezone as the NSRDB data.

Returns

The total power that could be produced by the panels [Watt].

Return type

float

conf: [*PanelsConf*](#) (page 97)

The panels configuration.

Type

[*PanelsConf*](#) (page 97)

pv_data: **Series**

The data row from the NSRDB for the current timestamp.

Type

Series

pv_loc: **Location**

The *pvlb* location info from the NSRDB meta data.

Type

Location

total_power: **float**

The theoretical total power produced by the panels [Watt].

Type

float

Battery

Solar system simulation battery.

class `rsgp.solar_system_sim.battery.Battery`

Bases: `object`

Battery.

charge(*power*, *time_interval*)

Charge the battery with a given power for a given time interval.

Parameters

- **power** (*float*) – The power to charge with [Watt].
- **time_interval** (*float*) – The time interval in seconds.

Returns

The actual power used for charging [Watt].

Return type

`float`

conf: [*BatteryConf*](#) (page 95)

The battery configuration.

Type

`BattConf`

discharge(*power*, *time_interval*)

Discharge the battery with a given power for a given time interval.

Parameters

- **power** (*float*) – The power to discharge with [Watt].
- **time_interval** (*float*) – The time interval in seconds.

Returns

The actual power discharged [Watt].

Return type

`float`

residual_capacity: `float`

Current charge level of the battery [Wh].

Type

`float`

Inverter

Solar system simulation inverter.

class `rsgp.solar_system_sim.inverter.Inverter`(*battery*, *panels*)

Bases: `object`

Inverter.

Parameters

- **battery** ([Battery](#) (page 92)) – The battery object.
- **panels** ([Panels](#) (page 91)) – The panels object.

ac_to_needed_dc(*p_ac_target*)

Convert a target AC power output to the required DC power input for the inverter.

Parameters

p_ac_target (*float*) – The target AC power output from the inverter [Watt].

Returns

The required DC power input to the inverter [Watt].

Return type

`float`

battery_exchange_power: `float`

The power exchanged with the battery (+ === charge, - === discharge) [Watt].

Type

`float`

conf: [InverterConf](#) (page 96)

The inverter configuration.

Type

[InverterConf](#) (page 96)

dc_to_ac(*p_dc*)

Convert DC power to AC power using the inverter's characteristics.

Parameters

p_dc (*float*) – DC power input to the inverter [Watt].

Returns

AC power output from the inverter [Watt].

Return type

`float`

load_line: `bool`

Flag for load line state (connected=1, disconnected=0).

Type

`bool`

load_power: float

Total load for the system.

Type

float

operate(*timestamp*, *dt_seconds*)

Calculate the power flow in the solar system for the given time interval.

It calculates the power flow in the solar system based on the current timestamp and time interval. It determines how power is distributed among the load, battery, solar panels, and utility grid, adhering to the inverter's operational mode and charge priority settings.

The power flow is calculated in DC, then converted to AC where needed, with the inverter's efficiency taken into account.

The `_S()`, `_B()`, and `_U()` helper functions encapsulate the logic for sourcing power from solar, battery, and utility respectively. They return the remaining power needs or surpluses after their operation.

The `battery_exchange_power` and `utility_exchange_power` are reset at the beginning of each `operate()` (page 94) call to ensure calculations are based on the current time step. The final `battery_exchange_power` is the sum of all battery charge/discharge events during the current operational cycle.

If there's any remaining `available_panels_dc_power` after meeting all demands and the utility line is connected, this surplus power is exported to the utility.

If `required_load_dc_power` is still greater than zero after all power sources have been considered, it means the load could not be fully met, and the `load_line` is disconnected.

Finally, if the battery needs charging from the utility based on the `charge_priority` settings, it charges the battery from the utility.

Parameters

- **timestamp** (*datetime*) – The current timestamp.
- **dt_seconds** (*float*) – The time interval in seconds.

Return type

None

panels_power: float

The power used by the solar panels [Watt].

Type

float

utility_exchange_power: float

Total power from/to the utility for the system (+ === export, - === import).

Type

float

utility_line: bool

Flag for utility line state (connected=1, disconnected=0).

Type

bool

Data

Solar system simulation data types.

class rsgp.solar_system_sim.data.**BatteryConf**(*total_capacity, charge_efficiency, max_charge_power, max_discharge_power*)

Bases: object

Battery configuration data type.

Parameters

- **total_capacity** (*float*)
- **charge_efficiency** (*float*)
- **max_charge_power** (*float*)
- **max_discharge_power** (*float*)

charge_efficiency: float

Efficiency of charging [%].

Type

float

max_charge_power: float

Maximum charge rate [Watt].

Type

float

max_discharge_power: float

Maximum discharge rate [Watt].

Type

float

total_capacity: float

Total capacity of the battery [Wh].

Type

float

class rsgp.solar_system_sim.data.**ChargePriority**(**values*)

Bases: Enum

Inverter charge priority.

`SOLAR_FIRST = 'Solar First'`

`SOLAR_ONLY = 'Solar Only'`

`UTILITY_AND_SOLAR = 'Utility + Solar'`

```
class rsgp.solar_system_sim.data.InverterConf(paco, pdco, pnt, eta_inv_nom,  
                                              eta_inv_ref, eta_inv_ovr, mode,  
                                              charge_priority)
```

Bases: object

Inverter configuration data type.

Parameters

- **paco** (*float*)
- **pdco** (*float*)
- **pnt** (*float*)
- **eta_inv_nom** (*float*)
- **eta_inv_ref** (*float*)
- **eta_inv_ovr** (*float*)
- **mode** ([InverterMode](#) (page 97))
- **charge_priority** ([ChargePriority](#) (page 95))

charge_priority: [ChargePriority](#) (page 95)

The current charge priority of the inverter.

Type

[ChargePriority](#) (page 95)

eta_inv_nom: float

Nominal efficiency of the inverter [%].

Type

float

eta_inv_ovr: float

Overall efficiency of the inverter [%].

Type

float

eta_inv_ref: float

Reference efficiency of the inverter [%].

Type

float

mode: [*InverterMode*](#) (page 97)

The current operating mode of the inverter.

Type

[*InverterMode*](#) (page 97)

paco: **float**

AC Power rating of the inverter [Watt].

Type

float

pdco: **float**

pnt: **float**

AC power consumed by the inverter at night [Watt].

Type

float

class rsgp.solar_system_sim.data.**InverterMode**(*values)

Bases: Enum

Inverter operating modes.

SBU = 'SBU'

SUB = 'SUB'

USB = 'USB'

class rsgp.solar_system_sim.data.**PanelsConf**(num_panels, panel_area,
panel_efficiency)

Bases: object

Panels configuration data type.

Parameters

- **num_panels** (*int*)
- **panel_area** (*float*)
- **panel_efficiency** (*float*)

num_panels: **int**

Panels count.

Type

int

panel_area: **float**

Panel area [m²].

Type

float

panel_efficiency: float

Panel efficiency multiplier.

Type

float

C.1.3 Power Mangament

Manager

Power management manager.

class rsgp.power_mng.manager.**PowerManager**(*houses_sim, solar_system_sim*)

Bases: object

Power manager.

Parameters

- **houses_sim** ([HousesSimulator](#) (page 79)) – Houses simulator.
- **solar_system_sim** ([SolarSystemSimulator](#) (page 89)) – Solar system simulator.

calc_houses_vb_usage(*houses_load_powers, inverter_panels_power, inverter_utility_exchange_power*)

Calculate virtual batteries usage of each house.

Parameters

- **houses_load_powers** (*ndarray*) – The load powers of the houses.
- **inverter_panels_power** (*float*) – The power from the panels.
- **inverter_utility_exchange_power** (*float*) – The power exchanged with the utility.

Returns

The virtual batteries usage of each house.

Return type

ndarray

charge_all_vb(*charging_power*)

Charge all virtual batteries by the given charging power.

Parameters

charging_power (*float*) – The power to charge with [Watt].

Return type

None

discharge_all_vb(*discharging_power, discharging_weights*)

Discharge all virtual batteries by the given discharging power given their discharging weights.

Parameters

- **discharging_power** (*float*) – The power to discharge with [Watt].
- **discharging_weights** (*ndarray*) – The weights for each virtual battery.

Returns

A boolean array indicating whether the usage was met fully for each virtual battery.

Return type

ndarray

is_running()

Check if the power manager is running.

Returns

True if the power manager is running, False otherwise.

Return type

bool

learn_step(houses_load_powers)

Learn a step for the virtual batteries.

Parameters

houses_load_powers (*ndarray*) – The load powers of the houses.

Returns

The total excess capacity.

Return type

float

pause()

Pause the power manager.

Return type

None

resume()

Resume the power manager.

Return type

None

start(dt=None)

Start the power manager.

Parameters

dt (*int*) – Simulation time step in milliseconds.

Return type

None

summary()

Get a summary of the houses simulation.

Returns

Summary of the houses simulation.

Return type

str

virtual_batteries: list[VirtualBattery (page 100)]

list of virtual batteries.

Type

list[VirtualBattery (page 100)]

Virtual Battery

Power management virtual battery.

```
class rsgp.power_mng.virtual_battery.VirtualBattery(idx, init_total_capacity,  
                                                    init_residual_capacity,  
                                                    charge_efficiency,  
                                                    min_weight, max_weight)
```

Bases: object

Virtual battery.

Parameters

- **idx** (*int*) – The index of the virtual battery.
- **init_total_capacity** (*float*) – The initial total capacity of the virtual battery [Wh].
- **init_residual_capacity** (*float*) – The initial residual capacity of the virtual battery [Wh].
- **charge_efficiency** (*float*)
- **min_weight** (*float*)
- **max_weight** (*float*)

adjust(*amount*)

Adjust the weight of the virtual battery.

Parameters

amount (*float*) – The amount to adjust the weight by.

Returns

The excess capacity after adjustment.

Return type

float

charge(*power*, *time_interval*)

Charge the virtual battery with a given power for a given time interval.

Parameters

- **power** (*float*) – The power to charge with [Watt].
- **time_interval** (*float*) – The time interval in seconds.

Returns

The actual power used for charging [Watt].

Return type

float

charge_efficiency: float

The charge efficiency of the virtual battery.

Type

float

discharge(*power*, *time_interval*)

Discharge the virtual battery with a given power for a given time interval.

Parameters

- **power** (*float*) – The power to discharge with [Watt].
- **time_interval** (*float*) – The time interval in seconds.

Returns

The actual power discharged [Watt].

Return type

float

residual_capacity: float

The residual capacity of the virtual battery [Wh].

Type

float

total_capacity: float

The total capacity of the virtual battery [Wh].

Type

float

weight: float

The weight of the virtual battery.

Type

float

C.1.4 RSGP Utilities

Time Simulation

RSGP Time simulator.

class `rsgp.utils.time_sim.TimeSimulator`

Bases: `object`

Time simulator for RSGP.

get_elapsed()

Get the elapsed time in seconds.

Returns

The elapsed time in seconds.

Return type

`float`

get_timestamp(*elapsed=None*)

Get the current timestamp.

Parameters

elapsed (*float, optional*) – The elapsed time in seconds. Defaults to `None`, if so, it will be calculated automatically.

Returns

The current timestamp.

Return type

`datetime`

pause()

Pause the time simulation.

Return type

`None`

resume()

Resume the time simulation.

Return type

`None`

start()

Start the time simulation.

Return type

`None`

`rsgp.utils.time_sim.time_sim = <rsgp.utils.time_sim.TimeSimulator object>`

Global time simulator instance.

Type*TimeSimulator* (page 102)**NSRDB Data**

NSRDB data handlers.

`rsgp.utils.nsrdb_data.get_nsrdb_data()`

Read NSRDB dataset, manipulate it, and return it as DataFrames.

Returns

NSRDB metadata, and NSRDB data.

Return type

tuple[DataFrame, DataFrame]

`rsgp.utils.nsrdb_data.get_nsrdb_location(nsrdb_meta)`

Get location basic info based on NSRDB dataset metadata.

Parameters**nsrdb_meta** (*DataFrame*) – *nsrdb_meta* from `get_nsrdb_data()` (page 103).**Returns**A dictionary of location info (*latitude*, *longitude*, *timezone*).**Return type**

dict[str, Any]

```

rsgp.utils.nsrdb_data.nsrdb_data = Timestamp Time of Day DNI DHI GHI
Temperature Wind Speed Relative Humidity Solar Zenith Angle 0 2022-12-31
17:30:00-07:00 Night 0 3 3 15.4 0.9 63.83 90.14 1 2022-12-31
18:30:00-07:00 Night 0 0 0 14.5 1.0 65.28 102.17 2 2022-12-31
19:30:00-07:00 Night 0 0 0 13.9 1.1 66.59 114.18 3 2022-12-31
20:30:00-07:00 Night 0 0 0 13.3 1.2 68.26 126.52 4 2022-12-31
21:30:00-07:00 Night 0 0 0 12.7 1.3 70.33 139.02 ... ..
... .. 8755 2023-12-31 12:30:00-07:00 Day 384 235 447 17.8 1.3
41.72 56.51 8756 2023-12-31 13:30:00-07:00 Day 520 183 457 17.6 1.1
41.19 58.21 8757 2023-12-31 14:30:00-07:00 Day 71 207 239 16.9 0.8 42.41
63.17 8758 2023-12-31 15:30:00-07:00 Day 17 136 142 15.9 0.5 47.32 70.69
8759 2023-12-31 16:30:00-07:00 Day 99 71 88 15.3 0.4 55.11 80.03 [8760
rows x 9 columns]

```

NSRDB meta data and data.

Type

tuple[DataFrame, DataFrame]

```

rsgp.utils.nsrdb_data.nsrdb_location = {'latitude': 33.45, 'longitude':
-112.06, 'timezone': datetime.timezone(datetime.timedelta(days=-1,
seconds=61200), 'America/Phoenix')}

```

NSRDB location.

Type

dict[str, Any]

```
rsgp.utils.nsrdb_data.nsrdb_meta = Source Location ID City State ...  
Fill Flag 4 Fill Flag 5 Surface Albedo Units Version 0 NSRDB 323705 - -  
... Missing Cloud Properties Rayleigh Violation NaN 4.0.1 [1 rows x 46  
columns]
```

NSRDB meta data and data.

Type

tuple[DataFrame, DataFrame]

```
rsgp.utils.nsrdb_data.nsrdb_start_point: datetime =  
Timestamp('2022-12-31 17:30:00-0700', tz='UTC-07:00')
```

NSRDB start point.

Type

datetime

Remote Object

Remote object interface for RSGP via Pyro5.

```
class rsgp.utils.remote_object.RemoteObjectServer(houses_sim,  
                                                    solar_system_sim,  
                                                    power_manager)
```

Bases: object

Remote object server for RSGP via Pyro5.

Parameters

- **houses_sim** ([HousesSimulator](#) (page 79)) – *HousesSimulator* instance.
- **solar_system_sim** ([SolarSystemSimulator](#) (page 89)) – *SolarSystemSimulator* instance.
- **power_manager** ([PowerManager](#) (page 98)) – *PowerManager* instance.

daemon: **Daemon**

The Pyro5 daemon.

Type

Daemon

is_running()

Check if the remote object server is running.

Returns

True if the server is running, False otherwise.

Return type

bool

start()

Start the remote object server.

Return type

None

stop()

Stop the remote object server.

Return type

None

thread: Thread

The thread for the daemon.

Type

Thread

Logger

RSGP logger.

`rsgp.utils.logger.configure_logger(name='log')`

Configure and return a logger with console and optional file handlers.

Parameters

name (*str*) – The name of the logger.

Returns

The configured logger.

Return type

Logger

`rsgp.utils.logger.logger = <Logger RSGP (INFO)>`

Global RSGP logger instance.

Type

Logger

Helpers

RSGP helpers.

`rsgp.utils.helpers.find_nearest_timestamp_row(dataframe, target_timestamp)`

Find datetime row with timestamp closest to the target timestamp.

Parameters

- **dataframe** (*DataFrame*) – pandas dataframe with *Timestamp* column.
- **target_timestamp** (*Timestamp*) – Timestamp to search for.

Returns

The row from df with nearest timestamp.

Return type

Series

`rsgp.utils.helpers.log_record_into_csv(csv_file_path, / (Positional-only parameter separator (PEP 570)), **record)`

Log a record into a CSV file.

Parameters

- **csv_file_path** (*str*) – Path to the CSV file.
- **record** (*dict[str, Any]*) – The record to log. The keys are the column names and the values are the record values.

Return type

None

Decorators

RSGP decorators.

`rsgp.utils.decorators.log_excution(func)`

Decorator to log the execution of the function.

Parameters

func (*callable*) – The function to log.

Returns

The decorated function.

Return type

callable

`rsgp.utils.decorators.log_start_end_error(start_msg=None, finish_msg=None, error_msg=None)`

Decorator generator to log when the function start and end and handle errors.

Parameters

- **start_msg** (*str, optional*) – The message to log when the function start.
- **finish_msg** (*str, optional*) – The message to log when the function finish.
- **error_msg** (*str, optional*) – The message to log when the function raise an error.

Returns

The generated decorator.

Return type

callable

C.2 Other Packages

C.2.1 Dashboard

App

Dashboard application.

class `dashboard.app.DashboardApp`

Bases: `object`

Dashboard app to browse and control the system.

destroy()

Destroy tkinter app and release pyro proxies.

Return type

`None`

mainloop()

Start tkinter app mainloop.

Return type

`None`

Views

Dashboard views.

class `dashboard.views.HSSummaryTabView`(*root, f_parent, rsgp_hs*)

Bases: `object`

Houses simulation summery tab view.

Parameters

- **root** (*Tk*) – Tk window root.
- **f_parent** (*Frame*) – Parent fram, master of the main frame.
- **rsgp_hs** (*HousesSimulator* (page 79)) – Solar system simulator instance.

class `dashboard.views.HSTabView`(*root, f_parent, rsgp_hs*)

Bases: `object`

Houses simulation tab view.

Parameters

- **root** (*Tk*) – Tk window root.
- **f_parent** (*Frame*) – Parent fram, master of the main frame.

- **rsgp_hs** ([HousesSimulator](#) (page 79)) – Houses simulator instance.

open_hc_window(*idx*)

Open house controls window.

Parameters

idx (*int*) – House index.

Return type

None

class dashboard.views.**HouseControlsWindowView**(*root, idx, rsgp_hs, variables*)

Bases: object

House controls window view.

Parameters

- **root** (*Tk*) – Tk window root.
- **idx** (*int*) – House index.
- **houses_sim** ([HousesSimulator](#) (page 79)) – Houses simulator instance.
- **variables** (*dict[str, ttk.Variable]*) – Passed UI variables, including:
 - **total_load** (*StringVar*): Total house load label text.
 - **utility_line** (*IntVar*): Utility line status (0 or 1).
 - **load_line** (*IntVar*): Load line status (0 or 1).
- **rsgp_hs** ([HousesSimulator](#) (page 79))

idx: *int*

House index.

Type

int

class dashboard.views.**MainWindowView**(*root, rsgp_ts, rsgp_hs, rsgp_sss, rsgp_pm*)

Bases: object

Main window view.

Parameters

- **root** (*Tk*) – Tk window root.
- **rsgp_ts** ([TimeSimulator](#) (page 102)) – Time simulator instance.
- **rsgp_hs** ([HousesSimulator](#) (page 79)) – Houses simulator instance.
- **rsgp_sss** ([SolarSystemSimulator](#) (page 89)) – Solar system simulator instance.

- **rsgp_pm** ([PowerManager](#) (page 98)) – Power manager instance.

pause_sim()

Pause simulation.

Return type

None

resume_sim()

Resume simulation.

Return type

None

set_all_loads(*state*)

Turn all loads lines for all houses to *state*.

Parameters

state (*bool*) – True to turn on, False to turn off.

Return type

None

set_all_utilities(*state*)

Turn all utility lines for all houses to *state*.

Parameters

state (*bool*) – True to turn on, False to turn off.

Return type

None

toggle_sim()

Toggle simulation state.

Return type

None

class dashboard.views.**PMSummaryTabView**(*root*, *f_parent*, *rsgp_pm*)

Bases: object

Power management summery tab view.

Parameters

- **root** (*Tk*) – Tk window root.
- **f_parent** (*Frame*) – Parent fram, master of the main frame.
- **rsgp_pm** ([PowerManager](#) (page 98)) – Power manager instance.

class dashboard.views.**SSSSummaryTabView**(*root*, *f_parent*, *rsgp_sss*)

Bases: object

Solar system simulation summery tab view.

Parameters

- **root** (*Tk*) – Tk window root.
- **f_parent** (*Frame*) – Parent fram, master of the main frame.
- **rsgp_sss** (*SolarSystemSimulator* (page 89)) – Solar system simulator instance.

Utilities

Dashboard utilities.

`dashboard.utils.build_scrollable_frame(f_parent, width=None, **kwargs)`

Build and return a scrollable frame.

Parameters

- **f_parent** (*Frame*) – Parent fram, master of the main frame.
- **width** (*int, optional*) – Minimum width for the scrollable frame. Defaults to None.
- **kwargs** (*dict[str, Any]*) – Keyword arguments to pass to scrollable frame when init.

Returns

The scrollable frame.

Return type

Frame

C.2.2 Raspberry Pi Controller

Hardware Configuration

Hardware configuration for Raspberry Pi GPIO controller.

`class rasp_controller.hardware_config.DeviceType(*values)`

Bases: Enum

Device type enumeration.

HVAC = 'HVAC'

LOAD_LINE = 'LOAD_LINE'

REFRIGERATOR = 'REFRIGERATOR'

UTILITY_LINE = 'UTILITY_LINE'

WATER_HEATER = 'WATER_HEATER'

`class rasp_controller.hardware_config.GPIOMapping(button_gpio, led_gpio,
house_id, device_type)`

Bases: object

GPIO pin mapping for a single control.

Parameters

- `button_gpio(int)`
- `led_gpio(int)`
- `house_id(int)`
- `device_type(DeviceType (page 110))`

`button_gpio: int`

`property control_name: str`

Generate control name for identification.

`device_type: DeviceType (page 110)`

`house_id: int`

`led_gpio: int`

`class rasp_controller.hardware_config.HardwareConfig`

Bases: `object`

Hardware configuration manager for Raspberry Pi controller.

```
GPIO_MAPPINGS: List[GPIOMapping (page 110)] =
[GPIOMapping(button_gpio=2, led_gpio=15, house_id=1,
device_type=<DeviceType.REFRIGERATOR: 'REFRIGERATOR'>),
GPIOMapping(button_gpio=3, led_gpio=16, house_id=1,
device_type=<DeviceType.HVAC: 'HVAC'>), GPIOMapping(button_gpio=4,
led_gpio=17, house_id=1, device_type=<DeviceType.WATER_HEATER:
'WATER_HEATER'>), GPIOMapping(button_gpio=5, led_gpio=18,
house_id=1, device_type=<DeviceType.LOAD_LINE: 'LOAD_LINE'>),
GPIOMapping(button_gpio=6, led_gpio=19, house_id=2,
device_type=<DeviceType.REFRIGERATOR: 'REFRIGERATOR'>),
GPIOMapping(button_gpio=7, led_gpio=20, house_id=2,
device_type=<DeviceType.HVAC: 'HVAC'>), GPIOMapping(button_gpio=8,
led_gpio=21, house_id=2, device_type=<DeviceType.WATER_HEATER:
'WATER_HEATER'>), GPIOMapping(button_gpio=9, led_gpio=22,
house_id=2, device_type=<DeviceType.LOAD_LINE: 'LOAD_LINE'>),
GPIOMapping(button_gpio=10, led_gpio=23, house_id=3,
device_type=<DeviceType.REFRIGERATOR: 'REFRIGERATOR'>),
GPIOMapping(button_gpio=11, led_gpio=24, house_id=3,
device_type=<DeviceType.HVAC: 'HVAC'>), GPIOMapping(button_gpio=12,
led_gpio=25, house_id=3, device_type=<DeviceType.WATER_HEATER:
'WATER_HEATER'>), GPIOMapping(button_gpio=13, led_gpio=26,
house_id=3, device_type=<DeviceType.LOAD_LINE: 'LOAD_LINE'>),
GPIOMapping(button_gpio=14, led_gpio=27, house_id=0,
device_type=<DeviceType.UTILITY_LINE: 'UTILITY_LINE'>)]
```

`classmethod get_button_pins()`

Get all button GPIO pins.

Return type

List[int]

classmethod `get_led_pins()`

Get all LED GPIO pins.

Return type

List[int]

classmethod `get_mapping_by_button(button_gpio)`

Get GPIO mapping by button pin.

Parameters

button_gpio (*int*)

Return type

[GPIONMapping](#) (page 110)

classmethod `get_mappings_by_house(house_id)`

Get all GPIO mappings for a specific house.

Parameters

house_id (*int*)

Return type

List[[GPIONMapping](#) (page 110)]

classmethod `get_utility_mapping()`

Get utility line GPIO mapping.

Return type

[GPIONMapping](#) (page 110)

GPIO Controller

GPIO controller for Raspberry Pi hardware interface.

class `rasp_controller.gpio_controller.GPIOController(rsgp_hs)`

Bases: `object`

GPIO controller for Raspberry Pi hardware interface.

Manages physical buttons and LEDs for controlling the smart grid simulation. Synchronizes hardware state with RSGP simulation components.

Parameters

rsgp_hs ([HousesSimulator](#) (page 79)) – Houses simulator instance for remote control.

mainloop (*dt*)

Main update loop for GPIO monitoring.

Parameters

dt (*float*) – Update interval in seconds.

Return type

None

stop()

Stop GPIO controller and cleanup.

Return type

None

